

Project handoff: MyBGRemover

This is an **existing Python + PySide6 desktop background remover app**, not a new project.

The user is beginner-level with coding, so responses should be:

- * step by step
- * explicit
- * practical
- * beginner-friendly
- * no skipped setup details

When updating code:

- * for `main_window.py`, if text needs to be replaced, provide **before and after**
- * for `main_window.py`, if text needs to be added, say the **exact function** it goes in
- * for all other files, provide the **full updated file**
- * clearly say which file(s) changed
- * explain what changed
- * explain how to run/test
- * make placeholders in code/commands very obvious

Current project status

We are around the later middle stages of the app. The app now has:

- * drag-and-drop loading
- * file list panel
- * preview panel

- * settings panel
- * batch processing flow
- * separated **Process** and **Export**
- * brush editing in preview
- * before/after switching
- * edit/preview modes
- * undo/redo working
- * export pipeline working again after thread crash fixes
- * multiple rembg model options added

Current workflow design

Important workflow decision

We changed the app so:

Process

- * only does preview/background removal
- * creates/updates the processed preview
- * does **not** export the final file

Export

- * creates the final full-resolution output file
- * can export even if the user did **not** process first
- * useful when the user only wants brush edits on the final result

This split was intentional and should remain.

Current UI action design

The settings panel was condensed to conserve space.

Instead of many buttons, the app now uses:

* Action combo:

* Process

* Export

* Target combo:

- * Checked

- * All

- * Run button

- * Cancel button

Important UI decision

We removed Selected from the settings panel because checked items already cover that use case.

The user specifically wanted:

- * no Selected target in the settings panel

- * Cancel only visible while a process/export job is running

That behavior is now in place.

Current model list in settings panel

The current model list in `ui/settings_panel.py` is:

- * u2net

- * u2netp

- * u2net_human_seg

- * isnet-general-use

- * isnet-anime

- * birefnet-general-lite

- * birefnet-general

- * birefnet-portrait

- * bria-rmbg

This was added by expanding the settings panel combo list only. No special engine branching was added for

these models.

Current known-good behavior

These are working now:

Preview / canvas behavior

- * preview image is centered and fitted
- * no more top-left cropped display issue
- * switching between files updates preview
- * before/after buttons work
- * preview mode and edit mode behavior were adjusted

Current mode semantics

The user wanted:

- * Preview Mode should show the **working image**, not the original
- * Before button shows the original image
- * After button shows the working image
- * Edit Mode edits the working image

That was implemented.

Brush editing

- * Erase works
- * Restore works
- * brush dragging works
- * undo works
- * redo works
- * edit mode stays active after a stroke
- * cycling through images preserves edit mode/view mode more reliably

Restore behavior

User specifically wanted:

- * restore should restore the ****original image****
- * brush editing should be usable ****before processing****
- * brush editing should still work ****after processing****
- * this was changed so restore uses the original preview image as its restore source

Export

- * export was separated from process
- * export can run without prior processing
- * preview edit deltas are used in export again
- * export pipeline is functional enough to continue from here

Thread crash

There was a major crash:

- * QThread: Destroyed while thread '' is still running
- * especially when using isnet-general-use and batch checked processing

This was fixed in `ui/main_window.py` by explicitly finalizing preview/export threads before moving to the next batch item.

This fix is important and should not be removed casually.

Important recent bug history

1. Preview panel issues that were fixed

Previously there were issues like:

- * preview only showed top-left area
- * preview did not update when multiple files were loaded
- * edit/preview mode toggles felt broken

These were fixed mostly in:

- * `ui/preview_canvas.py`
- * `ui/main_window.py`

2. Brush stroke resetting to preview mode

After editing, the app used to jump back to preview mode because:

- * `image_edited` emitted
- * file panel rebuilt
- * preview reloaded
- * canvas got reset to preview state

This was fixed by changing rebuild/reload behavior and preserving interaction/view mode across `select_item()`.

3. Undo/redo disabled after edits

This happened because preview reloads were wiping stacks.

That was fixed by avoiding unnecessary preview reloads on each edit.

4. Process/export redesign

Originally process also exported final output automatically.

This was changed so processing stops at processed

preview only.

5. Checked target batch thread crash

This happened especially with `isnet-general-use`.
The actual issue was not necessarily the model itself,
but thread lifecycle in batch processing.
This was fixed in `main_window.py`.

6. Settings panel indentation problems

There were several `TabError` / `IndentationError`
issues from manual pasting.
A clean, full rewritten `ui/settings_panel.py` fixed
that.

Files that were heavily modified recently

Main files involved:

- * `ui/main_window.py`
- * `ui/preview_canvas.py`
- * `ui/settings_panel.py`
- * `ui/file_panel.py`

Also involved in export pipeline:

- * `processing/mask_ops.py`
- * `processing/export_utils.py`
- * `workers/full_export_worker.py`
- * `workers/process_worker.py`
- * `processing/rembg_engine.py`

Current important logic in the app

```
### ui/main_window.py
```

Important current responsibilities:

- * route Action + Target + Run
- * batch process vs batch export
- * preserve preview/edit state across selection changes
- * keep checked/all workflows working
- * manage worker threads safely
- * do not auto-export after process
- * allow export without process first

```
### ui/preview_canvas.py
```

Important current responsibilities:

- * fit preview image to panel
- * brush erase / restore
- * before / after switching
- * edit / preview mode
- * undo / redo
- * emit image_edited
- * preserve working image logic

```
### ui/settings_panel.py
```

Important current responsibilities:

- * compact action controls
- * model list
- * tool controls
- * cancel hidden by default
- * target list is only Checked, All

```
### ui/file_panel.py
```

Important current state/status labels:

- * pending

- * processing
- * processed
- * edited
- * exported
- * error

The file panel wording was updated from older done language.

Current project structure

Relevant files:

- * main.py
- * ui/main_window.py
- * ui/preview_canvas.py
- * ui/settings_panel.py
- * ui/file_panel.py
- * core/models.py
- * core/state_manager.py
- * processing/rembg_engine.py
- * processing/mask_ops.py
- * processing/export_utils.py
- * workers/process_worker.py
- * workers/full_export_worker.py
- * workers/watch_folder_worker.py

Current user preferences / decisions that should be preserved

1. This is ****not**** a new project.
2. The user wants step-by-step beginner-friendly help.
3. The user prefers explicit file-by-file guidance.
4. The user wants full files for non- main_window.py files.

5. The user wants `main_window.py` changes as before/after or exact function placement.
6. Selected target in settings panel should stay removed.
7. Cancel button should only show during active jobs.
8. Process and Export must stay separate.
9. Export should be possible without running process first.
10. Preview Mode should show the working image, not the original.
11. Before should show original image.
12. Restore should restore original image content.
13. Checked-item workflow is important.
14. Space conservation in the settings panel matters.

Current likely next phase

The next recommended phase is ****Phase 24 — clean up and harden the UI****.

Best next steps:

- * hide or disable unfinished controls
- * add model descriptions/tooltips
- * reduce confusion in settings panel
- * optionally hide legacy redundant buttons in file panel if still visible
- * verify current model list behavior one by one
- * improve model failure messaging/logging

A strong next coding step would be:

- * simplify `ui/settings_panel.py`
- * hide unfinished controls like:
 - * softness
 - * opacity

- * spacing
- * color removal, if still not implemented
- * watch-folder automation, if not reliable yet
- * add helpful descriptions for model choices

Phases / workflow recap

This was the recommended order from earlier and we have already completed much of it:

Completed or mostly completed

- * Phase 18 — preview editing saves and persists
- * Phase 19 — undo/redo functional
- * Phase 20 — basic Erase/Restore
- * Phase 21 — brush dragging
- * Phase 22 — preview/edit mode cleanup
- * Phase 23 — export reliability improvements in progress / mostly working again

Current

- * moving into Phase 24 — UI cleanup and disabling unfinished controls

Later phases intentionally postponed

- * soft brush
- * opacity-based editing
- * spacing behavior
- * color removal mode
- * watch-folder automation restart/polish
- * extra UI polish
- * side-by-side comparison improvements

Important caution for future edits

Because `ui/main_window.py` has become large and stateful:

- * do not casually rewrite sections without checking batch flow and thread cleanup
- * do not reintroduce auto-export during process
- * do not break thread finalization for preview/export workers
- * do not reintroduce preview reloads on every edit, or undo/redo/edit mode stability may break again

Because the user is manually pasting code:

- * indentation errors are common
- * full-file rewrites are often safer than partial patching for `settings_panel.py`
- * for `main_window.py`, changes should be described very carefully

Current known issue area to keep an eye on

Even though the app is functioning again, these areas still need careful validation:

- * export parity for all models
- * export parity for pre-process edits vs post-process edits
- * BiRefNet and BR1A model stability/performance on this machine
- * whether all added models work equally well through current `rembg` runtime
- * whether some models need special user messaging if slow/heavy

If starting a new chat, the next good request would be

"Continue from the current MyBGRemover state. We already separated Process and Export, added the compact Action/Target/Run controls, added multiple rembg models, fixed preview/edit mode behavior, and fixed the batch thread crash. Next, help me clean up the settings panel by hiding unfinished controls and adding model descriptions/tooltips. Remember: full updated files for non-main_window files, and before/after or exact function placement for main_window.py."

Current clean summary

The app is in a good working state now:

- * processing and export are split
- * brush editing works
- * undo/redo works
- * process/export batch flow works again
- * compact controls are in place
- * model list is expanded
- * current next focus should be UI cleanup and clarity, not big architectural rewrites

Thought for a few seconds

Understood.

I will treat this as the active project state for **MyBGRemover** and continue from here rather than restarting or redesigning it.

What I am preserving going forward:

- **Process** and **Export** stay separate
- **Export** must work even if **Process** was not run first

- Selected stays removed from the settings panel
- Cancel should only appear during active jobs
- Preview Mode shows the **working image**
- Before shows the **original**
- Restore restores from the **original image**
- checked-item batch flow remains important
- main_window.py changes will be given as **before/after** or exact function placement
- non-main_window.py changes will be given as **full updated files**

The safest next phase is exactly what you identified: **Phase 24 – UI cleanup and hardening.**

Recommended next coding step:

1. Clean up ui/settings_panel.py
2. Hide unfinished controls that are confusing right now
3. Add model descriptions/tooltips so the model dropdown is easier to understand
4. Keep the current compact Action / Target / Run layout intact

The unfinished controls most likely to hide first are:

- softness
- opacity
- spacing
- color-removal controls, if not actually wired up
- watch-folder controls, if not reliable yet

To continue cleanly from here, send the current contents of:

- ui/settings_panel.py
- ui/main_window.py only if the settings panel currently depends on it for visibility/state logic

Then I will give you:

- the exact file(s) to change
- the full updated file for ui/settings_panel.py
- any main_window.py edits as careful before/after instructions
- step-by-step run/test instructions



main window:

```
from pathlib import Path
import hashlib
```

```
from PySide6.QtCore import Qt, QThread, QUrl,
QSettings
from PySide6.QtGui import QDesktopServices
from PySide6.QtWidgets import (
    QMainWindow,
    QWidget,
    QVBoxLayout,
    QProgressBar,
    QMessageBox,
    QSplitter,
    QFrame,
    QLabel,
    QScrollArea,
    QTextEdit,
)
```

```
from ui.file_panel import FilePanel
from ui.preview_canvas import PreviewCanvas
from ui.settings_panel import SettingsPanel
```

```
from core.state_manager import StateManager
from core.models import ImageItem
```

```
from utils.file_utils import scan_paths
from utils.image_utils import get_image_metadata,
create_thumbnail, create_preview_image
from workers.process_worker import ProcessWorker
from workers.full_export_worker import
FullExportWorker
from workers.watch_folder_worker import
```

```
WatchFolderWorker
```

```
from processing.export_utils import create_batch_zip
```

```
class MainWindow(QMainWindow):
```

```
    def _finalize_preview_thread(self):
        if self.preview_worker_thread is not None and
self.preview_worker_thread.isRunning():
            self.preview_worker_thread.quit()
            self.preview_worker_thread.wait(10000)
```

```
        if self.preview_worker is not None:
            self.preview_worker.deleteLater()
            self.preview_worker = None
```

```
        if self.preview_worker_thread is not None:
            self.preview_worker_thread.deleteLater()
            self.preview_worker_thread = None
```

```
    def _finalize_export_thread(self):
        if self.export_worker_thread is not None and
self.export_worker_thread.isRunning():
            self.export_worker_thread.quit()
            self.export_worker_thread.wait(10000)
```

```
        if self.export_worker is not None:
            self.export_worker.deleteLater()
            self.export_worker = None
```

```
        if self.export_worker_thread is not None:
            self.export_worker_thread.deleteLater()
            self.export_worker_thread = None
```

```
    def _shutdown_running_threads(self):
        if self.preview_worker_thread is not None and
self.preview_worker_thread.isRunning():
            self.preview_worker_thread.quit()
            self.preview_worker_thread.wait(3000)
```

```
        if self.export_worker_thread is not None and  
self.export_worker_thread.isRunning():  
            self.export_worker_thread.quit()  
            self.export_worker_thread.wait(3000)  
  
        if self.watch_worker is not None:  
            self.watch_worker.stop()  
  
        if self.watch_worker_thread is not None and  
self.watch_worker_thread.isRunning():  
            self.watch_worker_thread.quit()  
            self.watch_worker_thread.wait(3000)  
  
    def __init__(self):  
        super().__init__()  
        self.setWindowTitle("MyBGRemover")  
        self.resize(1500, 900)  
        self.setMinimumSize(1080, 700)  
  
        self.settings_store = QSettings("MyBGRemover",  
"MyBGRemover")  
  
        self.state = StateManager()  
  
        self.preview_worker_thread = None  
        self.preview_worker = None  
        self.export_worker_thread = None  
        self.export_worker = None  
  
        self.watch_worker_thread = None  
        self.watch_worker = None  
  
        self.processing_active = False  
        self.current_processing_index = None  
  
        self.batch_queue = []  
        self.batch_total = 0  
        self.batch_processed_count = 0  
        self.batch_mode = False  
        self.batch_action = None
```



```
        self.cancel_requested = False
        self.batch_exported_files = []

        self.visible_indices = []
        self._next_date_added = 0

        self.cache_root = Path("cache")
        self.thumb_cache_dir = self.cache_root / "thumbs"
        self.preview_cache_dir = self.cache_root /
"previews"
        self.processed_preview_cache_dir =
self.cache_root / "processed"
        self.edited_preview_cache_dir = self.cache_root /
"edited_previews"
        self.fullres_export_cache_dir = self.cache_root /
"fullres_exports"

        self.thumb_cache_dir.mkdir(parents=True,
exist_ok=True)
        self.preview_cache_dir.mkdir(parents=True,
exist_ok=True)

        self.processed_preview_cache_dir.mkdir(parents=True,
exist_ok=True)
        self.edited_preview_cache_dir.mkdir(parents=True,
exist_ok=True)
        self.fullres_export_cache_dir.mkdir(parents=True,
exist_ok=True)

        self.setAcceptDrops(True)

        central = QWidget()
        self.setCentralWidget(central)

        outer_layout = QVBoxLayout(central)
        outer_layout.setContentsMargins(8, 8, 8, 8)
        outer_layout.setSpacing(8)

        main_vertical_splitter =
QSplitter(Qt.Orientation.Vertical)
```

```
        content_splitter =  
        QSplitter(Qt.Orientation.Horizontal)  
  
        self.file_panel = FilePanel()  
        self.file_panel.setMinimumWidth(360)  
        self.file_panel.setMaximumWidth(520)  
  
        self.preview_canvas = PreviewCanvas()  
        self.preview_canvas.setMinimumWidth(300)  
  
        self.settings_panel = SettingsPanel()  
        self.settings_scroll = QScrollArea()  
        self.settings_scroll.setWidget(self.settings_panel)  
        self.settings_scroll.setWidgetResizable(True)  
        self.settings_scroll.setMinimumWidth(320)  
        self.settings_scroll.setMaximumWidth(380)  
  
        self.settings_scroll.setFrameShape(QFrame.Shape.NoFrame)  
  
        content_splitter.addWidget(self.file_panel)  
        content_splitter.addWidget(self.preview_canvas)  
        content_splitter.addWidget(self.settings_scroll)  
        content_splitter.setSizes([430, 780, 340])  
        content_splitter.setStretchFactor(0, 0)  
        content_splitter.setStretchFactor(1, 1)  
        content_splitter.setStretchFactor(2, 0)  
  
        content_container = QWidget()  
        content_layout = QVBoxLayout(content_container)  
        content_layout.setContentsMargins(0, 0, 0, 0)  
        content_layout.setSpacing(8)  
  
        self.progress_bar = QProgressBar()  
        self.progress_bar.setValue(0)  
        self.progress_bar.setMinimumHeight(22)  
        self.progress_bar.setMaximumHeight(24)  
  
        content_layout.addWidget(content_splitter, 1)  
        content_layout.addWidget(self.progress_bar, 0)
```

```
        self.log_frame = QFrame()
        self.log_frame.setObjectName("logFrame")

    self.log_frame.setFrameShape(QFrame.Shape.StyledPanel)
    self.log_frame.setMinimumHeight(100)

    log_layout = QVBoxLayout(self.log_frame)
    log_layout.setContentsMargins(10, 10, 10, 10)
    log_layout.setSpacing(8)

    self.log_title = QLabel("Log Panel")
    self.log_title.setObjectName("panelTitle")

    self.log_panel = QTextEdit()
    self.log_panel.setReadOnly(True)
    self.log_panel.setPlaceholderText("Status / log activity will appear here...")

    log_layout.addWidget(self.log_title)
    log_layout.addWidget(self.log_panel)

    main_vertical_splitter.addWidget(content_container)
    main_vertical_splitter.addWidget(self.log_frame)
    main_vertical_splitter.setSizes([680, 170])
    main_vertical_splitter.setStretchFactor(0, 1)
    main_vertical_splitter.setStretchFactor(1, 0)

    outer_layout.addWidget(main_vertical_splitter, 1)

    self._apply_basic_styles()
    self._connect_signals()
    self._restore_persistent_settings()
    self._sync_preview_tool_settings()
    self._update_watch_status_label(False, "")
    self.preview_canvas.clear_preview()

    self.log("MyBGRemover started.")
```

```
        self.log("Drag and drop files or folders into the  
window.")  
        self.log("Process = preview/background removal  
only.")  
        self.log("Export = final full-resolution file output.")  
        self.log("Preview processing uses preview-size  
images for speed.")  
        self.log("Final export uses the original full-  
resolution source image.")  
        self.log("Output folder is preserved between  
sessions.")  
        self.log("ZIP export is available for batch runs.")
```

```
def _apply_basic_styles(self):  
    self.setStyleSheet("""  
        QWidget {  
            background: #2b2b2b;  
            color: #f0f0f0;  
        }  
        QFrame#filePanel, QFrame#previewCanvas,  
QFrame#settingsPanel, QFrame#logFrame {  
            border: 2px solid #808080;  
            border-radius: 6px;  
            background: #353535;  
        }  
        QLabel#panelTitle {  
            font-size: 16px;  
            font-weight: bold;  
            color: #ffffff;  
            padding: 2px 0 6px 0;  
        }  
        QListWidget, QTextEdit, QLineEdit,  
QComboBox, QScrollArea {  
            background: #1f1f1f;  
            color: #f0f0f0;  
            border: 1px solid #666;  
        }  
        QPushButton {  
            background: #4a4a4a;  
            color: #ffffff;
```

```
        border: 1px solid #777;
        padding: 6px;
    }
    QPushButton:hover {
        background: #5a5a5a;
    }
    QPushButton:disabled {
        background: #3a3a3a;
        color: #999999;
    }
    QCheckBox, QLabel {
        color: #f0f0f0;
    }
    QProgressBar {
        border: 1px solid #666;
        background: #1f1f1f;
        color: #ffffff;
        text-align: center;
    }
    """)
```

```
def _connect_signals(self):
```

```
    self.file_panel.item_selected.connect(self.select_item_from_panel)
```

```
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
```

```
    self.file_panel.status_filter_changed.connect(self.rebuild_file_panel)
```

```
    self.file_panel.type_filter_changed.connect(self.rebuild_file_panel)
```

```
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)
```

```
    self.file_panel.remove_selected_btn.clicked.connect(sel
```

```
f.remove_selected_item)
```

```
self.file_panel.clear_completed_btn.clicked.connect(self  
.clear_completed_items)
```

```
self.file_panel.clear_all_btn.clicked.connect(self.clear_a  
ll_items)
```

```
self.file_panel.retry_failed_btn.clicked.connect(self.retr  
y_failed_items)
```

```
self.file_panel.open_output_btn.clicked.connect(self.op  
en_output_folder)
```

```
self.file_panel.process_checked_requested.connect(self  
.process_checked_items)
```

```
self.file_panel.remove_checked_requested.connect(self.  
remove_checked_items)
```

```
self.file_panel.uncheck_completed_requested.connect(s  
elf.uncheck_completed_items)
```

```
self.settings_panel.run_action_btn.clicked.connect(self.  
run_selected_action)
```

```
self.settings_panel.cancel_btn.clicked.connect(self.requ  
est_cancel)
```

```
self.settings_panel.tool_combo.currentTextChanged.co  
nnect(self._sync_preview_tool_settings)
```

```
self.settings_panel.apply_mode_combo.currentTextCha  
nged.connect(self._sync_preview_tool_settings)
```

```
self.settings_panel.brush_size_slider.valueChanged.con  
nect(self._sync_preview_tool_settings)
```

```
self.settings_panel.softness_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
self.settings_panel.watch_enable_check.toggled.connect(self._on_watch_enable_toggled)
```

```
self.settings_panel.output_dir_edit.textChanged.connect(self._persist_output_dir)
```

```
self.settings_panel.naming_edit.textChanged.connect(self._persist_naming_suffix)
```

```
self.preview_canvas.image_edited.connect(self._on_preview_image_edited)
```

```
def _restore_persistent_settings(self):
    output_dir = self.settings_store.value("output_dir",
    "", type=str)
    naming_suffix =
self.settings_store.value("naming_suffix", "_nobg",
type=str)
```

```
    if output_dir:
```

```
self.settings_panel.output_dir_edit.setText(output_dir)
    if naming_suffix:
```

```
self.settings_panel.naming_edit.setText(naming_suffix)
```

```
def _persist_output_dir(self, text: str):
    self.settings_store.setValue("output_dir", text)

def _persist_naming_suffix(self, text: str):
    self.settings_store.setValue("naming_suffix", text)

def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(

tool_name=self.settings_panel.tool_combo.currentText
()),

apply_mode=self.settings_panel.apply_mode_combo.cu
rrentText(),

brush_size=self.settings_panel.brush_size_slider.value(
),

softness=self.settings_panel.softness_slider.value(),

opacity=self.settings_panel.opacity_slider.value(),

spacing=self.settings_panel.spacing_slider.value(),

tolerance=self.settings_panel.tolerance_slider.value(),
    )

def run_selected_action(self):
    action =
self.settings_panel.action_combo.currentText()
    target =
self.settings_panel.target_combo.currentText()

    if action == "Process":
        if target == "Checked":
            self.process_checked_items()
        else:
            self.process_all_items()
    else:
```



```
        if target == "Checked":
            self.export_checked_items()
        else:
            self.export_all_items()

    def _short_text(self, text: str, max_len: int = 48) ->
str:
        return text if len(text) <= max_len else
text[:max_len - 3] + "..."

    def _cache_token(self, path: Path) -> str:
        return
hashlib.sha1(str(path.resolve()).encode("utf-8")).hexdigest()[:16]

    def _get_output_dir(self) -> Path | None:
        output_dir_text =
self.settings_panel.output_dir_edit.text().strip()
        if not output_dir_text:
            return None

        output_dir = Path(output_dir_text)
        try:
            output_dir.mkdir(parents=True, exist_ok=True)
        except Exception:
            return None

        return output_dir if output_dir.exists() and
output_dir.is_dir() else None

    def _build_export_suffix(self) -> str:
        model_name =
self.settings_panel.model_combo.currentText().strip()
        user_suffix =
self.settings_panel.naming_edit.text().strip()

        if not user_suffix:
            user_suffix = "_nobg"

        if not user_suffix.startswith("_"):
```

```
        user_suffix = f"_{user_suffix}"

    if model_name:
        return f"_{model_name}{user_suffix}"

    return user_suffix

def _get_checked_or_all_indices(self) -> list[int]:
    checked =
self.file_panel.get_checked_user_indices()
    if checked:
        return checked
    return list(range(len(self.state.items)))

def _has_processed_preview(self, item: ImageItem) -
> bool:
    return item.processed_preview_path is not None
and Path(item.processed_preview_path).exists()

def _has_edited_preview(self, item: ImageItem) ->
bool:
    return item.edited_preview_path is not None and
Path(item.edited_preview_path).exists()

def _update_watch_status_label(self, enabled: bool,
folder_path: str):
    if enabled and folder_path:

self.settings_panel.watch_status_label.setText(f"Watch
status: On ({folder_path})")
    else:

self.settings_panel.watch_status_label.setText("Watch
status: Off")

def _start_watch_folder(self):
    folder_path =
self.settings_panel.watch_folder_edit.text().strip()
    include_subfolders =
self.settings_panel.watch_include_subfolders_check.is
```

```
Checked()
```

```
        if not folder_path:
            self.log("Watch folder could not start: no folder
selected.")
```

```
self.settings_panel.watch_enable_check.blockSignals(T
rue)
```

```
self.settings_panel.watch_enable_check.setChecked(Fal
se)
```

```
self.settings_panel.watch_enable_check.blockSignals(F
alse)
        self._update_watch_status_label(False, "")
        return
```

```
        folder = Path(folder_path)
        if not folder.exists() or not folder.is_dir():
            self.log(f"Watch folder could not start: invalid
directory: {folder_path}")
```

```
self.settings_panel.watch_enable_check.blockSignals(T
rue)
```

```
self.settings_panel.watch_enable_check.setChecked(Fal
se)
```

```
self.settings_panel.watch_enable_check.blockSignals(F
alse)
        self._update_watch_status_label(False, "")
        return
```

```
        if self.watch_worker_thread is not None or
self.watch_worker is not None:
            self._stop_watch_folder()
```

```
        self.watch_worker_thread = QThread(self)
        self.watch_worker = WatchFolderWorker(
            folder_path=folder_path,
```

```
        include_subfolders=include_subfolders,
        poll_interval_seconds=2.0,
    )

    self.watch_worker.moveToThread(self.watch_worker_thread)

    self.watch_worker_thread.started.connect(self.watch_worker.run)
    self.watch_worker.status.connect(self.log)

    self.watch_worker.new_files_detected.connect(self._on_watch_new_files_detected)

    self.watch_worker.error.connect(self._on_watch_error)

    self.watch_worker.finished.connect(self.watch_worker_thread.quit)

    self.watch_worker_thread.finished.connect(self._cleanup_watch_worker)

    self.watch_worker_thread.start()
    self._update_watch_status_label(True, folder_path)

    def _stop_watch_folder(self):
        if self.watch_worker is not None:
            self.watch_worker.stop()

        self._update_watch_status_label(False, "")

    def _cleanup_watch_worker(self):
        if self.watch_worker is not None:
            self.watch_worker.deleteLater()
            self.watch_worker = None

        if self.watch_worker_thread is not None:
            if self.watch_worker_thread.isRunning():
```

```
        self.watch_worker_thread.quit()
        self.watch_worker_thread.wait(3000)
        self.watch_worker_thread.deleteLater()
        self.watch_worker_thread = None

    def _on_watch_enable_toggled(self, checked: bool):
        if checked:
            self._start_watch_folder()
        else:
            self._stop_watch_folder()

    def _on_watch_error(self, message: str):
        self.log(f"Watch folder error: {message}")

    self.settings_panel.watch_enable_check.blockSignals(True)

    self.settings_panel.watch_enable_check.setChecked(False)

    self.settings_panel.watch_enable_check.blockSignals(False)
    self._update_watch_status_label(False, "")

    def _on_watch_new_files_detected(self, file_paths:
list):
        if not file_paths:
            return

        existing_paths = {str(item.source_path.resolve())
for item in self.state.items}
        added_any = False

        for file_path_str in file_paths:
            path = Path(file_path_str)
            resolved = str(path.resolve())
            if resolved in existing_paths:
                continue

            self._add_image_item(path)
```

```
        existing_paths.add(resolved)
        added_any = True

    if added_any:
        self._refresh_type_filter_options()
        self.rebuild_file_panel()

    def log(self, message: str):
        self.log_panel.append(message)

    def dragEnterEvent(self, event):
        if event.mimeData().hasUrls():
            event.acceptProposedAction()
        else:
            event.ignore()

    def dropEvent(self, event):
        if not event.mimeData().hasUrls():
            event.ignore()
            return

        if self.processing_active:
            self.log("Drop ignored while processing is
active.")
            event.ignore()
            return

        urls = event.mimeData().urls()
        dropped_paths = [url.toLocalFile() for url in urls if
url.isLocalFile()]
        include_subfolders =
self.file_panel.include_subfolders_check.isChecked()
        files, skipped = scan_paths(dropped_paths,
include_subfolders=include_subfolders)

        existing_paths = {str(item.source_path.resolve())
for item in self.state.items}
        duplicate_files = [f for f in files if str(f.resolve()) in
existing_paths]
        new_files = [f for f in files if str(f.resolve()) not in
```

```
existing_paths]

if duplicate_files:
    reply = QMessageBox.question(
        self,
        "Duplicates Detected",
        f"{len(duplicate_files)} duplicate file(s) were
found.\nAdd duplicates anyway?",
        QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No,
    )
    if reply == QMessageBox.StandardButton.Yes:
        new_files.extend(duplicate_files)

for path in new_files:
    self._add_image_item(path)

for path in skipped:
    self.log(f"Skipped unsupported file: {path}")

self._refresh_type_filter_options()
self.rebuild_file_panel()

event.acceptProposedAction()

def _add_image_item(self, path: Path):
    metadata = get_image_metadata(path)
    token = self._cache_token(path)
    thumb_path = self.thumb_cache_dir / f"{token}
_thumb.png"
    preview_path = self.preview_cache_dir / f"{token}
_preview.png"

    thumb_ok = create_thumbnail(path, thumb_path)
    preview_ok = create_preview_image(path,
preview_path)

    item = ImageItem(
        source_path=path,
```

```
        filename=path.name,
        file_type=path.suffix.lower(),
        dimensions=metadata["dimensions"],
        dpi=metadata["dpi"],
        thumb_path=thumb_path if thumb_ok else
None,
        preview_path=preview_path if preview_ok else
None,
        date_added=self._next_date_added,
    )
    self._next_date_added += 1
    self.state.add_item(item)

    if not preview_ok:
        self.log(f"Could not create preview: {path}")
    if not thumb_ok:
        self.log(f"Could not create thumbnail: {path}")

    self.log(f"Loaded: {path}")

    def _refresh_type_filter_options(self):
        file_types = sorted({item.file_type for item in
self.state.items})
        self.file_panel.set_type_filter_options(file_types)

    def _item_matches_filter(self, item: ImageItem,
status_filter: str, type_filter: str) -> bool:
        effective_status = "All" if status_filter in ("Filter
status", "") else status_filter
        effective_type = "All Types" if type_filter in ("Filter
type", "") else type_filter

        status_ok = effective_status == "All" or
item.status.lower() == effective_status.lower()
        type_ok = effective_type == "All Types" or
item.file_type == effective_type
        return status_ok and type_ok

    def _sorted_indices(self) -> list[int]:
        sort_name =
```



```
self.file_panel.sort_combo.currentText()
    effective_sort = "Date Added" if sort_name ==
"Sort by" else sort_name

    indices = list(range(len(self.state.items)))

    if effective_sort == "Date Added":
        indices.sort(key=lambda i:
self.state.items[i].date_added)
    elif effective_sort == "Name":
        indices.sort(key=lambda i:
self.state.items[i].filename.lower())
    elif effective_sort == "Type":
        indices.sort(key=lambda i:
(self.state.items[i].file_type.lower(),
self.state.items[i].filename.lower()))
    elif effective_sort == "Status":
        indices.sort(key=lambda i:
(self.state.items[i].status.lower(),
self.state.items[i].filename.lower()))

    return indices

def _build_panel_text(self, item: ImageItem) -> str:
    if self.file_panel.current_view_mode ==
"thumbnail":
        return item.filename

    return (
        f"{self._short_text(item.filename, 42)}\n"
        f"{item.file_type} | {item.dimensions} | DPI:
{item.dpi} | {item.status}"
    )

def rebuild_file_panel(self, *_args, reload_preview:
bool = True):
    selected_index = self.state.selected_index
    checked_indices =
set(self.file_panel.get_checked_user_indices())
    status_filter =
```

```
self.file_panel.status_filter_combo.currentText()
    type_filter =
self.file_panel.type_filter_combo.currentText()

self.file_panel.file_view.blockSignals(True)
self.file_panel.clear_items()
self.visible_indices = []

for item_index in self._sorted_indices():
    item = self.state.items[item_index]
    if not self._item_matches_filter(item,
status_filter, type_filter):
        continue

    thumb_path = item.edited_preview_path or
item.processed_preview_path or item.thumb_path
    self.file_panel.add_item(
        self._build_panel_text(item),
        str(thumb_path) if thumb_path else None,
        item_index,
        checked=item_index in checked_indices,
    )
    self.visible_indices.append(item_index)

selected_to_show = None

if selected_index is not None and selected_index in
self.visible_indices:
    visible_row =
self.visible_indices.index(selected_index)

self.file_panel.set_current_visible_row(visible_row)
    selected_to_show = selected_index
elif self.visible_indices:
    selected_to_show = self.visible_indices[0]
    self.state.set_selected_index(selected_to_show)
    self.file_panel.set_current_visible_row(0)
else:
    self.state.set_selected_index(None)
    self.preview_canvas.clear_preview()
```

```
self.file_panel.file_view.blockSignals(False)

if hasattr(self.file_panel, "_update_counts"):
    self.file_panel._update_counts()

if selected_to_show is not None and
reload_preview:
    self.select_item(selected_to_show)

def _refresh_item_display(self, _index: int):
    self.rebuild_file_panel()

def select_item_from_panel(self, _visible_row: int):
    real_index =
self.file_panel.get_selected_user_index()
    if real_index >= 0:
        self.select_item(real_index, source="panel")

def select_item(self, index: int, source: str | None =
None):
    item = self.state.get_item(index)
    if item is None:
        return

    self.state.set_selected_index(index)

    if source != "panel" and index in self.visible_indices:
        self.file_panel.file_view.blockSignals(True)

self.file_panel.set_current_visible_row(self.visible_indices.index(index))
    self.file_panel.file_view.blockSignals(False)

    before_path = item.preview_path
    after_path = item.edited_preview_path or
item.processed_preview_path
    editable_save_path =
self.edited_preview_cache_dir /
f"{self._cache_token(item.source_path)}_edited.png"
```

```
        preferred_interaction_mode =
getattr(self.preview_canvas, "_interaction_mode",
"preview")
        preferred_view_mode =
getattr(self.preview_canvas, "_current_mode", "after")

        if before_path and Path(before_path).exists():
            title_text = (
                f"{self._short_text(item.filename, 60)} | "
                f"{item.dimensions} | DPI: {item.dpi} | "
                f"{item.status}"
            )

            self.preview_canvas.set_preview_images(
                before_image_path=str(before_path),
                after_image_path=str(after_path) if
after_path and Path(after_path).exists() else None,
                title_text=title_text,
                default_mode="after",
                editable_save_path=str(editable_save_path),

preferred_interaction_mode=preferred_interaction_mo
de,
                preferred_view_mode=preferred_view_mode,
            )
        else:
            self.preview_canvas.clear_preview()
            self.log(f"Could not load preview:
{item.source_path}")

    def _on_preview_image_edited(self, save_path_str:
str):
        selected_index = self.state.selected_index
        if selected_index is None:
            return

        item = self.state.get_item(selected_index)
        if item is None:
            return
```

```
        item.edited_preview_path = Path(save_path_str)

        if item.processed_preview_path is None:
            item.extra["preprocess_edit_preview_path"] =
save_path_str

        item.extra["preprocess_edit_base_preview_path"] = (
            str(item.preview_path) if item.preview_path is
not None else ""
        )

        if item.status != "processing":
            item.status = "edited"

        self.rebuild_file_panel(reload_preview=False)

    def remove_selected_item(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
"Wait for processing to finish first.")
            return

        selected_index = self.state.selected_index
        if selected_index is None:
            QMessageBox.information(self, "No Selection",
"Select an image first.")
            return

        item = self.state.get_item(selected_index)
        if item is None:
            return

        self.state.items.pop(selected_index)

        for idx, state_item in enumerate(self.state.items):
            state_item.date_added = idx
        self._next_date_added = len(self.state.items)

        self.state.set_selected_index(None)
```

```
        self._refresh_type_filter_options()
        self.rebuild_file_panel()
        self.preview_canvas.clear_preview()

        self.log(f"Removed: {item.source_path}")

    def remove_checked_items(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
                "Wait for processing to finish first.")
            return

        checked =
set(self.file_panel.get_checked_user_indices())
        if not checked:
            QMessageBox.information(self, "Nothing
Checked", "No checked files to remove.")
            return

        self.state.items = [item for idx, item in
enumerate(self.state.items) if idx not in checked]

        for idx, state_item in enumerate(self.state.items):
            state_item.date_added = idx
            self._next_date_added = len(self.state.items)

        self.state.set_selected_index(None)
        self._refresh_type_filter_options()
        self.rebuild_file_panel()
        self.preview_canvas.clear_preview()
        self.log(f"Removed {len(checked)} checked
item(s).")

    def clear_completed_items(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
                "Wait for processing to finish first.")
            return

        before_count = len(self.state.items)
```

```
        self.state.items = [item for item in self.state.items
                             if item.status != "exported"]

        for idx, state_item in enumerate(self.state.items):
            state_item.date_added = idx
            self._next_date_added = len(self.state.items)

        removed_count = before_count -
len(self.state.items)
        self.state.set_selected_index(None)
        self._refresh_type_filter_options()
        self.rebuild_file_panel()
        self.preview_canvas.clear_preview()

        self.log(f"Cleared exported items:
{removed_count}")

    def uncheck_completed_items(self):
        if not hasattr(self.file_panel, "file_view"):
            return

        self.file_panel.file_view.blockSignals(True)
        for i in range(self.file_panel.file_view.count()):
            item = self.file_panel.file_view.item(i)
            if item is None:
                continue

            index = item.data(Qt.ItemDataRole.UserRole)
            if index is None:
                continue

            state_item = self.state.get_item(index)
            if state_item is not None and state_item.status
            == "exported":

        item.setCheckState(Qt.CheckState.Unchecked)

        self.file_panel.file_view.blockSignals(False)

        if hasattr(self.file_panel,
```

```
        "_sync_master_checkbox"):
            self.file_panel._sync_master_checkbox()
            if hasattr(self.file_panel, "_update_counts"):
                self.file_panel._update_counts()

            self.log("Unchecked exported items.")

    def retry_failed_items(self):
        failed_indices = [idx for idx, item in
            enumerate(self.state.items) if item.status == "error"]
        if not failed_indices:
            self.log("No failed items to retry.")
            return

        for idx in failed_indices:
            item = self.state.items[idx]
            item.status = "pending"
            item.processed_preview_path = None
            item.edited_preview_path = None
            item.output_path = ""
            item.extra.pop("preprocess_edit_preview_path",
                None)

            item.extra.pop("preprocess_edit_base_preview_path",
                None)

            self.rebuild_file_panel()
            self.log(f"Reset failed items to pending:
                {len(failed_indices)}")

    def open_output_folder(self):
        output_dir = self._get_output_dir()
        if output_dir is not None and output_dir.exists():

            QDesktopServices.openUrl(QUrl.fromLocalFile(str(outp
                ut_dir)))

            self.log(f"Opened output folder: {output_dir}")
            return

        selected_index = self.state.selected_index
```



```
        if selected_index is not None:
            item = self.state.get_item(selected_index)
            if item is not None and
item.source_path.parent.exists():

QDesktopServices.openUrl(QUrl.fromLocalFile(str(item
.source_path.parent)))
            self.log(f"Opened source folder for selected
image: {item.source_path.parent}")
            return

        QMessageBox.information(
            self,
            "Folder Not Available",
            "No valid output folder is set, and no selected
image folder could be opened.",
        )

    def process_selected_item(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
            "An image is already being processed.")
            return

        selected_index = self.state.selected_index
        if selected_index is None:
            QMessageBox.information(self, "No Selection",
            "Select an image first.")
            return

        item = self.state.get_item(selected_index)
        if item is None:
            QMessageBox.information(self, "No Selection",
            "Select an image first.")
            return

        if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
            QMessageBox.information(
```

```
        self,
        "Not Yet Available",
        "Color Removal will be added later. Use AI
Removal for now.",
    )
    return
```

```
self.batch_mode = False
self.batch_action = "process"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.cancel_requested = False
self.batch_exported_files = []
```

```
self._start_preview_processing_item(selected_index,
force_reprocess=True)
```

```
def process_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
processing job is already active.")
    return
```

```
if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
    QMessageBox.information(
        self,
        "Not Yet Available",
        "Color Removal will be added later. Use AI
Removal for now.",
    )
    return
```

```
checked =
self.file_panel.get_checked_user_indices()
if not checked:
    QMessageBox.information(self, "Nothing
```

```
Checked", "No checked files to process.")
    return

    self.batch_queue = [
        index for index in checked
        if 0 <= index < len(self.state.items)
        and (
            (self.state.items[index].preview_path is not
None and
Path(self.state.items[index].preview_path).exists())
            or
self._has_edited_preview(self.state.items[index])
        )
    ]

    if not self.batch_queue:
        QMessageBox.information(
            self,
            "No Valid Images",
            "The checked files do not have valid preview
data to process.",
        )
        return

    self.batch_total = len(self.batch_queue)
    self.batch_processed_count = 0
    self.batch_mode = True
    self.batch_action = "process"
    self.cancel_requested = False
    self.batch_exported_files = []

    self.log(f"Processing {self.batch_total} checked
image(s).")
    self._start_next_batch_item()

    def process_all_items(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing", "A
processing job is already active.")
            return
```

```
        if not self.state.items:
            QMessageBox.information(self, "No Images",
                                    "Load images first.")
            return

        if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
            QMessageBox.information(
                self,
                "Not Yet Available",
                "Color Removal will be added later. Use AI
Removal for now.",
            )
            return

        self.batch_queue = [
            index
            for index in range(len(self.state.items))
            if (
                (self.state.items[index].preview_path is not
None and
Path(self.state.items[index].preview_path).exists())
                or
self._has_edited_preview(self.state.items[index])
            )
        ]

        if not self.batch_queue:
            QMessageBox.information(self, "No Valid
Images", "There are no valid images to process.")
            return

        self.batch_mode = True
        self.batch_action = "process"
        self.batch_total = len(self.batch_queue)
        self.batch_processed_count = 0
        self.cancel_requested = False
        self.batch_exported_files = []
```

```
        self.log(f"Starting batch processing for
{self.batch_total} image(s).")
        self._start_next_batch_item()

    def export_selected_item(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing", "A
job is already active.")
            return

        selected_index = self.state.selected_index
        if selected_index is None:
            QMessageBox.information(self, "No Selection",
"Select an image first.")
            return

        item = self.state.get_item(selected_index)
        if item is None:
            QMessageBox.information(self, "No Selection",
"Select an image first.")
            return

        if self._get_output_dir() is None:
            QMessageBox.information(
                self,
                "Output Folder Required",
                "Choose a valid output folder before
exporting.",
            )
            return

        if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
            QMessageBox.information(
                self,
                "Not Yet Available",
                "Color Removal export will be added later. Use
AI Removal for now.",
```

```
)
return

self.batch_mode = False
self.batch_action = "export"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self._start_export_item(selected_index)

def export_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
job is already active.")
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before
exporting.",
        )
        return

    if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use
AI Removal for now.",
        )
        return

    checked =
```

```
self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing
Checked", "No checked files to export.")
        return

    self.batch_queue = [index for index in checked if 0
<= index < len(self.state.items)]

    if not self.batch_queue:
        QMessageBox.information(self, "No Valid
Images", "The checked files are not valid for export.")
        return

    self.batch_total = len(self.batch_queue)
    self.batch_processed_count = 0
    self.batch_mode = True
    self.batch_action = "export"
    self.cancel_requested = False
    self.batch_exported_files = []

    self.log(f"Exporting {self.batch_total} checked
image(s).")
    self._start_next_batch_item()

def export_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
job is already active.")
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images",
"Load images first.")
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
```

```
        "Choose a valid output folder before
exporting.",
    )
    return

    if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use
AI Removal for now.",
        )
        return

        self.batch_queue = list(range(len(self.state.items)))
        self.batch_mode = True
        self.batch_action = "export"
        self.batch_total = len(self.batch_queue)
        self.batch_processed_count = 0
        self.cancel_requested = False
        self.batch_exported_files = []

        self.log(f"Starting batch export for
{self.batch_total} image(s).")
        self._start_next_batch_item()

def request_cancel(self):
    if not self.processing_active:
        self.log("No active job to cancel.")
        return

        self.cancel_requested = True
        self.log("Cancel requested. The current item will
finish, then the batch will stop.")

def _start_next_batch_item(self):
    if self.cancel_requested:
        self.log("Batch canceled by user.")
```



```
        self._finish_processing_session()
        return

    if not self.batch_queue:
        if self.batch_action == "export":
            self.log("Batch export complete.")
        else:
            self.log("Batch processing complete.")

    self._finish_processing_session(final_progress=100)
    return

    next_index = self.batch_queue.pop(0)

    if self.batch_action == "export":
        self._start_export_item(next_index)
    else:
        self._start_preview_processing_item(next_index,
        force_reprocess=True)

    def _set_processing_controls_enabled(self, enabled:
    bool):

        self.settings_panel.run_action_btn.setEnabled(enabled)
        self.settings_panel.cancel_btn.setVisible(not
        enabled)
        self.settings_panel.cancel_btn.setEnabled(not
        enabled)

        if hasattr(self.file_panel, "process_checked_btn"):

            self.file_panel.process_checked_btn.setEnabled(enable
            d)

    def _update_batch_progress_busy(self):
        completed_percent =
        int((self.batch_processed_count / self.batch_total) *
        100) if self.batch_total > 0 else 0
        self.progress_bar.setRange(0, 100)
        self.progress_bar.setValue(completed_percent)
```

```
def _update_batch_progress(self):
    percent = int((self.batch_processed_count /
self.batch_total) * 100) if self.batch_total > 0 else 0
    self.progress_bar.setRange(0, 100)
    self.progress_bar.setValue(percent)

def _start_preview_processing_item(self, item_index:
int, force_reprocess: bool = False):
    item = self.state.get_item(item_index)
    if item is None:
        if self.batch_mode:
            self._start_next_batch_item()
        return

    preview_input_path = item.edited_preview_path or
item.preview_path

    if item.edited_preview_path is not None and
Path(item.edited_preview_path).exists():
        item.extra["preprocess_edit_preview_path"] =
str(item.edited_preview_path)

    item.extra["preprocess_edit_base_preview_path"] = (
        str(item.preview_path) if item.preview_path is
not None else ""
    )
    else:
        item.extra.pop("preprocess_edit_preview_path",
None)

    item.extra.pop("preprocess_edit_base_preview_path",
None)

    if preview_input_path is None or not
Path(preview_input_path).exists():
        self.log(f"Cannot process preview for:
{item.source_path}")
        item.status = "error"
        self._refresh_item_display(item_index)
```

```
        if self.batch_mode:
            self.batch_processed_count += 1
            self._update_batch_progress()
            self._start_next_batch_item()
        return

    if self.processing_active:
        self.log("Processing is already active; new start
request ignored.")
        return

    if self._has_processed_preview(item) and not
force_reprocess:
        item.status = "processed"
        self._refresh_item_display(item_index)
        self.select_item(item_index)

    if self.batch_mode:
        self.batch_processed_count += 1
        self._update_batch_progress()
        self._start_next_batch_item()
    return

    model_name =
self.settings_panel.model_combo.currentText()
    token = self._cache_token(item.source_path)
    processed_preview_path =
self.processed_preview_cache_dir / f"{token}
_processed.png"

    self.current_processing_index = item_index
    item.status = "processing"
    self._refresh_item_display(item_index)
    self.select_item(item_index)

    self.processing_active = True
    self._set_processing_controls_enabled(False)

    if self.batch_mode:
```

```
        self._update_batch_progress_busy()
    else:
        self.progress_bar.setRange(0, 0)

    self.log(f"Preview processing item {item_index +
1}: {item.filename}")
    self.log(f"Model: {model_name}")
    self.log(f"Input preview path:
{preview_input_path}")
    self.log(f"Output preview path:
{processed_preview_path}")

    self.preview_worker_thread = QThread(self)
    self.preview_worker = ProcessWorker(
        item_index=item_index,
        input_path=Path(preview_input_path),
        output_path=processed_preview_path,
        model_name=model_name,
    )

    self.preview_worker.moveToThread(self.preview_worke
r_thread)

    self.preview_worker_thread.started.connect(self.prie
w_worker.run)
    self.preview_worker.status.connect(self.log)

    self.preview_worker.finished.connect(self._on_preview
_process_finished)

    self.preview_worker.error.connect(self._on_preview_pr
ocess_error)

    self.preview_worker.finished.connect(self.preview_wor
ker_thread.quit)

    self.preview_worker.error.connect(self.preview_worker
_thread.quit)
```

```
self.preview_worker_thread.finished.connect(self._cleanup_preview_worker)

self.preview_worker_thread.start()

def _start_export_item(self, item_index: int):
    item = self.state.get_item(item_index)
    if item is None:
        if self.batch_mode:
            self._start_next_batch_item()
        return

    self.current_processing_index = item_index
    item.status = "processing"
    self._refresh_item_display(item_index)
    self.select_item(item_index)

    self.processing_active = True
    self._set_processing_controls_enabled(False)

    if self.batch_mode:
        self._update_batch_progress_busy()
    else:
        self.progress_bar.setRange(0, 0)

    self._start_full_export_item(item_index,
    apply_preview_edits=True)

def _cleanup_preview_worker(self):
    if self.preview_worker is not None:
        self.preview_worker.deleteLater()
        self.preview_worker = None

    if self.preview_worker_thread is not None and not
    self.preview_worker_thread.isRunning():
        self.preview_worker_thread.deleteLater()
        self.preview_worker_thread = None

def _start_full_export_item(self, item_index: int,
    apply_preview_edits: bool = False):
```

```
        item = self.state.get_item(item_index)
        if item is None:
            self._handle_post_item_completion()
            return

        output_dir = self._get_output_dir()
        if output_dir is None:
            item.status = "error"
            self.log("Export failed: output directory is not
available.")
            self._refresh_item_display(item_index)
            self._handle_post_item_completion(error=True)
            return

        model_name =
self.settings_panel.model_combo.currentText()
        suffix = self._build_export_suffix()
        output_format =
self.settings_panel.output_format_combo.currentText()
        .upper()
        background_mode =
self.settings_panel.background_mode_combo.currentTe
xt()
        background_color =
self.settings_panel.background_color_combo.currentTe
xt()

        token = self._cache_token(item.source_path)
        fullres_cache_path = self.fullres_export_cache_dir
/ f"{token}_fullres.png"

        base_preview_mask_path = None
        edited_preview_mask_path = None
        resolved_apply_preview_edits = False

        if self._has_processed_preview(item) and
self._has_edited_preview(item):
            base_preview_mask_path =
item.processed_preview_path
            edited_preview_mask_path =
```

```
item.edited_preview_path
    resolved_apply_preview_edits = True
else:
    preprocess_edit_preview =
item.extra.get("preprocess_edit_preview_path")
    preprocess_base_preview =
item.extra.get("preprocess_edit_base_preview_path")

    if preprocess_edit_preview and
preprocess_base_preview:
        preprocess_edit_preview_path =
Path(preprocess_edit_preview)
        preprocess_base_preview_path =
Path(preprocess_base_preview)

        if preprocess_edit_preview_path.exists() and
preprocess_base_preview_path.exists():
            base_preview_mask_path =
preprocess_base_preview_path
            edited_preview_mask_path =
preprocess_edit_preview_path
            resolved_apply_preview_edits = True

        self.log(f"Starting full-resolution export for:
{item.filename}")
        self.log(f"Source image: {item.source_path}")
        self.log(f"Full-res temp path:
{fullres_cache_path}")
        self.log(f"Filename suffix: {suffix}")
        self.log(f"Apply preview edits: {'yes' if
resolved_apply_preview_edits and
edited_preview_mask_path else 'no'}")

        if base_preview_mask_path is not None:
            self.log(f"Base preview mask path:
{base_preview_mask_path}")
            if edited_preview_mask_path is not None:
                self.log(f"Edited preview mask path:
{edited_preview_mask_path}")
```

```
self.export_worker_thread = QThread(self)
self.export_worker = FullExportWorker(
    item_index=item_index,
    source_path=item.source_path,
    processed_cache_path=fullres_cache_path,
    output_dir=output_dir,
    output_format=output_format,
    suffix=suffix,
    model_name=model_name,
    background_mode=background_mode,
    background_color_name=background_color,

    base_preview_mask_path=base_preview_mask_path,

    edited_preview_mask_path=edited_preview_mask_path
    ,
    apply_preview_edits=(
        resolved_apply_preview_edits
        and base_preview_mask_path is not None
        and edited_preview_mask_path is not None
    ),
)

self.export_worker.moveToThread(self.export_worker_thread)

self.export_worker_thread.started.connect(self.export_worker.run)
    self.export_worker.status.connect(self.log)

self.export_worker.finished.connect(self._on_full_export_finished)

self.export_worker.error.connect(self._on_full_export_error)

self.export_worker.finished.connect(self.export_worker_thread.quit)
```



```
self.export_worker.error.connect(self.export_worker_thread.quit)

self.export_worker_thread.finished.connect(self._cleanup_export_worker)

self.export_worker_thread.start()

def _cleanup_export_worker(self):
    if self.export_worker is not None:
        self.export_worker.deleteLater()
        self.export_worker = None

    if self.export_worker_thread is not None and not self.export_worker_thread.isRunning():
        self.export_worker_thread.deleteLater()
        self.export_worker_thread = None

def _on_preview_process_finished(self, item_index: int, processed_preview_path_str: str):
    item = self.state.get_item(item_index)
    if item is None:
        self._finalize_preview_thread()
        self._handle_post_item_completion(error=True)
        return

    processed_preview_path = Path(processed_preview_path_str)
    item.processed_preview_path = processed_preview_path
    item.edited_preview_path = None
    item.status = "processed"

    self.log(f"Preview processing complete: {processed_preview_path}")
    if processed_preview_path.exists():
        self.log(f"Processed preview size: {processed_preview_path.stat().st_size} bytes")

    self._refresh_item_display(item_index)
```

```
        self.select_item(item_index)
        self._finalize_preview_thread()
        self._handle_post_item_completion()

    def _on_preview_process_error(self, item_index: int,
error_message: str):
        item = self.state.get_item(item_index)
        if item is not None:
            item.status = "error"
            item.processed_preview_path = None
            item.edited_preview_path = None
            self._refresh_item_display(item_index)

        self.log(f"Preview processing failed for item index
{item_index}")
        self.log(f"Error: {error_message}")
        self._finalize_preview_thread()
        self._handle_post_item_completion(error=True)

    def _on_full_export_finished(self, item_index: int,
_fullres_cache_path: str, output_path_str: str):
        item = self.state.get_item(item_index)
        if item is not None:
            item.output_path = output_path_str
            item.status = "exported"
            self.log(f"Exported: {output_path_str}")
            self._refresh_item_display(item_index)
            self.select_item(item_index)

        if self.batch_mode:

            self.batch_exported_files.append(output_path_str)

        self._finalize_export_thread()
        self._handle_post_item_completion()

    def _on_full_export_error(self, item_index: int,
error_message: str):
        item = self.state.get_item(item_index)
        if item is not None:
```

```
        item.status = "error"
        self._refresh_item_display(item_index)

    self.log(f"Full export failed for item index
{item_index}")
    self.log(f"Error: {error_message}")
    self._finalize_export_thread()
    self._handle_post_item_completion(error=True)

def _handle_post_item_completion(self, error: bool =
False):
    self.current_processing_index = None

    if self.batch_mode:
        self.batch_processed_count += 1
        self._update_batch_progress()
    else:
        self.progress_bar.setRange(0, 100)
        self.progress_bar.setValue(0 if error else 100)

    self.processing_active = False

    if self.batch_mode:
        self._start_next_batch_item()
    else:
        self._set_processing_controls_enabled(True)

def _finish_processing_session(self, final_progress:
int | None = None):
    if self.batch_action == "export" and
self.settings_panel.export_zip_check.isChecked() and
self.batch_exported_files:
        output_dir = self._get_output_dir()
        if output_dir is not None:
            ok, zip_path, err =
create_batch_zip(self.batch_exported_files, output_dir)
            if ok:
                self.log(f"ZIP export created: {zip_path}")
            else:
                self.log(f"ZIP export failed: {err}")
```

```
self.batch_mode = False
self.batch_action = None
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.cancel_requested = False
self.current_processing_index = None
self.batch_exported_files = []

self._set_processing_controls_enabled(True)

self.progress_bar.setRange(0, 100)
if final_progress is not None:
    self.progress_bar.setValue(final_progress)

def clear_all_items(self):
    if self.processing_active:
        QMessageBox.information(
            self,
            "Processing",
            "Wait for the current processing job to finish
first.",
        )
    return

reply = QMessageBox.question(
    self,
    "Clear All",
    "Remove all loaded items from MyBGRemover?",
    QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No,
    QMessageBox.StandardButton.No,
)
if reply != QMessageBox.StandardButton.Yes:
    return

self.state.clear_all()
self.file_panel.clear_items()
self.preview_canvas.clear_preview()
```

```
        self.file_panel.process_checked_btn.hide()
        self.progress_bar.setRange(0, 100)
        self.progress_bar.setValue(0)
        self.batch_queue = []
        self.batch_total = 0
        self.batch_processed_count = 0
        self.batch_mode = False
        self.batch_action = None
        self.cancel_requested = False
        self.batch_exported_files = []
        self.visible_indices = []
        self._refresh_type_filter_options()

        self.log("Cleared all items.")
```

```
def closeEvent(self, event):
    self.cancel_requested = True
    self._stop_watch_folder()
    self._shutdown_running_threads()
    self.settings_store.sync()
    super().closeEvent(event)
```

settings panel:

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt
```

```
class SettingsPanel(QFrame):
```

```
def __init__(self):
    super().__init__()

    self.setObjectName("settingsPanel")
    self setFrameShape(QFrame.StyledPanel)
    self.setMinimumWidth(320)

    layout = QVBoxLayout(self)
    layout.setContentsMargins(10, 10, 10, 10)
    layout.setSpacing(8)

    title = QLabel("Settings Panel")
    title.setObjectName("panelTitle")
    layout.addWidget(title)

    self.removal_mode_label = QLabel("Removal
Mode")
    self.removal_mode_combo = QComboBox()
    self.removal_mode_combo.addItem(["AI
Removal", "Color Removal"])
    layout.addWidget(self.removal_mode_label)
    layout.addWidget(self.removal_mode_combo)

    self.model_label = QLabel("Model")
    self.model_combo = QComboBox()
    self.model_combo.addItem([
        "u2net",
        "u2netp",
        "u2net-human-seg",
        "isnet-general-use",
        "isnet-anime",
        "birefnet-general-lite",
        "birefnet-general",
        "birefnet-portrait",
        "bria-rmbg",
    ])
    self.model_combo.setCurrentText("u2netp")
    layout.addWidget(self.model_label)
    layout.addWidget(self.model_combo)
```

```
        self.output_format_label = QLabel("Output
Format")
        self.output_format_combo = QComboBox()
        self.output_format_combo.addItem(["PNG",
"JPG", "WEBP"])
        self.output_format_combo.setCurrentText("PNG")
        layout.addWidget(self.output_format_label)
        layout.addWidget(self.output_format_combo)

        self.background_mode_label =
QLabel("Background Mode")
        self.background_mode_combo = QComboBox()

        self.background_mode_combo.addItem(["Transparent"
, "Solid Color"])

        self.background_mode_combo.setCurrentText("Transpa
rent")
        layout.addWidget(self.background_mode_label)
        layout.addWidget(self.background_mode_combo)

        self.background_color_label = QLabel("Solid
Background Color")
        self.background_color_combo = QComboBox()
        self.background_color_combo.addItem(["White",
"Black"])
        layout.addWidget(self.background_color_label)
        layout.addWidget(self.background_color_combo)

        self.output_dir_label = QLabel("Output Directory")
        self.output_dir_edit = QLineEdit()
        self.output_dir_btn = QPushButton("Choose
Output Folder")
        self.open_output_btn = QPushButton("Open
Output Folder")

        output_btn_row = QHBoxLayout()
        output_btn_row.addWidget(self.output_dir_btn)
        output_btn_row.addWidget(self.open_output_btn)
```

```
        layout.addWidget(self.output_dir_label)
        layout.addWidget(self.output_dir_edit)
        layout.addLayout(output_btn_row)

        self.naming_label = QLabel("Filename Suffix")
        self.naming_edit = QLineEdit("_nobg")
        layout.addWidget(self.naming_label)
        layout.addWidget(self.naming_edit)

        self.export_zip_check = QCheckBox("Export
successful results as ZIP")
        layout.addWidget(self.export_zip_check)

        self.tool_section_label = QLabel("Editing Tools")

        self.tool_section_label.setObjectName("panelTitle")
        layout.addWidget(self.tool_section_label)

        self.tool_label = QLabel("Tool")
        self.tool_combo = QComboBox()
        self.tool_combo.addItem(["Erase", "Restore",
"Magic Erase", "Background Erase"])
        layout.addWidget(self.tool_label)
        layout.addWidget(self.tool_combo)

        self.apply_mode_label = QLabel("Apply Mode")
        self.apply_mode_combo = QComboBox()
        self.apply_mode_combo.addItem(["Brush",
"Click"])
        self.apply_mode_combo.setCurrentText("Brush")
        layout.addWidget(self.apply_mode_label)
        layout.addWidget(self.apply_mode_combo)

        self.brush_size_label = QLabel("Brush Size")
        self.brush_size_slider =
QSlider(Qt.Orientation.Horizontal)
        self.brush_size_slider.setRange(1, 200)
        self.brush_size_slider.setValue(40)
        layout.addWidget(self.brush_size_label)
        layout.addWidget(self.brush_size_slider)
```



```
        self.tolerance_label = QLabel("Tolerance")
        self.tolerance_slider =
QSlider(Qt.Orientation.Horizontal)
        self.tolerance_slider.setRange(0, 255)
        self.tolerance_slider.setValue(35)
        layout.addWidget(self.tolerance_label)
        layout.addWidget(self.tolerance_slider)

        self.phase_note = QLabel(
            "Watch folder automation can monitor a folder
and auto-load new images."
        )
        self.phase_note.setWordWrap(True)
        layout.addWidget(self.phase_note)

        self.softness_label = QLabel("Softness")
        self.softness_slider =
QSlider(Qt.Orientation.Horizontal)
        self.softness_slider.setRange(0, 100)
        self.softness_slider.setValue(35)
        layout.addWidget(self.softness_label)
        layout.addWidget(self.softness_slider)

        self.opacity_label = QLabel("Opacity")
        self.opacity_slider =
QSlider(Qt.Orientation.Horizontal)
        self.opacity_slider.setRange(1, 100)
        self.opacity_slider.setValue(100)
        layout.addWidget(self.opacity_label)
        layout.addWidget(self.opacity_slider)

        self.spacing_label = QLabel("Spacing")
        self.spacing_slider =
QSlider(Qt.Orientation.Horizontal)
        self.spacing_slider.setRange(1, 100)
        self.spacing_slider.setValue(20)
        layout.addWidget(self.spacing_label)
        layout.addWidget(self.spacing_slider)
```

```
        self.threshold_label = QLabel("Color Threshold")
        self.threshold_slider =
QSlider(Qt.Orientation.Horizontal)
        self.threshold_slider.setRange(0, 255)
        self.threshold_slider.setValue(30)
        self.pick_color_btn = QPushButton("Pick Color")

        layout.addWidget(self.threshold_label)
        layout.addWidget(self.threshold_slider)
        layout.addWidget(self.pick_color_btn)

        self.watch_section_label = QLabel("Watch Folder")

        self.watch_section_label.setObjectName("panelTitle")
        layout.addWidget(self.watch_section_label)

        self.watch_folder_edit = QLineEdit()
        self.watch_folder_btn = QPushButton("Choose
Watch Folder")
        self.watch_include_subfolders_check =
QCheckBox("Include subfolders in watch folder")
        self.watch_enable_check = QCheckBox("Enable
watch folder automation")
        self.watch_status_label = QLabel("Watch status:
Off")

        layout.addWidget(QLabel("Watch Folder Path"))
        layout.addWidget(self.watch_folder_edit)
        layout.addWidget(self.watch_folder_btn)

        layout.addWidget(self.watch_include_subfolders_check
)
        layout.addWidget(self.watch_enable_check)
        layout.addWidget(self.watch_status_label)

        self.action_section_label = QLabel("Actions")

        self.action_section_label.setObjectName("panelTitle")
        layout.addWidget(self.action_section_label)
```

```
        action_row = QHBoxLayout()

        self.action_combo = QComboBox()
        self.action_combo.addItem("Process", "Export")

        self.target_combo = QComboBox()
        self.target_combo.addItem("Checked", "All")

        self.run_action_btn = QPushButton("Run")

        action_row.addWidget(self.action_combo, 1)
        action_row.addWidget(self.target_combo, 1)
        action_row.addWidget(self.run_action_btn)

        layout.addLayout(action_row)

        self.cancel_btn = QPushButton("Cancel")
        self.cancel_btn.hide()
        layout.addWidget(self.cancel_btn)

        layout.addStretch()

    def choose_output_dir(self):
        self.output_dir_btn.clicked.connect(self.choose_output_dir)

    def choose_watch_folder(self):
        self.watch_folder_btn.clicked.connect(self.choose_watch_folder)

    def update_mode_visibility(self):
        self.removal_mode_combo.currentTextChanged.connect(self.update_mode_visibility)

    def update_background_visibility(self):
        self.background_mode_combo.currentTextChanged.connect(self.update_background_visibility)

    def update_tool_visibility(self):
        self.tool_combo.currentTextChanged.connect(self.update_tool_visibility)

    def update_mode_visibility(self):
        self.update_background_visibility()
```

```
self.update_tool_visibility()

def choose_output_dir(self):
    folder = QFileDialog.getExistingDirectory(self,
"Choose Output Folder")
    if folder:
        self.output_dir_edit.setText(folder)

def choose_watch_folder(self):
    folder = QFileDialog.getExistingDirectory(self,
"Choose Watch Folder")
    if folder:
        self.watch_folder_edit.setText(folder)

def update_mode_visibility(self):
    is_color_mode =
self.removal_mode_combo.currentText() == "Color
Removal"

    self.threshold_label.setVisible(is_color_mode)
    self.threshold_slider.setVisible(is_color_mode)
    self.pick_color_btn.setVisible(is_color_mode)

    self.model_label.setEnabled(not is_color_mode)
    self.model_combo.setEnabled(not is_color_mode)

def update_background_visibility(self):
    is_solid =
self.background_mode_combo.currentText() == "Solid
Color"

    self.background_color_label.setVisible(is_solid)
    self.background_color_combo.setVisible(is_solid)

def update_tool_visibility(self):
    tool_name = self.tool_combo.currentText()
    is_click_tool = tool_name in ("Magic Erase",
"Background Erase")

    self.tolerance_label.setVisible(is_click_tool)
    self.tolerance_slider.setVisible(is_click_tool)
```

```
self.brush_size_label.setVisible(not is_click_tool)
self.brush_size_slider.setVisible(not is_click_tool)

self.apply_mode_label.setVisible(True)
self.apply_mode_combo.setVisible(True)

self.phase_note.setVisible(True)

self.softness_label.setVisible(True)
self.softness_slider.setVisible(True)
self.opacity_label.setVisible(True)
self.opacity_slider.setVisible(True)
self.spacing_label.setVisible(True)
self.spacing_slider.setVisible(True)

self.softness_slider.setEnabled(False)
self.opacity_slider.setEnabled(False)
self.spacing_slider.setEnabled(False)

self.softness_label.setText("Softness (disabled)")
self.opacity_label.setText("Opacity (disabled)")
self.spacing_label.setText("Spacing (disabled)")

if is_click_tool:
    self.apply_mode_combo.setCurrentText("Click")
    self.apply_mode_combo.setEnabled(False)
else:

self.apply_mode_combo.setCurrentText("Brush")
    self.apply_mode_combo.setEnabled(False)
```

I have enough to do the Phase 24 cleanup safely. The main issue is in settings_panel.py ; I can simplify the panel without disturbing the working batch/thread logic in main_window.py .

Thought for 6s

Changed file:

- `ui/settings_panel.py`

No functional `main_window.py` change is required for this cleanup step.

One important thing to notice in the code you pasted:

- your `closeEvent` indentation is wrong in the pasted `main_window.py`
- it currently appears outside the class
- if that is also true in your real file, it should be fixed separately
- I am **not** changing it in this step because this step is focused on settings panel cleanup only

What this cleanup does

This version of `ui/settings_panel.py` keeps the current working behavior, but makes the UI less confusing by:

- hiding unfinished controls instead of showing disabled ones
- hiding color-removal controls unless `Color Removal` is selected
- hiding watch-folder controls by default, since that area was marked as not fully reliable yet
- adding model tooltips/descriptions
- adding clearer tooltips for major controls
- keeping:
 - `Action`
 - `Target`
 - `Run`
 - `Cancel`
 - model selection
 - output settings
 - edit tool basics
- preserving the widget names that `main_window.py` already uses, so you do **not** need to rewire that file for this step

Full updated file: `ui/settings_panel.py`

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


What changed

1. Unfinished controls are now hidden

These are still created internally so `main_window.py` can safely keep reading them, but the user will no longer see them:

- Softness
- Opacity
- Spacing

This is safer than deleting them right now.

2. Watch Folder UI is hidden

The watch-folder widgets still exist, so your current `main_window.py` signal connections do not break, but the section is hidden to reduce clutter until that feature is ready.

3. Model descriptions were added

Now the user gets:

- a tooltip on the model dropdown
- a short description label under the model combo

4. Panel wording is clearer

Tooltips were added to reduce confusion around:

- Process vs Export
- Target selection
- Filename suffix
- Tool behavior

No `main_window.py` code changes needed for this step

Because I kept the hidden controls alive, these existing lines can stay as-is:

- `softness_slider.valueChanged.connect(...)`
- `opacity_slider.valueChanged.connect(...)`
- `spacing_slider.valueChanged.connect(...)`
- watch-folder signal hookups
- `_sync_preview_tool_settings()`

That avoids risking the working thread/process/export flow.

How to apply this

Step 1

Open:

```
Python                                                                    Run
ui/settings_panel.py
```

Step 2

Replace the **entire file** with the version above.

Step 3

Save the file.

How to run

From your project folder:

```
Bash
cd /home/yma/MyBGRemover
python3 main.py
```

If you use your virtual environment, activate it first with **your actual environment**

command.

Example only:

Bash

```
source /home/yma/MyBGRemover/venv/bin/activate  
python3 main.py
```

What to test

Test 1: app still opens

Confirm the window opens with no syntax or indentation errors.

Test 2: settings panel is cleaner

Confirm these are **hidden**:

- Softness
- Opacity
- Spacing
- Watch Folder section

Test 3: working controls still exist

Confirm these still show and work:

- Removal Mode
- Model
- Output Format
- Background Mode
- Output Directory
- Filename Suffix
- Tool
- Apply Mode
- Brush Size
- Action
- Target

- Run
- Cancel

Test 4: model hint updates

Change the model dropdown and confirm the description text under it changes.

Test 5: cancel behavior still works

Start a batch process/export and confirm:

- Run becomes disabled
- Cancel becomes visible
- after completion, Cancel hides again

Test 6: processing/export still works

Run:

- Process + Checked
- Export + Checked

Confirm the batch still runs normally.

One important note about your pasted `main_window.py`

At the very end, you pasted:

Python	Run
<pre>def closeEvent(self, event): self.cancel_requested = True self._stop_watch_folder() self._shutdown_running_threads() self.settings_store.sync() super().closeEvent(event)</pre>	

That appears to be outside the class because of indentation.

Inside the class, it should look like this shape:

Python	Run
--------	-----

```
def closeEvent(self, event):  
    self.cancel_requested = True  
    self._stop_watch_folder()  
    self._shutdown_running_threads()  
    self.settings_store.sync()  
    super().closeEvent(event)
```

I did not include that as a formal file change in this step, but if your real file matches the pasted version, that needs to be corrected.

Next best step after this is to clean up `main_window.py` signal hookups so hidden legacy controls are no longer part of the active logic.



i dont need directions on how to apply the changed code or how to run the project. i only need to know what to test.

so first, i want the tooltips/description to be shown when hovering over the specific model in the drop down as well.

second, when i click "run" with process and checked selected, it runs the images that are selected (the text is highlighted, NOT checked) and then ones that are checked. i want it to run the images checked, and i want to make it easier to check them, by double clicking on an image. and in the actions panel, where "checked" is an option i want to change it to "selected". and when running processes and exports, i want it to scroll to the bottom automatically so that the "cancel" button is instantly visible

also, the thumbnail panel in filepanel extends behind the buttons below it. i want it to stop above the buttons.

and in file panel, lets remove "process checked", "remove checked", and "uncheck exported" altogether.

and, in the settings panel, for the filename suffix, i want the suffix to include the model that was used to remove.

I can cover the settings and batch-selection behavior now from the code you pasted. The thumbnail panel overlap and removing the extra file-panel buttons depend on `ui/file_panel.py`, so I need that file to give you the required full-file replacement for it.

Thought for 32s >

Below are the changes I can give you now from the files you pasted, plus the one item that is already working.

Files changed now

- `ui/settings_panel.py`
- `ui/main_window.py`

File still needed before I can finish the rest

- `ui/file_panel.py`

I need `ui/file_panel.py` for these remaining requested changes:

- double-click an image to check/uncheck it
- stop the thumbnail/file view from extending behind the bottom buttons
- remove:
 - Process Checked
 - Remove Checked
 - Uncheck Exported
- fix the root cause of highlighted-row vs checked-item confusion, if that behavior is coming from `FilePanel`

Important note: filename suffix already includes the model

Your current `main_window.py` already does this in `_build_export_suffix()` :

Python

Run

```
return f"_{model_name}{user_suffix}"
```

So if the model is `u2netp` and the suffix is `_nobg` , the export name already becomes:

Python

Run

```
_originalname_u2netp_nobg
```

That part does **not** need a change unless you want a different naming format.

1) Full updated file: `ui/settings_panel.py`

This version does these things:

- changes Action target text from `Checked` to `Selected`
- keeps backend compatibility by only changing the displayed text
- adds per-model hover tooltips inside the dropdown items
- keeps the model description label too
- keeps the hidden unfinished controls from the previous cleanup

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially better",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


2) ui/main_window.py changes

A. In run_selected_action()

Replace this:

Python

Run

```
def run_selected_action(self):
    action = self.settings_panel.action_combo.currentText()
    target = self.settings_panel.target_combo.currentText()

    if action == "Process":
        if target == "Checked":
            self.process_checked_items()
        else:
            self.process_all_items()
    else:
        if target == "Checked":
            self.export_checked_items()
        else:
            self.export_all_items()
```

With this:

Python

Run

```
def run_selected_action(self):
    action = self.settings_panel.action_combo.currentText()
    target = self.settings_panel.target_combo.currentText()

    if action == "Process":
        if target == "Selected":
            self.process_checked_items()
        else:
            self.process_all_items()
    else:
        if target == "Selected":
            self.export_checked_items()
        else:
            self.export_all_items()
```

B. Add this new helper function

Add this function anywhere inside the `MainWindow` class, for example right below `_set_processing_controls_enabled()`:

Python

Run

```
def _scroll_settings_to_bottom(self):
    if hasattr(self, "settings_scroll") and self.settings_scroll is not None:
        bar = self.settings_scroll.verticalScrollBar()
        if bar is not None:
            bar.setValue(bar.maximum())
```

C. In `_set_processing_controls_enabled()`

Replace this:

Python

Run

```
def _set_processing_controls_enabled(self, enabled: bool):
    self.settings_panel.run_action_btn.setEnabled(enabled)
    self.settings_panel.cancel_btn.setVisible(not enabled)
    self.settings_panel.cancel_btn.setEnabled(not enabled)

    if hasattr(self.file_panel, "process_checked_btn"):
        self.file_panel.process_checked_btn.setEnabled(enabled)
```

With this:

Python

Run

```
def _set_processing_controls_enabled(self, enabled: bool):
    self.settings_panel.run_action_btn.setEnabled(enabled)
    self.settings_panel.cancel_btn.setVisible(not enabled)
    self.settings_panel.cancel_btn.setEnabled(not enabled)

    if not enabled:
        self._scroll_settings_to_bottom()

    if hasattr(self.file_panel, "process_checked_btn"):
        self.file_panel.process_checked_btn.setEnabled(enabled)
```

That makes the settings panel jump to the bottom as soon as the job starts, so the Cancel button is immediately visible.

D. Update the wording in process_checked_items()

Replace this:

Python

Run

```
QMessageBox.information(self, "Nothing Checked", "No checked f
```

With this:

Python

Run

```
QMessageBox.information(self, "Nothing Selected", "No selected
```

Replace this:

Python

Run

```
"The checked files do not have valid preview data to proce
```

With this:

Python

Run

```
"The selected files do not have valid preview data to proc
```

Replace this:

Python

Run

```
self.log(f"Processing {self.batch_total} checked image(s).")
```

With this:

Python

Run

```
self.log(f"Processing {self.batch_total} selected image(s).")
```

E. Update the wording in `export_checked_items()`

Replace this:

Python

Run

```
QMessageBox.information(self, "Nothing Checked", "No checked f
```

With this:

Python

Run

```
QMessageBox.information(self, "Nothing Selected", "No selected
```

Replace this:

Python

Run

```
if not self.batch_queue:  
    QMessageBox.information(self, "No Valid Images", "The checked  
    return
```

With this:

Python

Run

```
if not self.batch_queue:
    QMessageBox.information(self, "No Valid Images", "The selected
return
```

Replace this:

Python

Run

```
self.log(f"Exporting {self.batch_total} checked image(s).")
```

With this:

Python

Run

```
self.log(f"Exporting {self.batch_total} selected image(s).")
```

What I still need from ui/file_panel.py

To finish the rest exactly the way you want, send the current contents of:

Python

Run

```
ui/file_panel.py
```

That will let me give you the full updated file for these remaining changes:

- double-click item = check/uncheck item
- remove Process Checked
- remove Remove Checked
- remove Uncheck Exported
- fix the list/thumbnail area so it stops above the bottom buttons
- verify whether highlighted rows are accidentally being treated as checked rows

What to test now

Model tooltip behavior

- open the model dropdown
- hover over each model entry
- confirm each model shows its own tooltip/description
- confirm the description label under the dropdown also updates when a model is selected

Action target text

- confirm the target dropdown now shows:
 - Selected
 - All

Run behavior with Selected

- choose Process
- choose Selected
- with no checked items, confirm it says no selected files
- with checked items, confirm it runs those items

Auto-scroll to Cancel

- start a process or export job
- confirm the settings panel scrolls to the bottom automatically
- confirm the `Cancel` button becomes immediately visible

Filename suffix behavior

- export one image
- confirm the output filename includes:
 - the model name
 - your suffix text

Example expected pattern:

Python

Run

`originalname_u2netp_nobg.png`

Send `ui/file_panel.py` next and I will finish the remaining file-panel changes in one

pass.



```
file panel:from PySide6.QtCore import Qt, QSize,
Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()

    process_checked_requested = Signal()
    remove_checked_requested = Signal()
    uncheck_completed_requested = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
```

```
self.setFrameShape(QFrame.StyledPanel)
self.setMinimumWidth(320)

self.current_view_mode = "thumbnail"

layout = QVBoxLayout(self)
layout.setContentsMargins(10, 10, 10, 10)
layout.setSpacing(8)

title = QLabel("Files")
title.setObjectName("panelTitle")
layout.addWidget(title)

self.include_subfolders_check =
QCheckBox("Include subfolders on drag/drop")
layout.addWidget(self.include_subfolders_check)

controls_row = QHBoxLayout()
controls_row.setSpacing(6)

self.sort_combo = QComboBox()
self.sort_combo.addItem(["Sort by", "Date
Added", "Name", "Type", "Status"])
self.sort_combo.setCurrentText("Sort by")

self.status_filter_combo = QComboBox()
self.status_filter_combo.addItem(
    ["Filter status", "All", "pending", "processing",
"processed", "edited", "exported", "error"]
)
self.status_filter_combo.setCurrentText("Filter
status")

self.type_filter_combo = QComboBox()
self.type_filter_combo.addItem(["Filter type", "All
Types"])
self.type_filter_combo.setCurrentText("Filter
type")

controls_row.addWidget(self.sort_combo, 1)
```

```
        controls_row.addWidget(self.status_filter_combo,
1)
        controls_row.addWidget(self.type_filter_combo,
1)
        layout.addLayout(controls_row)

        top_actions = QHBoxLayout()
        top_actions.setSpacing(6)

        self.check_all_box = QCheckBox("Check All
Visible")

        self.thumbnail_view_btn = QPushButton("🖼️")
        self.thumbnail_view_btn.setCheckable(True)
        self.thumbnail_view_btn.setFixedSize(32, 32)
        self.thumbnail_view_btn.setToolTip("Thumbnail
View")

        self.list_view_btn = QPushButton("☰")
        self.list_view_btn.setCheckable(True)
        self.list_view_btn.setFixedSize(32, 32)
        self.list_view_btn.setToolTip("List View")

        top_actions.addWidget(self.check_all_box)
        top_actions.addStretch()
        top_actions.addWidget(self.thumbnail_view_btn)
        top_actions.addWidget(self.list_view_btn)
        layout.addLayout(top_actions)

        counts_row = QHBoxLayout()
        counts_row.setSpacing(12)

        self.visible_count_label = QLabel("Visible: 0")
        self.checked_count_label = QLabel("Checked: 0")

        counts_row.addWidget(self.visible_count_label)
        counts_row.addWidget(self.checked_count_label)
        counts_row.addStretch()
        layout.addLayout(counts_row)
```

```
self.file_view = QListWidget()

self.file_view.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
self.file_view.setAlternatingRowColors(False)
self.file_view.setSpacing(10)
self.file_view.setUniformItemSizes(False)

self.file_view.setResizeMode(QListWidget.ResizeMode.Adjust)

self.file_view.setMovement(QListWidget.Movement.Static)
self.file_view.setWordWrap(True)
self.file_view.setMinimumHeight(420)
layout.addWidget(self.file_view, 1)

checked_row = QHBoxLayout()
checked_row.setSpacing(6)

self.process_checked_btn =
QPushButton("Process Checked")
self.remove_checked_btn = QPushButton("Remove Checked")
self.uncheck_completed_btn =
QPushButton("Uncheck Exported")

checked_row.addWidget(self.process_checked_btn)

checked_row.addWidget(self.remove_checked_btn)

checked_row.addWidget(self.uncheck_completed_btn)
layout.addLayout(checked_row)

action_row_1 = QHBoxLayout()
self.remove_selected_btn =
QPushButton("Remove Selected")
self.clear_completed_btn = QPushButton("Clear Exported")
```

```
        action_row_1.addWidget(self.remove_selected_btn)

        action_row_1.addWidget(self.clear_completed_btn)
        layout.addLayout(action_row_1)

        action_row_2 = QHBoxLayout()
        self.clear_all_btn = QPushButton("Clear All")
        self.retry_failed_btn = QPushButton("Retry Failed")
        action_row_2.addWidget(self.clear_all_btn)
        action_row_2.addWidget(self.retry_failed_btn)
        layout.addLayout(action_row_2)

        self.open_output_btn = QPushButton("Open Output Folder")
        layout.addWidget(self.open_output_btn)

        self.file_view.currentRowChanged.connect(self._emit_selected_item)

        self.file_view.itemChanged.connect(self._on_item_changed)

        self.sort_combo.currentTextChanged.connect(self._emit_sort_changed)

        self.status_filter_combo.currentTextChanged.connect(self._emit_status_filter_changed)

        self.type_filter_combo.currentTextChanged.connect(self._emit_type_filter_changed)

        self.thumbnail_view_btn.clicked.connect(lambda: self.set_view_mode("thumbnail"))
        self.list_view_btn.clicked.connect(lambda: self.set_view_mode("list"))
```

```
self.check_all_box.toggled.connect(self._toggle_all_visible)

self.process_checked_btn.clicked.connect(self.process_checked_requested)

self.remove_checked_btn.clicked.connect(self.remove_checked_requested)

self.uncheck_completed_btn.clicked.connect(self.uncheck_completed_requested)

    self._update_view_buttons()
    self._apply_view_mode()
    self._update_counts()

def set_type_filter_options(self, file_types: list[str]):
    current = self.type_filter_combo.currentText()

    self.type_filter_combo.blockSignals(True)
    self.type_filter_combo.clear()
    self.type_filter_combo.addItem("Filter type")
    self.type_filter_combo.addItem("All Types")
    self.type_filter_combo.addItems(file_types)

    if current in ["Filter type", "All Types", *file_types]:
        self.type_filter_combo.setCurrentText(current)
    else:
        self.type_filter_combo.setCurrentText("Filter type")

    self.type_filter_combo.blockSignals(False)

def set_view_mode(self, mode: str):
    mode = "thumbnail" if mode == "thumbnail" else "list"
    if self.current_view_mode == mode:
        return
```



```
        self.current_view_mode = mode
        self._update_view_buttons()
        self._apply_view_mode()
        self.view_mode_changed.emit(mode)

    def _update_view_buttons(self):
        is_thumbnail = self.current_view_mode ==
"thumbnail"
        self.thumbnail_view_btn.setChecked(is_thumbnail)
        self.list_view_btn.setChecked(not is_thumbnail)

    def _apply_view_mode(self):
        if self.current_view_mode == "thumbnail":

self.file_view.setViewMode(QListWidget.ViewMode.Ico
nMode)

        self.file_view.setFlow(QListWidget.Flow.LeftToRight)
        self.file_view.setWrapping(True)
        self.file_view.setGridSize(QSize(170, 200))
        self.file_view.setIconSize(QSize(132, 132))
        else:

self.file_view.setViewMode(QListWidget.ViewMode.Lis
tMode)

        self.file_view.setFlow(QListWidget.Flow.TopToBottom
)
        self.file_view.setWrapping(False)
        self.file_view.setGridSize(QSize())
        self.file_view.setIconSize(QSize(56, 56))

    def clear_items(self):
        self.file_view.clear()
        self._sync_master_checkbox()
        self._update_counts()

    def add_item(self, text: str, thumbnail_path: str |
None, user_index: int, checked: bool = False):
        item = QListWidgetItem()
```

```
        item.setData(Qt.ItemDataRole.UserRole,
user_index)
        item.setFlags(
            item.flags()
            | Qt.ItemFlag.ItemIsUserCheckable
            | Qt.ItemFlag.ItemIsSelectable
            | Qt.ItemFlag.ItemIsEnabled
        )
        item.setCheckState(Qt.CheckState.Checked if
checked else Qt.CheckState.Unchecked)

        if self.current_view_mode == "thumbnail":
            display_text = text.split("\n")[0].strip()
            item.setText(display_text)

        item.setTextAlignment(Qt.AlignmentFlag.AlignHCenter
| Qt.AlignmentFlag.AlignTop)
        item.setSizeHint(QSize(158, 190))
    else:
        item.setText(text)
        item.setSizeHint(QSize(0, 76))

    if thumbnail_path:
        thumb = QPixmap(thumbnail_path)
        if not thumb.isNull():
            if self.current_view_mode == "thumbnail":
                scaled = thumb.scaled(
                    132,
                    132,
                    Qt.AspectRatioMode.KeepAspectRatio,

Qt.TransformationMode.SmoothTransformation,
                )
                item.setIcon(QIcon(scaled))
            else:
                scaled = thumb.scaled(
                    56,
                    56,
                    Qt.AspectRatioMode.KeepAspectRatio,
```

```
Qt.TransformationMode.SmoothTransformation,
    )
    item.setIcon(QIcon(scaled))

    self.file_view.addItem(item)
    self._sync_master_checkbox()
    self._update_counts()

def set_current_visible_row(self, row: int):
    if 0 <= row < self.file_view.count():
        self.file_view.setCurrentRow(row)

def get_selected_user_index(self) -> int:
    item = self.file_view.currentItem()
    if item is None:
        return -1

    data = item.data(Qt.ItemDataRole.UserRole)
    return int(data) if data is not None else -1

def get_checked_user_indices(self) -> list[int]:
    checked = []
    for row in range(self.file_view.count()):
        item = self.file_view.item(row)
        if item is not None and item.checkState() ==
Qt.CheckState.Checked:
            data = item.data(Qt.ItemDataRole.UserRole)
            if data is not None:
                checked.append(int(data))
    return checked

def set_checked_user_indices(self, indices: set[int]):
    self.file_view.blockSignals(True)
    for row in range(self.file_view.count()):
        item = self.file_view.item(row)
        if item is None:
            continue
        user_index =
item.data(Qt.ItemDataRole.UserRole)
        item.setCheckState(
```

```
        Qt.CheckState.Checked if user_index in
indices else Qt.CheckState.Unchecked
    )
    self.file_view.blockSignals(False)
    self._sync_master_checkbox()
    self._update_counts()
    self.checked_items_changed.emit()

def _toggle_all_visible(self, checked: bool):
    self.file_view.blockSignals(True)
    for row in range(self.file_view.count()):
        item = self.file_view.item(row)
        if item is not None:
            item.setCheckState(Qt.CheckState.Checked if
checked else Qt.CheckState.Unchecked)
    self.file_view.blockSignals(False)
    self._update_counts()
    self.checked_items_changed.emit()

def _sync_master_checkbox(self):
    total = self.file_view.count()
    checked_count = 0

    for row in range(total):
        item = self.file_view.item(row)
        if item is not None and item.checkState() ==
Qt.CheckState.Checked:
            checked_count += 1

    self.check_all_box.blockSignals(True)
    if total == 0 or checked_count == 0:

self.check_all_box.setCheckState(Qt.CheckState.Unche
cked)
        elif checked_count == total:

self.check_all_box.setCheckState(Qt.CheckState.Check
ed)
    else:
```

```
self.check_all_box.setCheckState(Qt.CheckState.PartiallyChecked)
    self.check_all_box.blockSignals(False)

def _update_counts(self):
    visible_count = self.file_view.count()
    checked_count =
len(self.get_checked_user_indices())
    self.visible_count_label.setText(f"Visible:
{visible_count}")
    self.checked_count_label.setText(f"Checked:
{checked_count}")

def _emit_selected_item(self, row: int):
    if row < 0:
        return
    self.item_selected.emit(row)

def _on_item_changed(self, _item:
QListWidgetItem):
    self._sync_master_checkbox()
    self._update_counts()
    self.checked_items_changed.emit()

def _emit_sort_changed(self, text: str):
    if text == "Sort by":
        return
    self.sort_changed.emit(text)

def _emit_status_filter_changed(self, text: str):
    if text == "Filter status":
        self.status_filter_changed.emit("All")
        return
    self.status_filter_changed.emit(text)

def _emit_type_filter_changed(self, text: str):
    if text == "Filter type":
        self.type_filter_changed.emit("All Types")
        return
    self.type_filter_changed.emit(text)
```

I have the remaining file-panel changes now. The main fixes are: remove the extra checked-action buttons, make double-click toggle the checkmark, and stop the file list from crowding the lower button area.

Thought for 21s >

Changed files:

- `ui/settings_panel.py`
- `ui/file_panel.py`
- `ui/main_window.py`

Full updated file: `ui/settings_panel.py`

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


Full updated file: `ui/file_panel.py`

Python

Run

```
from PySide6.QtCore import Qt, QSize, Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
    QSizePolicy,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.current_view_mode = "thumbnail"

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Files")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.include_subfolders_check = QCheckBox("Include subfolders on d
        layout.addWidget(self.include_subfolders_check)

        controls_row = QHBoxLayout()
        controls_row.setSpacing(6)
```


ui/main_window.py changes

1) In _connect_signals()

Before

Python

Run

```
def _connect_signals(self):
    self.file_panel.item_selected.connect(self.select_item_from_panel)
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
    self.file_panel.status_filter_changed.connect(self.rebuild_file_pa
    self.file_panel.type_filter_changed.connect(self.rebuild_file_pane
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)

    self.file_panel.remove_selected_btn.clicked.connect(self.remove_se
    self.file_panel.clear_completed_btn.clicked.connect(self.clear_com
    self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items
    self.file_panel.retry_failed_btn.clicked.connect(self.retry_failed
    self.file_panel.open_output_btn.clicked.connect(self.open_output_f

    self.file_panel.process_checked_requested.connect(self.process_che
    self.file_panel.remove_checked_requested.connect(self.remove_check
    self.file_panel.uncheck_completed_requested.connect(self.uncheck_c

    self.settings_panel.run_action_btn.clicked.connect(self.run_select
    self.settings_panel.cancel_btn.clicked.connect(self.request_cancel
```

After

Python

Run

```
def _connect_signals(self):
    self.file_panel.item_selected.connect(self.select_item_from_panel)
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
    self.file_panel.status_filter_changed.connect(self.rebuild_file_pa
    self.file_panel.type_filter_changed.connect(self.rebuild_file_pane
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)

    self.file_panel.remove_selected_btn.clicked.connect(self.remove_se
    self.file_panel.clear_completed_btn.clicked.connect(self.clear_com
    self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items
    self.file_panel.retry_failed_btn.clicked.connect(self.retry_failed
    self.file_panel.open_output_btn.clicked.connect(self.open_output_f

    self.settings_panel.run_action_btn.clicked.connect(self.run_select
    self.settings_panel.cancel_btn.clicked.connect(self.request_cancel
```

2) In run_selected_action()

Before

Python

Run

```
def run_selected_action(self):
    action = self.settings_panel.action_combo.currentText()
    target = self.settings_panel.target_combo.currentText()

    if action == "Process":
        if target == "Checked":
            self.process_checked_items()
        else:
            self.process_all_items()
    else:
        if target == "Checked":
            self.export_checked_items()
        else:
            self.export_all_items()
```

After

Python

Run

```
def run_selected_action(self):
    action = self.settings_panel.action_combo.currentText()
    target = self.settings_panel.target_combo.currentText()

    if action == "Process":
        if target == "Selected":
            self.process_checked_items()
        else:
            self.process_all_items()
    else:
        if target == "Selected":
            self.export_checked_items()
        else:
            self.export_all_items()
```

3) Add this new function inside the MainWindow class

Add it directly below `_set_processing_controls_enabled()`.

Python

Run

```
def _scroll_settings_to_bottom(self):
    if hasattr(self, "settings_scroll") and self.settings_scroll is not None:
        bar = self.settings_scroll.verticalScrollBar()
        if bar is not None:
            bar.setValue(bar.maximum())
```

4) In `_set_processing_controls_enabled()`

Before

Python

Run


```
def _set_processing_controls_enabled(self, enabled: bool):
    self.settings_panel.run_action_btn.setEnabled(enabled)
    self.settings_panel.cancel_btn.setVisible(not enabled)
    self.settings_panel.cancel_btn.setEnabled(not enabled)

    if hasattr(self.file_panel, "process_checked_btn"):
        self.file_panel.process_checked_btn.setEnabled(enabled)
```

After

Python

Run

```
def _set_processing_controls_enabled(self, enabled: bool):
    self.settings_panel.run_action_btn.setEnabled(enabled)
    self.settings_panel.cancel_btn.setVisible(not enabled)
    self.settings_panel.cancel_btn.setEnabled(not enabled)

    if not enabled:
        self._scroll_settings_to_bottom()
```

5) In clear_all_items()

Before

Python

Run

```
self.file_panel.process_checked_btn.hide()
```

After

Delete that line completely.

So that section becomes:

Python

Run

```
self.state.clear_all()
self.file_panel.clear_items()
self.preview_canvas.clear_preview()
self.progress_bar.setRange(0, 100)
self.progress_bar.setValue(0)
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_mode = False
self.batch_action = None
self.cancel_requested = False
self.batch_exported_files = []
self.visible_indices = []
self._refresh_type_filter_options()
```

6) In process_checked_items()

Replace these text lines.

Before

Python

Run

```
QMessageBox.information(self, "Nothing Checked", "No checked f
```

After

Python

Run

```
QMessageBox.information(self, "Nothing Selected", "No selected
```

Before

Python

Run

```
"The checked files do not have valid preview data to proce
```

After

Python

Run

```
"The selected files do not have valid preview data to proc
```

Before

Python

Run

```
self.log(f"Processing {self.batch_total} checked image(s).")
```

After

Python

Run

```
self.log(f"Processing {self.batch_total} selected image(s).")
```

7) In export_checked_items()

Replace these text lines.

Before

Python

Run

```
QMessageBox.information(self, "Nothing Checked", "No checked f
```

After

Python

Run

```
QMessageBox.information(self, "Nothing Selected", "No selected
```

Before

Python

Run

```
if not self.batch_queue:  
    QMessageBox.information(self, "No Valid Images", "The checked  
    return
```

After

Python

Run

```
if not self.batch_queue:
    QMessageBox.information(self, "No Valid Images", "The selected
return
```

Before

Python

Run

```
self.log(f"Exporting {self.batch_total} checked image(s).")
```

After

Python

Run

```
self.log(f"Exporting {self.batch_total} selected image(s).")
```

What changed

Settings panel

- target label now says Selected instead of Checked
- model descriptions now appear both:
 - under the model dropdown
 - when hovering each specific model inside the dropdown list

File panel

- removed:
 - Process Checked
 - Remove Checked
 - Uncheck Exported
- double-click now toggles check/uncheck
- selected count label now says Selected
- file view is set up to size more cleanly above the lower buttons

Main window

- action routing now uses `Selected`
- removed connections to deleted file-panel buttons
- removed old hide call for deleted button
- when a job starts, settings panel scrolls to the bottom so `Cancel` is visible immediately

What to test

1) Model hover tooltips

- open the **Model** dropdown
- move the mouse over each model name in the opened list
- confirm a different description appears for each model
- select a model and confirm the description label under the dropdown also updates

2) Selected target wording

- open the **Target** dropdown in the settings panel
- confirm the options are now:
 - `Selected`
 - `All`

3) Double-click check/uncheck

- in the file panel, double-click one image
- confirm it gains a checkmark
- double-click it again
- confirm the checkmark is removed
- confirm the text highlight/selection can still move independently

4) Run uses checked items only

- check two images
- highlight a different image that is **not** checked
- set:
 - `Action = Process`
 - `Target = Selected`
- click **Run**

- confirm only the checked images are processed
- repeat the same test with `Export`

5) No old extra checked-action buttons

- in the file panel, confirm these are gone:
 - `Process Checked`
 - `Remove Checked`
 - `Uncheck Exported`

6) File panel bottom layout

- switch to thumbnail view
- load enough images to fill the panel
- resize the window shorter and taller
- confirm the thumbnail/list area stops above the lower buttons instead of visually extending behind them

7) Cancel button auto-scroll

- scroll the settings panel upward so the bottom is not visible
- start a process or export job
- confirm the settings panel automatically scrolls down
- confirm the **Cancel** button is immediately visible

8) Selected counter

- check and uncheck items
- confirm the count label updates as:
 - `Selected: 0`
 - `Selected: 1`
 - etc.

9) Filename suffix still includes the model

- export an image
- confirm the output filename includes:
 - the model name
 - the suffix text

Example pattern:

`yourimage_u2netp_nobg.png`

10) Regression check

- select an item normally with one click
- confirm preview switching still works
- confirm removing selected still works
- confirm clear exported / clear all / retry failed still work



ok all features work so far. i want to make a desktop icon that i can click to open up the app rather than doing it in terminal

Thought for 5s

Yes. On Linux Mint, the usual way is to create a `.desktop` launcher.

Use this version, adjusted for your project.

1) Create the launcher file

Run:

```
Bash
```

```
nano /home/yma/Desktop/MyBGRemover.desktop
```

Paste this into it:

```
INI
```

```
[Desktop Entry]
Version=1.0
Type=Application
Name=MyBGRemover
Comment=Open the MyBGRemover app
Exec=bash -c 'cd /home/yma/MyBGRemover && /home/yma/MyBGRemover/venv/bin/p
Icon=/home/yma/MyBGRemover/icon.png
Terminal=false
Categories=Graphics;
StartupNotify=true
Path=/home/yma/MyBGRemover
```

2) Make it executable

Run:

Bash

```
chmod +x /home/yma/Desktop/MyBGRemover.desktop
```

3) If Linux Mint warns that it is not trusted

Right-click the desktop icon and choose:

- **Allow Launching**

If you do not see that, run this too:

Bash

```
gio set /home/yma/Desktop/MyBGRemover.desktop metadata::trusted true
```

Then log out and back in if needed.

4) If you do not have an icon file yet

The line below expects this file to exist:

INI

```
Icon=/home/yma/MyBGRemover/icon.png
```

If you do **not** have that file yet, either:

Option A: temporarily remove the icon line

Delete this line from the launcher:

```
INI
```

```
Icon=/home/yma/MyBGRemover/icon.png
```

Option B: use a different image file you already have

Example:

```
INI
```

```
Icon=/home/yma/Pictures/mybgremover.png
```

5) If your virtual environment path is different

This line assumes your venv Python is here:

```
INI
```

```
Exec=bash -c 'cd /home/yma/MyBGRemover && /home/yma/MyBGRemover/venv/bin/p
```

If your venv folder has a different name, replace:

```
/home/yma/MyBGRemover/venv/bin/python
```

with your real path.

For example, if the venv were named `myenv`, it would be:

```
INI
```

```
Exec=bash -c 'cd /home/yma/MyBGRemover && /home/yma/MyBGRemover/myenv/bin/
```

What to test

- double-click the desktop icon
- confirm the app opens without a terminal window
- confirm it opens your current MyBGRemover project
- if it does nothing, right-click it and check whether Linux Mint still wants **Allow Launching**

- if it opens but fails, the most likely cause is the Python path in the `Exec=` line

If you want, I can also give you a version that adds it to the Mint app menu, not just the desktop.



ok it works but it doesnt show up in application launcher

Thought for a second

Put the launcher in the applications folder too.

Do this

Run:

Bash

```
mkdir -p /home/yma/.local/share/applications
cp /home/yma/Desktop/MyBGRemover.desktop /home/yma/.local/share/applicatio
```

Then refresh permissions/trust:

Bash

```
chmod +x /home/yma/.local/share/applications/MyBGRemover.desktop
gio set /home/yma/.local/share/applications/MyBGRemover.desktop metadata::
```

If it still does not appear

Sometimes Mint menu launchers need a valid icon or a menu update. Edit this file:

Bash

```
nano /home/yma/.local/share/applications/MyBGRemover.desktop
```

Use this version:

INI

```
[Desktop Entry]
Version=1.0
Type=Application
Name=MyBGRemover
Comment=Open the MyBGRemover app
Exec=bash -c 'cd /home/yma/MyBGRemover && /home/yma/MyBGRemover/venv/bin/p
Path=/home/yma/MyBGRemover
Icon=accessories-image-viewer
Terminal=false
Categories=Graphics;
StartupNotify=true
```

Using `Icon=accessories-image-viewer` avoids problems from a missing custom icon file.

Then run:

Bash

```
update-desktop-database /home/yma/.local/share/applications
```

Then either:

- log out and back in, or
- open the menu and search for `MyBGRemover`

What to test

- open the application launcher/menu
- search for `MyBGRemover`
- confirm it appears under Graphics or in search results
- click it from the launcher and confirm the app opens

If it still does not show, send me the exact contents of:

Bash

```
cat /home/yma/.local/share/applications/MyBGRemover.desktop
```



ok that worked. whats the next step now

Thought for a couple of seconds

Next step should be **Phase 25: polish and simplify the active workflow.**

You have the core app working well enough now:

- loading works
- preview/edit works
- process/export split works
- batch flow works
- selection/checking works
- desktop launcher works

So the best next step is not a big rewrite. It is to make the current workflow clearer and harder to break.

Best next coding targets

1. Improve status/log messages

Make the log easier to understand by showing clearer messages such as:

- when a batch starts
- which file is currently running
- which model is being used
- when a file finishes
- when a file fails
- final batch summary like:
 - processed: 8
 - exported: 8
 - failed: 1

This is a high-value improvement and low risk.

2. Make model behavior clearer in the UI

Now that you have multiple models, the next polish step is:

- show a warning for heavier/slower models
- possibly mark recommended/default models
- maybe auto-suggest:
 - portraits → `u2net_human_seg` or `birefnnet-portrait`
 - anime → `isnet-anime`
 - general → `u2netp` or `isnet-general-use`

3. Clean up hidden/unfinished logic

Right now some unfinished controls are hidden but still exist in code. A safe cleanup phase would be:

- remove unused signal hookups for features you are postponing
- remove dead UI logic that is no longer intended for current use
- keep the codebase easier to maintain

4. Add a small confirmation/summary area

A simple label in the settings or log area could show:

- current action
- current target
- selected model
- export format
- output folder

That reduces confusion before clicking Run.

5. Stress-test export parity

This is the most practical validation step:

test whether export behaves correctly for:

- no edits, export directly
- edits before processing, then export
- process first, then edit, then export
- checked batch export
- all export
- multiple models

This may reveal the next real bug worth fixing.

Best immediate next step

The strongest next step is:

Add a batch summary and clearer per-file log messages in `ui/main_window.py`.

That gives you:

- better visibility
- easier debugging
- easier testing of the multiple-model setup

What I recommend after that

After logging polish, the next likely step should be:

validate each model one by one and improve user messaging for slow/heavy models

That is probably the most useful thing before adding new advanced features.

If you want, the next thing I can do is help you add a clean batch summary and better logging in `main_window.py`.



k

I am moving to the logging/batch-summary cleanup in `main_window.py`. This is a contained change and should not disturb the working process/export split.

Thought for 7s

Changed file:

- `ui/main_window.py`

This step adds:

- clearer batch start messages
- clearer per-file progress messages
- success/failure counters
- final batch summary

What changed

New behavior

You will now get clearer log output for:

- batch start
- each item starting
- each item succeeding
- each item failing
- final summary at the end of the batch

New counters

The window will now track:

- `batch_success_count`
- `batch_error_count`

These reset at the start of each new run.

`ui/main_window.py` changes

1) In `__init__`

Add these two lines.

Before

Python

Run

```
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_mode = False
self.batch_action = None
self.cancel_requested = False
self.batch_exported_files = []
```

After

Python

Run

```
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.batch_mode = False
self.batch_action = None
self.cancel_requested = False
self.batch_exported_files = []
```

2) In process_selected_item()

Reset the new counters.

Before

Python

Run

```
self.batch_mode = False
self.batch_action = "process"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.cancel_requested = False
self.batch_exported_files = []
```


After

Python

Run

```
self.batch_mode = False
self.batch_action = "process"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.batch_exported_files = []
```

3) In process_checked_items()

Reset the new counters and improve the start log.

Before

Python

Run

```
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_mode = True
self.batch_action = "process"
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Processing {self.batch_total} selected image(s).")
self._start_next_batch_item()
```

After

Python

Run

```
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.batch_mode = True
self.batch_action = "process"
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch process for {self.batch_total} selected .
self._start_next_batch_item()
```

4) In process_all_items()

Reset the new counters.

Before

Python

Run

```
self.batch_mode = True
self.batch_action = "process"
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch processing for {self.batch_total} image(
self._start_next_batch_item()
```

After

Python

Run

```
self.batch_mode = True
self.batch_action = "process"
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch processing for {self.batch_total} image(
self._start_next_batch_item()
```

5) In export_selected_item()

Reset the new counters.

Before

Python

Run

```
self.batch_mode = False
self.batch_action = "export"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self._start_export_item(selected_index)
```

After

Python

Run

```
self.batch_mode = False
self.batch_action = "export"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self._start_export_item(selected_index)
```

6) In export_checked_items()

Reset the new counters and improve the start log.

Before

Python

Run

```
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_mode = True
self.batch_action = "export"
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Exporting {self.batch_total} selected image(s).")
self._start_next_batch_item()
```

After

Python

Run

```
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.batch_mode = True
self.batch_action = "export"
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch export for {self.batch_total} selected i
self._start_next_batch_item()
```

7) In export_all_items()

Reset the new counters.

Before

Python

Run

```
self.batch_queue = list(range(len(self.state.items)))
self.batch_mode = True
self.batch_action = "export"
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch export for {self.batch_total} image(s).")
self._start_next_batch_item()
```

After

Python

Run

```
self.batch_queue = list(range(len(self.state.items)))
self.batch_mode = True
self.batch_action = "export"
self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch export for {self.batch_total} image(s).")
self._start_next_batch_item()
```

8) In _start_next_batch_item()

Improve the end-of-batch logging.

Before

Python

Run

```
def _start_next_batch_item(self):
    if self.cancel_requested:
        self.log("Batch canceled by user.")
        self._finish_processing_session()
        return

    if not self.batch_queue:
        if self.batch_action == "export":
            self.log("Batch export complete.")
        else:
            self.log("Batch processing complete.")
        self._finish_processing_session(final_progress=100)
        return

    next_index = self.batch_queue.pop(0)

    if self.batch_action == "export":
        self._start_export_item(next_index)
    else:
        self._start_preview_processing_item(next_index, force_reproces
```

After

Python

Run

```

def _start_next_batch_item(self):
    if self.cancel_requested:
        self.log("Batch canceled by user.")
        self.log(
            f"Batch summary: completed {self.batch_processed_count}/{s
            f"succeeded {self.batch_success_count}, failed {self.batch
        )
        self._finish_processing_session()
        return

    if not self.batch_queue:
        if self.batch_action == "export":
            self.log("Batch export complete.")
        else:
            self.log("Batch processing complete.")

        self.log(
            f"Batch summary: completed {self.batch_processed_count}/{s
            f"succeeded {self.batch_success_count}, failed {self.batch
        )
        self._finish_processing_session(final_progress=100)
        return

    next_index = self.batch_queue.pop(0)

    if self.batch_action == "export":
        self._start_export_item(next_index)
    else:
        self._start_preview_processing_item(next_index, force_reproces

```

9) In _start_preview_processing_item()

Improve the item-start logging.

Before

Python

Run

```
self.log(f"Preview processing item {item_index + 1}: {item.filename}")
self.log(f"Model: {model_name}")
self.log(f"Input preview path: {preview_input_path}")
self.log(f"Output preview path: {processed_preview_path}")
```

After

Python

Run

```
item_number = self.batch_processed_count + 1 if self.batch_mode else self.batch_processed_count
item_total = self.batch_total if self.batch_mode else 1

self.log(f"[{item_number}/{item_total}] Processing preview: {item.filename}")
self.log(f"Model: {model_name}")
self.log(f"Input preview path: {preview_input_path}")
self.log(f"Output preview path: {processed_preview_path}")
```

10) In _start_full_export_item()

Improve the item-start logging.

Before

Python

Run

```
self.log(f"Starting full-resolution export for: {item.filename}")
self.log(f"Source image: {item.source_path}")
self.log(f"Full-res temp path: {fullres_cache_path}")
self.log(f"Filename suffix: {suffix}")
self.log(f"Apply preview edits: {'yes' if resolved_apply_preview_edits else 'no'}
```

After

Python

Run


```

item_number = self.batch_processed_count + 1 if self.batch_mode el
item_total = self.batch_total if self.batch_mode else 1

self.log(f"[{item_number}/{item_total}] Exporting full-resolution
self.log(f"Source image: {item.source_path}")
self.log(f"Full-res temp path: {fullres_cache_path}")
self.log(f"Filename suffix: {suffix}")
self.log(f"Apply preview edits: {'yes' if resolved_apply_preview_e

```

11) In `_on_preview_process_finished()`

Count successes and add a clearer success log.

Before

Python

Run

```

def _on_preview_process_finished(self, item_index: int, processed_prev
    item = self.state.get_item(item_index)
    if item is None:
        self._finalize_preview_thread()
        self._handle_post_item_completion(error=True)
        return

    processed_preview_path = Path(processed_preview_path_str)
    item.processed_preview_path = processed_preview_path
    item.edited_preview_path = None
    item.status = "processed"

    self.log(f"Preview processing complete: {processed_preview_path}")
    if processed_preview_path.exists():
        self.log(f"Processed preview size: {processed_preview_path.sta

    self._refresh_item_display(item_index)
    self.select_item(item_index)
    self._finalize_preview_thread()
    self._handle_post_item_completion()

```

After

Python

Run

```

def _on_preview_process_finished(self, item_index: int, processed_prev
    item = self.state.get_item(item_index)
    if item is None:
        self._finalize_preview_thread()
        self._handle_post_item_completion(error=True)
        return

    processed_preview_path = Path(processed_preview_path_str)
    item.processed_preview_path = processed_preview_path
    item.edited_preview_path = None
    item.status = "processed"

    self.batch_success_count += 1
    self.log(f"Process success: {item.filename}")
    self.log(f"Preview processing complete: {processed_preview_path}")
    if processed_preview_path.exists():
        self.log(f"Processed preview size: {processed_preview_path.sta

    self._refresh_item_display(item_index)
    self.select_item(item_index)
    self._finalize_preview_thread()
    self._handle_post_item_completion()

```

12) In _on_preview_process_error()

Count failures and add a clearer failure log.

Before

Python

Run

```

def _on_preview_process_error(self, item_index: int, error_message: st
    item = self.state.get_item(item_index)
    if item is not None:
        item.status = "error"
        item.processed_preview_path = None
        item.edited_preview_path = None
        self._refresh_item_display(item_index)

    self.log(f"Preview processing failed for item index {item_index}")
    self.log(f"Error: {error_message}")
    self._finalize_preview_thread()
    self._handle_post_item_completion(error=True)

```

After

Python

Run

```
def _on_preview_process_error(self, item_index: int, error_message: str):
    item = self.state.get_item(item_index)
    if item is not None:
        item.status = "error"
        item.processed_preview_path = None
        item.edited_preview_path = None
        self._refresh_item_display(item_index)
        self.log(f"Process failed: {item.filename}")
    else:
        self.log(f"Preview processing failed for item index {item_index}")

    self.batch_error_count += 1
    self.log(f"Error: {error_message}")
    self._finalize_preview_thread()
    self._handle_post_item_completion(error=True)
```

13) In _on_full_export_finished()

Count successes and add a clearer success log.

Before

Python

Run

```
def _on_full_export_finished(self, item_index: int, _fullres_cache_path: str):
    item = self.state.get_item(item_index)
    if item is not None:
        item.output_path = output_path_str
        item.status = "exported"
        self.log(f"Exported: {output_path_str}")
        self._refresh_item_display(item_index)
        self.select_item(item_index)

    if self.batch_mode:
        self.batch_exported_files.append(output_path_str)

    self._finalize_export_thread()
    self._handle_post_item_completion()
```

After

Python

Run

```
def _on_full_export_finished(self, item_index: int, _fullres_cache_path):
    item = self.state.get_item(item_index)
    if item is not None:
        item.output_path = output_path_str
        item.status = "exported"
        self.batch_success_count += 1
        self.log(f"Export success: {item.filename}")
        self.log(f"Exported: {output_path_str}")
        self._refresh_item_display(item_index)
        self.select_item(item_index)

    if self.batch_mode:
        self.batch_exported_files.append(output_path_str)

    self._finalize_export_thread()
    self._handle_post_item_completion()
```

14) In _on_full_export_error()

Count failures and add a clearer failure log.

Before

Python

Run

```
def _on_full_export_error(self, item_index: int, error_message: str):
    item = self.state.get_item(item_index)
    if item is not None:
        item.status = "error"
        self._refresh_item_display(item_index)

    self.log(f"Full export failed for item index {item_index}")
    self.log(f"Error: {error_message}")
    self._finalize_export_thread()
    self._handle_post_item_completion(error=True)
```

After

Python

Run

```
def _on_full_export_error(self, item_index: int, error_message: str):
    item = self.state.get_item(item_index)
    if item is not None:
        item.status = "error"
        self._refresh_item_display(item_index)
        self.log(f"Export failed: {item.filename}")
    else:
        self.log(f"Full export failed for item index {item_index}")

    self.batch_error_count += 1
    self.log(f"Error: {error_message}")
    self._finalize_export_thread()
    self._handle_post_item_completion(error=True)
```

15) In _finish_processing_session()

Reset the new counters.

Before

Python

Run

```
self.batch_mode = False
self.batch_action = None
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.cancel_requested = False
self.current_processing_index = None
self.batch_exported_files = []

self._set_processing_controls_enabled(True)
```

After

Python

Run

```
self.batch_mode = False
self.batch_action = None
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.current_processing_index = None
self.batch_exported_files = []

self._set_processing_controls_enabled(True)
```

16) In clear_all_items()

Reset the new counters there too.

Before

Python

Run

```
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_mode = False
self.batch_action = None
self.cancel_requested = False
self.batch_exported_files = []
self.visible_indices = []
self._refresh_type_filter_options()
```

After

Python

Run

```
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.batch_mode = False
self.batch_action = None
self.cancel_requested = False
self.batch_exported_files = []
self.visible_indices = []
self._refresh_type_filter_options()
```

What to test

Batch process logging

- select several checked items
- run Process -> Selected
- confirm the log shows:
 - batch start
 - [1/total] , [2/total] , etc.
 - success lines for completed items
 - final batch summary

Batch export logging

- select several checked items
- run Export -> Selected
- confirm the log shows:
 - batch export start
 - [1/total] , [2/total] , etc.
 - export success lines
 - final batch summary

Failure summary

- test with a file/model combination that fails, if you have one
- confirm the log shows:
 - failed filename

- error text
- batch summary with nonzero failed count

Cancel summary

- start a multi-item batch
- click `Cancel`
- confirm:
 - current item finishes
 - batch stops after that
 - summary shows completed / succeeded / failed counts

Single-item runs

- run one `Process`
- run one `Export`
- confirm logging still looks correct and nothing crashes

Next best step after this is model validation: go through the model list one by one and document which are fast, slow, stable, or unreliable on your machine.



i want some way to be able to toggle before and after in file panel. and i noticed that when exporting images that havent been processed, it automatically processes the images. i dont want that, in case using the editor brush without processing was good enough. and when exporting, if the optoin for zip file is selected, it will export a zip with the images AND the selected images not in a zip file. so youll end up with duplicates. so if zip is selected, then only export the zip no duplicates

I am checking the export path before changing it, because the ZIP-only behavior and "export without auto-processing" likely depend on `full_export_worker.py` rather than only `main_window.py`.

Thought for 44s >

I can do the **file-panel before/after toggle** now from the files you already provided.

I cannot safely finish the two export changes from `main_window.py` alone, because those behaviors are almost certainly implemented inside the export worker/export helpers:

- **export without auto-processing**
- **ZIP only, no duplicate loose files**

For those, I need these current files:

- `workers/full_export_worker.py`
- `processing/export_utils.py`

Without those, I would be guessing at the export pipeline.

What I can give you now

Files changed for the file-panel before/after toggle

- `ui/file_panel.py`
- `ui/main_window.py`

This adds a simple toggle in the file panel so you can switch the file panel thumbnails/list icons between:

- **Before** = original thumbnail
- **After** = edited/processed thumbnail when available

Full updated file: `ui/file_panel.py`

Python

Run

```
from PySide6.QtCore import Qt, QSize, Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
    QSizePolicy,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()
    thumbnail_mode_changed = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.current_view_mode = "thumbnail"
        self.current_thumbnail_mode = "after"

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Files")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.include_subfolders_check = QCheckBox("Include subfolders on d
        layout.addWidget(self.include_subfolders_check)

        controls_row = QHBoxLayout()
```


ui/main_window.py changes

1) In `_ ---



start over.. here are the files

full export worker:

```
from pathlib import Path
```

```
from PySide6.QtCore import QObject, Signal, Slot
```

```
from processing.rembg_engine import RembgEngine
from processing.export_utils import build_output_path,
export_processed_image
from processing.mask_ops import
apply_preview_edit_delta_to_fullres
```

```
class FullExportWorker(QObject):
```

```
    finished = Signal(int, str, str)
```

```
    error = Signal(int, str)
```

```
    status = Signal(str)
```

```
    def __init__(
```

```
        self,
```

```
        item_index: int,
```

```
        source_path: Path,
```

```
        processed_cache_path: Path,
```

```
        output_dir: Path,
```

```
        output_format: str,
```

```
        suffix: str,
```

```
        model_name: str,
```

```
        background_mode: str,
```

```
        background_color_name: str,
```

```
        base_preview_mask_path: Path | None = None,
        edited_preview_mask_path: Path | None = None,
        apply_preview_edits: bool = False,
    ):
        super().__init__()

        self.item_index = item_index
        self.source_path = Path(source_path)
        self.processed_cache_path =
Path(processed_cache_path)
        self.output_dir = Path(output_dir)

        self.output_format = output_format
        self.suffix = suffix
        self.model_name = model_name

        self.background_mode = background_mode
        self.background_color_name =
background_color_name

        self.base_preview_mask_path =
Path(base_preview_mask_path) if
base_preview_mask_path else None
        self.edited_preview_mask_path =
Path(edited_preview_mask_path) if
edited_preview_mask_path else None
        self.apply_preview_edits = apply_preview_edits

    @Slot()
    def run(self):
        try:
            self.status.emit(f"Full export worker started for
item index {self.item_index}")
            self.status.emit(f"Generating full-resolution
cutout from: {self.source_path}")
            self.status.emit(f"Model: {self.model_name}")
            self.status.emit(f"Output format:
{self.output_format}")
            self.status.emit(f"Background mode:
{self.background_mode}")
```

```
        self.status.emit(f"Background color:
{self.background_color_name}")
        self.status.emit(f"Full-res cache path:
{self.processed_cache_path}")

        engine = RembgEngine()

        ok, error_message =
engine.remove_background_fullres(
    input_path=self.source_path,
    output_path=self.processed_cache_path,
    model_name=self.model_name,
)

        if not ok:
            self.error.emit(
                self.item_index,
                error_message or "Full-resolution
background removal failed."
            )
            return

        if not self.processed_cache_path.exists():
            self.error.emit(
                self.item_index,
                "Full-resolution processed image was not
created."
            )
            return

        self.status.emit(f"Full-res processed image
created: {self.processed_cache_path}")

        if (
            self.apply_preview_edits
            and self.base_preview_mask_path is not None
            and self.edited_preview_mask_path is not
None
        ):
            self.status.emit(
```

```
        f"Applying preview edit delta to full-res
export: {self.edited_preview_mask_path}"
    )

    ok, error_message =
    apply_preview_edit_delta_to_fullres(

    fullres_image_path=self.processed_cache_path,

    base_preview_mask_image_path=self.base_preview_m
ask_path,

    edited_preview_mask_image_path=self.edited_preview
_mask_path,
    )

    if not ok:
        self.error.emit(
            self.item_index,
            error_message or "Applying preview edits
to full-resolution export failed."
        )
        return

    self.status.emit("Preview edit delta applied to
full-resolution export.")

    output_path = build_output_path(
        source_path=self.source_path,
        output_dir=self.output_dir,
        suffix=self.suffix,
        output_format=self.output_format,
    )

    self.status.emit(f"Preparing final export:
{output_path}")

    ok, error_message = export_processed_image(

    processed_image_path=self.processed_cache_path,
```

```
        output_path=output_path,
        output_format=self.output_format,
        background_mode=self.background_mode,

background_color_name=self.background_color_name,
    )

    if not ok:
        self.error.emit(
            self.item_index,
            error_message or "Final export failed."
        )
        return

    if not Path(output_path).exists():
        self.error.emit(
            self.item_index,
            "Export finished but output file was not
created."
        )
        return

    self.status.emit(f"Full export complete:
{output_path}")

    self.finished.emit(
        self.item_index,
        str(self.processed_cache_path),
        str(output_path),
    )

except Exception as e:
    self.error.emit(self.item_index, str(e))
```

export utils:

```
from pathlib import Path
from PIL import Image
import zipfile
```

```
FORMAT_MAP = {  
    "PNG": ".png",  
    "JPG": ".jpg",  
    "WEBP": ".webp",  
}
```

```
BACKGROUND_COLOR_MAP = {  
    "White": (255, 255, 255),  
    "Black": (0, 0, 0),  
}
```

```
def sanitize_output_stem(stem: str) -> str:  
    bad_chars = '<>:"\\|?*'  
    cleaned = "".join("_" if ch in bad_chars else ch for ch  
in stem)  
    return cleaned.strip().rstrip(".")
```

```
def build_output_path(  
    source_path: Path,  
    output_dir: Path,  
    suffix: str,  
    output_format: str,  
) -> Path:  
    extension =  
FORMAT_MAP.get(output_format.upper(), ".png")  
    safe_stem =  
sanitize_output_stem(source_path.stem)  
    base_name = f"{safe_stem}{suffix}"  
  
    candidate = output_dir / f"{base_name}{extension}"  
    counter = 1  
  
    while candidate.exists():  
        candidate = output_dir / f"{base_name}_{counter}"  
{extension}
```

```
        counter += 1

    return candidate

def export_processed_image(
    processed_image_path: Path,
    output_path: Path,
    output_format: str,
    background_mode: str,
    background_color_name: str,
) -> tuple[bool, str | None]:
    try:
        processed_image_path =
        Path(processed_image_path)
        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True,
        exist_ok=True)

        with Image.open(processed_image_path) as img:
            img = img.convert("RGBA")

            fmt = output_format.upper()

            if fmt == "PNG":
                if background_mode == "Transparent":
                    img.save(output_path, format="PNG")
                else:
                    bg_color =
                    BACKGROUND_COLOR_MAP.get(background_color_
                    name, (255, 255, 255))
                    background = Image.new("RGBA", img.size,
                    bg_color + (255,))
                    composited =
                    Image.alpha_composite(background,
                    img).convert("RGB")
                    composited.save(output_path,
                    format="PNG")

            elif fmt == "JPG":
```



```
        bg_color =
BACKGROUND_COLOR_MAP.get(background_color_
name, (255, 255, 255))
        background = Image.new("RGBA", img.size,
bg_color + (255,))
        composited =
Image.alpha_composite(background,
img).convert("RGB")
        composited.save(output_path,
format="JPEG", quality=95)

    elif fmt == "WEBP":
        if background_mode == "Transparent":
            img.save(output_path, format="WEBP",
quality=95)
        else:
            bg_color =
BACKGROUND_COLOR_MAP.get(background_color_
name, (255, 255, 255))
            background = Image.new("RGBA", img.size,
bg_color + (255,))
            composited =
Image.alpha_composite(background,
img).convert("RGB")
            composited.save(output_path,
format="WEBP", quality=95)

    else:
        return False, f"Unsupported output format:
{output_format}"

    if not output_path.exists():
        return False, f"Output file was not created:
{output_path}"

    return True, None

except Exception as e:
    return False, str(e)
```

```
def create_batch_zip(
    output_paths: list[str],
    output_dir: Path,
) -> tuple[bool, str | None, str | None]:
    try:
        if not output_paths:
            return False, None, "No files available for ZIP
export."

        output_dir = Path(output_dir)
        output_dir.mkdir(parents=True, exist_ok=True)

        zip_path = output_dir / "batch_export.zip"
        counter = 1

        while zip_path.exists():
            zip_path = output_dir /
f"batch_export_{counter}.zip"
            counter += 1

        with zipfile.ZipFile(zip_path, "w",
compression=zipfile.ZIP_DEFLATED) as zipf:
            for file_path in output_paths:
                p = Path(file_path)
                if p.exists() and p.is_file():
                    zipf.write(p, arcname=p.name)

        if not zip_path.exists():
            return False, None, "ZIP file was not created."

        return True, str(zip_path), None

    except Exception as e:
        return False, None, str(e)
```

I have the export files now. The safe fix is to change the export worker so it can either:
use full background removal, or skip removal and export only the brush-edit delta

when the image was never processed.

Thought for 1m 16s >

Changed files:

- `ui/file_panel.py`
- `workers/full_export_worker.py`
- `processing/export_utils.py`

`main_window.py` changes are below as **before/after**.

What these changes do

1) File panel gets a Before / After toggle

This changes the thumbnails/icons shown in the file panel:

- **Before** = original thumbnail
- **After** = edited/processed thumbnail when available

2) Export no longer forces AI processing when pre-process brush edits already exist

If the image was **edited before processing**, export will now:

- use the original full-resolution image
- apply the brush edit delta
- **skip AI background removal**

That prevents the app from auto-processing in cases where your manual brush work was enough.

3) ZIP export no longer leaves duplicate loose files behind

If batch ZIP export is enabled and the ZIP is created successfully:

- the individual exported image files are deleted
- only the ZIP remains in the output folder

Full updated file: `ui/file_panel.py`

Python

Run

```
from PySide6.QtCore import Qt, QSize, Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
    QSizePolicy,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()
    thumbnail_mode_changed = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.current_view_mode = "thumbnail"
        self.current_thumbnail_mode = "after"

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Files")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.include_subfolders_check = QCheckBox("Include subfolders on d
        layout.addWidget(self.include_subfolders_check)

        controls_row = QHBoxLayout()
```


Full updated file: `workers/full_export_worker.py`

Python

Run

```
from pathlib import Path

from PySide6.QtCore import QObject, Signal, Slot

from processing.rembg_engine import RembgEngine
from processing.export_utils import (
    build_output_path,
    export_processed_image,
    prepare_base_export_image,
)
from processing.mask_ops import apply_preview_edit_delta_to_fullres


class FullExportWorker(QObject):
    finished = Signal(int, str, str)
    error = Signal(int, str)
    status = Signal(str)

    def __init__(
        self,
        item_index: int,
        source_path: Path,
        processed_cache_path: Path,
        output_dir: Path,
        output_format: str,
        suffix: str,
        model_name: str,
        background_mode: str,
        background_color_name: str,
        base_preview_mask_path: Path | None = None,
        edited_preview_mask_path: Path | None = None,
        apply_preview_edits: bool = False,
        skip_background_removal: bool = False,
    ):
        super().__init__()

        self.item_index = item_index
        self.source_path = Path(source_path)
        self.processed_cache_path = Path(processed_cache_path)
        self.output_dir = Path(output_dir)

        self.output_format = output_format
        self.suffix = suffix
        self.model_name = model_name

        self.background_mode = background_mode
        self.background_color_name = background_color_name
```


Full updated file: `processing/export_utils.py`

Python

Run

```
from pathlib import Path
from PIL import Image
import zipfile

FORMAT_MAP = {
    "PNG": ".png",
    "JPG": ".jpg",
    "WEBP": ".webp",
}

BACKGROUND_COLOR_MAP = {
    "White": (255, 255, 255),
    "Black": (0, 0, 0),
}

def sanitize_output_stem(stem: str) -> str:
    bad_chars = '<>:"/\|?*'
    cleaned = "".join("_" if ch in bad_chars else ch for ch in stem)
    return cleaned.strip().rstrip(".")

def build_output_path(
    source_path: Path,
    output_dir: Path,
    suffix: str,
    output_format: str,
) -> Path:
    extension = FORMAT_MAP.get(output_format.upper(), ".png")
    safe_stem = sanitize_output_stem(source_path.stem)
    base_name = f"{safe_stem}{suffix}"

    candidate = output_dir / f"{base_name}{extension}"
    counter = 1

    while candidate.exists():
        candidate = output_dir / f"{base_name}_{counter}{extension}"
        counter += 1

    return candidate

def prepare_base_export_image(
    source_path: Path,
    destination_path: Path,
) -> tuple[bool, str | None]:
```


ui/main_window.py changes

1) In _connect_signals()

Before

Python

Run

```
def _connect_signals(self):
    self.file_panel.item_selected.connect(self.select_item_from_panel)
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
    self.file_panel.status_filter_changed.connect(self.rebuild_file_pa
    self.file_panel.type_filter_changed.connect(self.rebuild_file_pane
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)
```

After

Python

Run

```
def _connect_signals(self):
    self.file_panel.item_selected.connect(self.select_item_from_panel)
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
    self.file_panel.status_filter_changed.connect(self.rebuild_file_pa
    self.file_panel.type_filter_changed.connect(self.rebuild_file_pane
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)
    self.file_panel.thumbnail_mode_changed.connect(self.rebuild_file_p
```

2) In rebuild_file_panel()

Find this line:

Before

Python

Run

```
thumb_path = item.edited_preview_path or item.processed_previe
```

Replace it with this:

After

Python

Run

```
if getattr(self.file_panel, "current_thumbnail_mode", "after")
    thumb_path = item.thumb_path
else:
    thumb_path = item.edited_preview_path or item.processed_pr
```

3) In `_start_full_export_item()`

You need to add a new flag named `skip_background_removal`.

Find this block:

Before

Python

Run

```
base_preview_mask_path = None
edited_preview_mask_path = None
resolved_apply_preview_edits = False

if self._has_processed_preview(item) and self._has_edited_preview(
    base_preview_mask_path = item.processed_preview_path
    edited_preview_mask_path = item.edited_preview_path
    resolved_apply_preview_edits = True
else:
    preprocess_edit_preview = item.extra.get("preprocess_edit_preview")
    preprocess_base_preview = item.extra.get("preprocess_edit_base_preview")

    if preprocess_edit_preview and preprocess_base_preview:
        preprocess_edit_preview_path = Path(preprocess_edit_preview)
        preprocess_base_preview_path = Path(preprocess_base_preview)

        if preprocess_edit_preview_path.exists() and preprocess_base_preview_path.exists():
            base_preview_mask_path = preprocess_base_preview_path
            edited_preview_mask_path = preprocess_edit_preview_path
            resolved_apply_preview_edits = True
```

Replace it with this:

After

Python

Run

```
base_preview_mask_path = None
edited_preview_mask_path = None
resolved_apply_preview_edits = False
skip_background_removal = False

if self._has_processed_preview(item) and self._has_edited_preview(
    base_preview_mask_path = item.processed_preview_path
    edited_preview_mask_path = item.edited_preview_path
    resolved_apply_preview_edits = True
else:
    preprocess_edit_preview = item.extra.get("preprocess_edit_preview")
    preprocess_base_preview = item.extra.get("preprocess_edit_base")

    if preprocess_edit_preview and preprocess_base_preview:
        preprocess_edit_preview_path = Path(preprocess_edit_preview)
        preprocess_base_preview_path = Path(preprocess_base_preview)

        if preprocess_edit_preview_path.exists() and preprocess_base_preview_path.exists():
            base_preview_mask_path = preprocess_base_preview_path
            edited_preview_mask_path = preprocess_edit_preview_path
            resolved_apply_preview_edits = True
            skip_background_removal = True
```

Now find this log line block:

Before

Python

Run

```
self.log(f"Starting full-resolution export for: {item.filename}")
self.log(f"Source image: {item.source_path}")
self.log(f"Full-res temp path: {fullres_cache_path}")
self.log(f"Filename suffix: {suffix}")
self.log(f"Apply preview edits: {'yes' if resolved_apply_preview_edits else 'no'}
```

Replace it with this:

After

Python

Run


```
self.log(f"Starting full-resolution export for: {item.filename}")
self.log(f"Source image: {item.source_path}")
self.log(f"Full-res temp path: {fullres_cache_path}")
self.log(f"Filename suffix: {suffix}")
self.log(f"Apply preview edits: {'yes' if resolved_apply_preview_e
self.log(f"Skip AI background removal: {'yes' if skip_background_r
```

Now find the `FullExportWorker(...)` call and add the new argument.

Before

Python

Run

```
self.export_worker = FullExportWorker(
    item_index=item_index,
    source_path=item.source_path,
    processed_cache_path=fullres_cache_path,
    output_dir=output_dir,
    output_format=output_format,
    suffix=suffix,
    model_name=model_name,
    background_mode=background_mode,
    background_color_name=background_color,
    base_preview_mask_path=base_preview_mask_path,
    edited_preview_mask_path=edited_preview_mask_path,
    apply_preview_edits=(
        resolved_apply_preview_edits
        and base_preview_mask_path is not None
        and edited_preview_mask_path is not None
    ),
)
```

After

Python

Run

```

self.export_worker = FullExportWorker(
    item_index=item_index,
    source_path=item.source_path,
    processed_cache_path=fullres_cache_path,
    output_dir=output_dir,
    output_format=output_format,
    suffix=suffix,
    model_name=model_name,
    background_mode=background_mode,
    background_color_name=background_color,
    base_preview_mask_path=base_preview_mask_path,
    edited_preview_mask_path=edited_preview_mask_path,
    apply_preview_edits=(
        resolved_apply_preview_edits
        and base_preview_mask_path is not None
        and edited_preview_mask_path is not None
    ),
    skip_background_removal=skip_background_removal,
)

```

4) In `_finish_processing_session()`

This is the ZIP-only fix.

Find this block:

Before

Python

Run

```

def _finish_processing_session(self, final_progress: int | None = None
    if self.batch_action == "export" and self.settings_panel.export_zi
        output_dir = self._get_output_dir()
        if output_dir is not None:
            ok, zip_path, err = create_batch_zip(self.batch_exported_f
            if ok:
                self.log(f"ZIP export created: {zip_path}")
            else:
                self.log(f"ZIP export failed: {err}")

```

Replace it with this:

After

Python

Run

```
def _finish_processing_session(self, final_progress: int | None = None
    if self.batch_action == "export" and self.settings_panel.export_zi
        output_dir = self._get_output_dir()
        if output_dir is not None:
            ok, zip_path, err = create_batch_zip(self.batch_exported_f
            if ok:
                self.log(f"ZIP export created: {zip_path}")

            for exported_file in self.batch_exported_files:
                exported_path = Path(exported_file)
                try:
                    if exported_path.exists() and exported_path.is
                        exported_path.unlink()
                        self.log(f"Removed loose export after ZIP
                except Exception as e:
                    self.log(f"Could not remove loose export {expo
            else:
                self.log(f"ZIP export failed: {err}")
```

What to test

1) File panel Before / After toggle

- load images
- in the file panel, click **Before**
- confirm thumbnails/icons show the original versions
- click **After**
- confirm thumbnails/icons switch back to processed/edited versions when available

2) Pre-process edit export should not auto-process

Use this exact workflow:

- load an image
- do **not** click Process
- use the brush editor on the image
- export it

Confirm:

- export still succeeds
- the log says `Skip AI background removal: yes`
- the final result reflects only the brush edits
- it does **not** run full AI removal first

3) Normal processed export should still use AI removal

Use this workflow:

- load an image
- click Process first
- optionally edit after processing
- export it

Confirm:

- export still succeeds
- the log says `Skip AI background removal: no`
- processed export still behaves the same as before

4) Export with no processing and no edits

- load an image
- do not process it
- do not edit it
- export it

Confirm:

- it still performs AI background removal
- the log should say `Skip AI background removal: no`

5) ZIP export should not leave duplicates

Use a batch export with multiple selected files:

- enable `Export successful results as ZIP`
- export selected files

Confirm:

- a ZIP file is created

- the loose individual exported image files are deleted afterward
- only the ZIP remains in the output folder

6) ZIP failure safety

If you can force or observe a ZIP failure:

- confirm the app logs the ZIP failure
- confirm the loose exported files are still kept in that case

7) Regression check

- batch process still works
- batch export still works
- Cancel still works
- single-item export still works
- file panel selection/checking still works



instead of before/after in file panel, have a button that say "original" that shows the originals when clicked, and when not clicked it shows the working image. and when i would click before/after it would jump to the opposite end of the file list that i was at, but that may no longer be a problem..

when editing an image, undo only took me back a couple steps. i want undo to go all the way back to original image.

the zip export works fine now.

heres the log if it helps

MyBGRemover started.

Drag and drop files or folders into the window.

Process = preview/background removal only.

Export = final full-resolution file output.

Preview processing uses preview-size images for

speed.

Final export uses the original full-resolution source image.

Output folder is preserved between sessions.

ZIP export is available for batch runs.

Loaded: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg

Loaded: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Loaded: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3.jpg

Loaded: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Loaded: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-1.jpg

Starting batch export for 1 selected image(s).

Starting full-resolution export for: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg

Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print

aesthetic, vector illustration for t-shirt,-1.jpg
Full-res temp path: cache/
fullres_exports/44c357d7a773a0cc_fullres.png
Filename suffix: _u2netp_nobg
Apply preview edits: yes
Skip AI background removal: yes
Base preview mask path: cache/
previews/44c357d7a773a0cc_preview.png
Edited preview mask path: cache/
edited_previews/44c357d7a773a0cc_edited.png
Full export worker started for item index 0
Source image: /home/yma/Downloads/Fluffy black cat
eating ramen with chopsticks, minimalist line art style,
thick outlines, limited color palette, flat shading, bold
silhouette, clean graphic composition, retro print
aesthetic, vector illustration for t-shirt,-1.jpg
Model: u2netp
Output format: PNG
Background mode: Transparent
Background color: White
Full-res cache path: cache/
fullres_exports/44c357d7a773a0cc_fullres.png
Skipping AI background removal for export.
Preparing full-resolution base image from original
source.
Full-res working image ready: cache/
fullres_exports/44c357d7a773a0cc_fullres.png
Applying preview edit delta to full-res export: cache/
edited_previews/44c357d7a773a0cc_edited.png
Preview edit delta applied to full-resolution export.
Preparing final export: /home/yma/
mybgremoversamples/Fluffy black cat eating ramen
with chopsticks, minimalist line art style, thick outlines,
limited color palette, flat shading, bold silhouette, clean
graphic composition, retro print aesthetic, vector
illustration for t-shirt,-1_u2netp_nobg.png
Full export complete: /home/yma/mybgremoversamples/
Fluffy black cat eating ramen with chopsticks,
minimalist line art style, thick outlines, limited color
palette, flat shading, bold silhouette, clean graphic

composition, retro print aesthetic, vector illustration for t-shirt,-1_u2netp_nobg.png

Export success: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg

Exported: /home/yma/mybgremoversamples/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1_u2netp_nobg.png

Batch export complete.

Batch summary: completed 1/1, succeeded 1, failed 0.

Starting batch export for 1 selected image(s).

Starting full-resolution export for: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Full-res temp path: cache/
fullres_exports/507ad6270aaa1ee0_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: no

Skip AI background removal: no

Full export worker started for item index 1

Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Model: u2netp

Output format: PNG

Background mode: Transparent

Background color: White
Full-res cache path: cache/
fullres_exports/507ad6270aaa1ee0_fullres.png
Generating full-resolution cutout from: /home/yma/
Downloads/Fluffy black cat eating ramen with
chopsticks, minimalist line art style, thick outlines,
limited color palette, flat shading, bold silhouette, clean
graphic composition, retro print aesthetic, vector
illustration for t-shirt,.jpg
Full-res working image ready: cache/
fullres_exports/507ad6270aaa1ee0_fullres.png
Preparing final export: /home/yma/
mybgremoversamples/Fluffy black cat eating ramen
with chopsticks, minimalist line art style, thick outlines,
limited color palette, flat shading, bold silhouette, clean
graphic composition, retro print aesthetic, vector
illustration for t-shirt,_u2netp_nobg.png
Full export complete: /home/yma/mybgremoversamples/
Fluffy black cat eating ramen with chopsticks,
minimalist line art style, thick outlines, limited color
palette, flat shading, bold silhouette, clean graphic
composition, retro print aesthetic, vector illustration for
t-shirt,_u2netp_nobg.png
Export success: Fluffy black cat eating ramen with
chopsticks, minimalist line art style, thick outlines,
limited color palette, flat shading, bold silhouette, clean
graphic composition, retro print aesthetic, vector
illustration for t-shirt,.jpg
Exported: /home/yma/mybgremoversamples/Fluffy
black cat eating ramen with chopsticks, minimalist line
art style, thick outlines, limited color palette, flat
shading, bold silhouette, clean graphic composition,
retro print aesthetic, vector illustration for t-
shirt,_u2netp_nobg.png
Batch export complete.
Batch summary: completed 1/1, succeeded 1, failed 0.
Starting batch export for 1 selected image(s).
Starting full-resolution export for: Fluffy black cat
eating ramen with chopsticks, minimalist line art style,
thick outlines, limited color palette, flat shading, bold

silhouette, clean graphic composition, retro print
aesthetic, vector illustration for t-shirt,.jpg
Source image: /home/yma/Downloads/Fluffy black cat
eating ramen with chopsticks, minimalist line art style,
thick outlines, limited color palette, flat shading, bold
silhouette, clean graphic composition, retro print
aesthetic, vector illustration for t-shirt,.jpg
Full-res temp path: cache/
fullres_exports/507ad6270aaa1ee0_fullres.png
Filename suffix: _u2netp_nobg
Apply preview edits: no
Skip AI background removal: no
Full export worker started for item index 1
Source image: /home/yma/Downloads/Fluffy black cat
eating ramen with chopsticks, minimalist line art style,
thick outlines, limited color palette, flat shading, bold
silhouette, clean graphic composition, retro print
aesthetic, vector illustration for t-shirt,.jpg
Model: u2netp
Output format: PNG
Background mode: Transparent
Background color: White
Full-res cache path: cache/
fullres_exports/507ad6270aaa1ee0_fullres.png
Generating full-resolution cutout from: /home/yma/
Downloads/Fluffy black cat eating ramen with
chopsticks, minimalist line art style, thick outlines,
limited color palette, flat shading, bold silhouette, clean
graphic composition, retro print aesthetic, vector
illustration for t-shirt,.jpg
Full-res working image ready: cache/
fullres_exports/507ad6270aaa1ee0_fullres.png
Preparing final export: /home/yma/
mybgremoversamples/Fluffy black cat eating ramen
with chopsticks, minimalist line art style, thick outlines,
limited color palette, flat shading, bold silhouette, clean
graphic composition, retro print aesthetic, vector
illustration for t-shirt,_u2netp_nobg_1.png
Full export complete: /home/yma/mybgremoversamples/
Fluffy black cat eating ramen with chopsticks,

minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,_u2netp_nobg_1.png

Export success: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Exported: /home/yma/mybgremoversamples/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,_u2netp_nobg_1.png

Batch export complete.

Batch summary: completed 1/1, succeeded 1, failed 0.
Starting batch export for 1 selected image(s).

Starting full-resolution export for: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Full-res temp path: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: no

Skip AI background removal: no

Full export worker started for item index 3

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Model: u2netp

Output format: PNG

Background mode: Transparent

Background color: White

Full-res cache path: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Generating full-resolution cutout from: /home/yma/

Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Full-res working image ready: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Preparing final export: /home/yma/

mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

Full export complete: /home/yma/mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

Export success: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Exported: /home/yma/mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

Batch export complete.

Batch summary: completed 1/1, succeeded 1, failed 0.

Starting batch export for 1 selected image(s).

Starting full-resolution export for: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Full-res temp path: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: yes

Skip AI background removal: yes

Base preview mask path: cache/

previews/4130268cb2b398cb_preview.png

Edited preview mask path: cache/

edited_previews/4130268cb2b398cb_edited.png

Full export worker started for item index 3

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Model: u2netp

Output format: PNG

Background mode: Transparent

Background color: White

Full-res cache path: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Skipping AI background removal for export.

Preparing full-resolution base image from original source.

Full-res working image ready: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Applying preview edit delta to full-res export: cache/

edited_previews/4130268cb2b398cb_edited.png

Preview edit delta applied to full-resolution export.

Preparing final export: /home/yma/

mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

Full export complete: /home/yma/mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

Export success: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Exported: /home/yma/mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

Batch export complete.

Batch summary: completed 1/1, succeeded 1, failed 0.

Starting batch export for 5 image(s).

Starting full-resolution export for: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg

Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg

Full-res temp path: cache/fullres_exports/44c357d7a773a0cc_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: yes

Skip AI background removal: yes
Base preview mask path: cache/
previews/44c357d7a773a0cc_preview.png
Edited preview mask path: cache/
edited_previews/44c357d7a773a0cc_edited.png
Full export worker started for item index 0
Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg
Model: u2netp
Output format: PNG
Background mode: Transparent
Background color: White
Full-res cache path: cache/
fullres_exports/44c357d7a773a0cc_fullres.png
Skipping AI background removal for export.
Preparing full-resolution base image from original source.
Full-res working image ready: cache/
fullres_exports/44c357d7a773a0cc_fullres.png
Applying preview edit delta to full-res export: cache/
edited_previews/44c357d7a773a0cc_edited.png
Preview edit delta applied to full-resolution export.
Preparing final export: /home/yma/
mybgremoversamples/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1_u2netp_nobg.png
Full export complete: /home/yma/mybgremoversamples/
Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1_u2netp_nobg.png
Export success: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean

graphic composition, retro print aesthetic, vector illustration for t-shirt,-1.jpg

Exported: /home/yma/mybgremoversamples/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,-1_u2netp_nobg.png

Starting full-resolution export for: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Full-res temp path: cache/

fullres_exports/507ad6270aaa1ee0_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: no

Skip AI background removal: no

Full export worker started for item index 1

Source image: /home/yma/Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-shirt,.jpg

Model: u2netp

Output format: PNG

Background mode: Transparent

Background color: White

Full-res cache path: cache/

fullres_exports/507ad6270aaa1ee0_fullres.png

Generating full-resolution cutout from: /home/yma/

Downloads/Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector

illustration for t-shirt,.jpg

Full-res working image ready: cache/

fullres_exports/507ad6270aaa1ee0_fullres.png

Preparing final export: /home/yma/

mybgremoversamples/Fluffy black cat eating ramen

with chopsticks, minimalist line art style, thick outlines,

limited color palette, flat shading, bold silhouette, clean

graphic composition, retro print aesthetic, vector

illustration for t-shirt,_u2netp_nobg.png

Full export complete: /home/yma/mybgremoversamples/

Fluffy black cat eating ramen with chopsticks,

minimalist line art style, thick outlines, limited color

palette, flat shading, bold silhouette, clean graphic

composition, retro print aesthetic, vector illustration for

t-shirt,_u2netp_nobg.png

Export success: Fluffy black cat eating ramen with

chopsticks, minimalist line art style, thick outlines,

limited color palette, flat shading, bold silhouette, clean

graphic composition, retro print aesthetic, vector

illustration for t-shirt,.jpg

Exported: /home/yma/mybgremoversamples/Fluffy

black cat eating ramen with chopsticks, minimalist line

art style, thick outlines, limited color palette, flat

shading, bold silhouette, clean graphic composition,

retro print aesthetic, vector illustration for t-

shirt,_u2netp_nobg.png

Starting full-resolution export for: Vintage botanical

linocut illustration of fully bloomed roses with layered

spiral petals and elegant curved stems with thorns and

leaves. Hand-carved linoleum relief print style with bold

organic shapes, visible carving lines,-3.jpg

Source image: /home/yma/Downloads/Vintage botanical

linocut illustration of fully bloomed roses with layered

spiral petals and elegant curved stems with thorns and

leaves. Hand-carved linoleum relief print style with bold

organic shapes, visible carving lines,-3.jpg

Full-res temp path: cache/

fullres_exports/9cdddd214b93883b_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: no

Skip AI background removal: no

Full export worker started for item index 2

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3.jpg

Model: u2netp

Output format: PNG

Background mode: Transparent

Background color: White

Full-res cache path: cache/

fullres_exports/9cdddd214b93883b_fullres.png

Generating full-resolution cutout from: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3.jpg

Full-res working image ready: cache/

fullres_exports/9cdddd214b93883b_fullres.png

Preparing final export: /home/yma/

mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving

lines,-3_u2netp_nobg.png

Full export complete: /home/yma/mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3_u2netp_nobg.png

Export success: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3.jpg

Exported: /home/yma/mybgremoversamples/Vintage

botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3_u2netp_nobg.png

Starting full-resolution export for: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Full-res temp path: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: yes

Skip AI background removal: yes

Base preview mask path: cache/

previews/4130268cb2b398cb_preview.png

Edited preview mask path: cache/

edited_previews/4130268cb2b398cb_edited.png

Full export worker started for item index 3

Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2.jpg

Model: u2netp

Output format: PNG

Background mode: Transparent

Background color: White

Full-res cache path: cache/

fullres_exports/4130268cb2b398cb_fullres.png

Skipping AI background removal for export.

Preparing full-resolution base image from original source.

Full-res working image ready: cache/

fullres_exports/4130268cb2b398cb_fullres.png
Applying preview edit delta to full-res export: cache/
edited_previews/4130268cb2b398cb_edited.png
Preview edit delta applied to full-resolution export.
Preparing final export: /home/yma/
mybgremoversamples/Vintage botanical linocut
illustration of fully bloomed roses with layered spiral
petals and elegant curved stems with thorns and leaves.
Hand-carved linoleum relief print style with bold
organic shapes, visible carving
lines,-2_u2netp_nobg.png
Full export complete: /home/yma/mybgremoversamples/
Vintage botanical linocut illustration of fully bloomed
roses with layered spiral petals and elegant curved
stems with thorns and leaves. Hand-carved linoleum
relief print style with bold organic shapes, visible
carving lines,-2_u2netp_nobg.png
Export success: Vintage botanical linocut illustration of
fully bloomed roses with layered spiral petals and
elegant curved stems with thorns and leaves. Hand-
carved linoleum relief print style with bold organic
shapes, visible carving lines,-2.jpg
Exported: /home/yma/mybgremoversamples/Vintage
botanical linocut illustration of fully bloomed roses with
layered spiral petals and elegant curved stems with
thorns and leaves. Hand-carved linoleum relief print
style with bold organic shapes, visible carving
lines,-2_u2netp_nobg.png
Starting full-resolution export for: Vintage botanical
linocut illustration of fully bloomed roses with layered
spiral petals and elegant curved stems with thorns and
leaves. Hand-carved linoleum relief print style with bold
organic shapes, visible carving lines,-1.jpg
Source image: /home/yma/Downloads/Vintage botanical
linocut illustration of fully bloomed roses with layered
spiral petals and elegant curved stems with thorns and
leaves. Hand-carved linoleum relief print style with bold
organic shapes, visible carving lines,-1.jpg
Full-res temp path: cache/
fullres_exports/79f6ae6f1f455a60_fullres.png

Filename suffix: _u2netp_nobg
Apply preview edits: no
Skip AI background removal: no
Full export worker started for item index 4
Source image: /home/yma/Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-1.jpg
Model: u2netp
Output format: PNG
Background mode: Transparent
Background color: White
Full-res cache path: cache/
fullres_exports/79f6ae6f1f455a60_fullres.png
Generating full-resolution cutout from: /home/yma/
Downloads/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-1.jpg
Full-res working image ready: cache/
fullres_exports/79f6ae6f1f455a60_fullres.png
Preparing final export: /home/yma/
mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-1_u2netp_nobg.png
Full export complete: /home/yma/mybgremoversamples/
Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-1_u2netp_nobg.png
Export success: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic

shapes, visible carving lines,-1.jpg

Exported: /home/yma/mybgremoversamples/Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-1_u2netp_nobg.png

Batch export complete.

Batch summary: completed 5/5, succeeded 5, failed 0.

ZIP export created: /home/yma/mybgremoversamples/batch_export.zip

Removed loose export after ZIP creation: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-

shirt,-1_u2netp_nobg.png

Removed loose export after ZIP creation: Fluffy black cat eating ramen with chopsticks, minimalist line art style, thick outlines, limited color palette, flat shading, bold silhouette, clean graphic composition, retro print aesthetic, vector illustration for t-

shirt,_u2netp_nobg.png

Removed loose export after ZIP creation: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-3_u2netp_nobg.png

lines,-3_u2netp_nobg.png

Removed loose export after ZIP creation: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving lines,-2_u2netp_nobg.png

lines,-2_u2netp_nobg.png

Removed loose export after ZIP creation: Vintage botanical linocut illustration of fully bloomed roses with layered spiral petals and elegant curved stems with thorns and leaves. Hand-carved linoleum relief print style with bold organic shapes, visible carving

lines,-1_u2netp_nobg.png

I can change the file-panel toggle now. The undo-to-original issue is in `ui/preview_canvas.py`, so I need that file to fix it correctly rather than guessing and breaking the edit stack.

Thought for 6s

Changed now:

- `ui/file_panel.py`
- `ui/main_window.py`

Still needed to finish the undo change:

- `ui/preview_canvas.py`

What this step does

- replaces the file-panel Before/After toggle with a single **Original** button
- when **Original** is checked, file panel shows original thumbnails
- when **Original** is not checked, file panel shows the working image thumbnails
- rebuild now preserves your current scroll position better, which should prevent the jump-to-opposite-end behavior

Full updated file: `ui/file_panel.py`

Python

Run

```
from PySide6.QtCore import Qt, QSize, Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
    QSizePolicy,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()
    thumbnail_mode_changed = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.current_view_mode = "thumbnail"
        self.current_thumbnail_mode = "working"

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Files")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.include_subfolders_check = QCheckBox("Include subfolders on d")
        layout.addWidget(self.include_subfolders_check)

        controls_row = QHBoxLayout()
```


ui/main_window.py **changes**

1) In rebuild_file_panel()

Replace this:

Python

Run

```
def rebuild_file_panel(self, *_args, reload_preview: bool = True):
    selected_index = self.state.selected_index
    checked_indices = set(self.file_panel.get_checked_user_indices())
    status_filter = self.file_panel.status_filter_combo.currentText()
    type_filter = self.file_panel.type_filter_combo.currentText()

    self.file_panel.file_view.blockSignals(True)
    self.file_panel.clear_items()
    self.visible_indices = []

    for item_index in self._sorted_indices():
        item = self.state.items[item_index]
        if not self._item_matches_filter(item, status_filter, type_fil
            continue

        if getattr(self.file_panel, "current_thumbnail_mode", "after")
            thumb_path = item.thumb_path
        else:
            thumb_path = item.edited_preview_path or item.processed_pr

        self.file_panel.add_item(
            self._build_panel_text(item),
            str(thumb_path) if thumb_path else None,
            item_index,
            checked=item_index in checked_indices,
        )
        self.visible_indices.append(item_index)

    selected_to_show = None

    if selected_index is not None and selected_index in self.visible_i
        visible_row = self.visible_indices.index(selected_index)
        self.file_panel.set_current_visible_row(visible_row)
        selected_to_show = selected_index
    elif self.visible_indices:
        selected_to_show = self.visible_indices[0]
        self.state.set_selected_index(selected_to_show)
        self.file_panel.set_current_visible_row(0)
    else:
        self.state.set_selected_index(None)
        self.preview_canvas.clear_preview()

    self.file_panel.file_view.blockSignals(False)

    if hasattr(self.file_panel, "_update_counts"):
        self.file_panel._update_counts()

    if selected_to_show is not None and reload_preview:
```


With this:

Python

Run

```
def rebuild_file_panel(self, *_args, reload_preview: bool = True):
    selected_index = self.state.selected_index
    checked_indices = set(self.file_panel.get_checked_user_indices())
    status_filter = self.file_panel.status_filter_combo.currentText()
    type_filter = self.file_panel.type_filter_combo.currentText()

    scroll_bar = self.file_panel.file_view.verticalScrollBar()
    previous_scroll_value = scroll_bar.value() if scroll_bar is not None

    self.file_panel.file_view.blockSignals(True)
    self.file_panel.clear_items()
    self.visible_indices = []

    for item_index in self._sorted_indices():
        item = self.state.items[item_index]
        if not self._item_matches_filter(item, status_filter, type_filter):
            continue

        if getattr(self.file_panel, "current_thumbnail_mode", "working"):
            thumb_path = item.thumb_path
        else:
            thumb_path = item.edited_preview_path or item.processed_preview_path

        self.file_panel.add_item(
            self._build_panel_text(item),
            str(thumb_path) if thumb_path else None,
            item_index,
            checked=item_index in checked_indices,
        )
        self.visible_indices.append(item_index)

    selected_to_show = None

    if selected_index is not None and selected_index in self.visible_indices:
        visible_row = self.visible_indices.index(selected_index)
        self.file_panel.set_current_visible_row(visible_row)
        selected_to_show = selected_index
    elif self.visible_indices:
        selected_to_show = self.visible_indices[0]
        self.state.set_selected_index(selected_to_show)
        self.file_panel.set_current_visible_row(0)
    else:
        self.state.set_selected_index(None)
        self.preview_canvas.clear_preview()

    self.file_panel.file_view.blockSignals(False)

    if scroll_bar is not None:
```

What to test for this step

Original button

- load several images
- in the file panel, click **Original**
- confirm file panel thumbnails switch to the originals
- click it again to turn it off
- confirm file panel thumbnails switch back to the working image versions

Scroll stability

- scroll to the middle or bottom of the file panel
- click **Original** on and off
- confirm the list stays near the same place instead of jumping to the opposite end

What I need next to fix undo properly

Send the current full contents of:

Python

Run

```
ui/preview_canvas.py
```

The undo issue is almost certainly there.

Based on your description, the likely problem is one of these:

- the original state is not being pushed as the first undo state
- the undo stack is capped too aggressively
- only some strokes are being committed to the stack

- the stack is being reset during certain updates

Once you send `ui/preview_canvas.py` , I will give you:

- the exact changed file
- the full updated file
- only the test checklist, since you do not want apply/run directions



```
preview canvas:
```

```
from io import BytesIO
from pathlib import Path
```

```
import numpy as np
from PIL import Image
```

```
from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)
```

```
class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner
```

```
def mousePressEvent(self, event):
    self.owner.handle_mouse_press(event)
```

```
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)

        self._current_pixmap = None
        self._base_fit_scale = 1.0
        self._zoom_factor = 1.0

        self._before_image_pil = None
        self._editable_after_image_pil = None
        self._editable_rgba = None
        self._restore_source_rgba = None

        self._interaction_mode = "preview"
        self._current_mode = "after"

        self._tool_name = "Erase"
        self._apply_mode = "Brush"

        self._brush_size = 40
        self._tolerance = 35

        self._magic_mode = "Connected Region"
```

```
self._mouse_down = False
self._last_image_point = None
self._drag_edit_active = False

self._undo_stack = []
self._redo_stack = []

self._editable_save_path = None
self._title_text = "Preview"

layout = QVBoxLayout(self)

self.title_label = QLabel("Preview")

toolbar = QHBoxLayout()

self.preview_mode_btn = QPushButton("Preview
Mode ✓")
self.edit_mode_btn = QPushButton("Edit Mode")

self.before_btn = QPushButton("Before")
self.after_btn = QPushButton("After ✓")

self.undo_btn = QPushButton("Undo")
self.redo_btn = QPushButton("Redo")

self.zoom_in_btn = QPushButton("Zoom +")
self.zoom_out_btn = QPushButton("Zoom -")

toolbar.addWidget(self.preview_mode_btn)
toolbar.addWidget(self.edit_mode_btn)
toolbar.addWidget(self.before_btn)
toolbar.addWidget(self.after_btn)
toolbar.addWidget(self.undo_btn)
toolbar.addWidget(self.redo_btn)
toolbar.addStretch()
toolbar.addWidget(self.zoom_in_btn)
toolbar.addWidget(self.zoom_out_btn)

self.image_label = _CanvasLabel(self)
```

```
self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
self.image_label.setSizePolicy(QSizePolicy.Policy.Fixed, QSizePolicy.Policy.Fixed)
```

```
    self.image_label.setStyleSheet("background: transparent;")
```

```
    self.image_label.setMouseTracking(True)
```

```
    self.image_label.hide()
```

```
self.image_container = QWidget()
```

```
self.image_container.setStyleSheet("background: transparent;")
```

```
self.image_container_layout = QVBoxLayout(self.image_container)
```

```
self.image_container_layout.setContentsMargins(0, 0, 0, 0)
```

```
self.image_container_layout.addStretch()
```

```
self.image_container_layout.addWidget(
```

```
    self.image_label,
```

```
    0,
```

```
    Qt.AlignmentFlag.AlignCenter,
```

```
)
```

```
self.image_container_layout.addStretch()
```

```
self.scroll_area = QScrollArea()
```

```
self.scroll_area.setWidget(self.image_container)
```

```
self.scroll_area.setWidgetResizable(True)
```

```
self.scroll_area.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```
    self.scroll_area.setStyleSheet("background: transparent; border: 0;")
```

```
self.scroll_area.viewport().setStyleSheet("background: transparent;")
```

```
layout.addWidget(self.title_label)
```

```
        layout.addLayout(toolbar)
        layout.addWidget(self.scroll_area)

    self.preview_mode_btn.clicked.connect(self.set_preview_mode)

    self.edit_mode_btn.clicked.connect(self.set_edit_mode)
    self.before_btn.clicked.connect(self.show_before)
    self.after_btn.clicked.connect(self.show_after)
    self.undo_btn.clicked.connect(self.undo)
    self.redo_btn.clicked.connect(self.redo)
    self.zoom_in_btn.clicked.connect(self.zoom_in)
    self.zoom_out_btn.clicked.connect(self.zoom_out)

    self._update_mode_buttons()

    def resizeEvent(self, event):
        super().resizeEvent(event)
        if self._current_pixmap is not None:
            self._render_current_pixmap()

    def clear_preview(self, reset_title: bool = True):
        self._current_pixmap = None
        self._base_fit_scale = 1.0
        self._zoom_factor = 1.0

        self._before_image_pil = None
        self._editable_after_image_pil = None
        self._editable_rgba = None
        self._restore_source_rgba = None

        self._interaction_mode = "preview"
        self._current_mode = "after"

        self._mouse_down = False
        self._last_image_point = None
        self._drag_edit_active = False

        self._undo_stack = []
```



```
self._redo_stack = []

self._editable_save_path = None

if reset_title:
    self._title_text = "Preview"
    self.title_label.setText("Preview")

self.image_label.clear()
self.image_label.resize(0, 0)
self.image_label.hide()
self._update_mode_buttons()

def set_magic_mode(self, mode: str):
    self._magic_mode = mode

def set_tool_settings(
    self,
    tool_name,
    apply_mode,
    brush_size,
    softness,
    opacity,
    spacing,
    tolerance,
):
    self._tool_name = tool_name
    self._apply_mode = apply_mode
    self._brush_size = brush_size
    self._tolerance = tolerance

def set_preview_images(
    self,
    before_image_path,
    after_image_path=None,
    title_text="Preview",
    default_mode="after",
    editable_save_path=None,
    preferred_interaction_mode="preview",
    preferred_view_mode="after",
```

```
        **_,
    ):
        self.clear_preview(reset_title=False)

        self._title_text = title_text or "Preview"
        self.title_label.setText(self._title_text)

        self._editable_save_path =
        Path(editable_save_path) if editable_save_path else
        None
        if self._editable_save_path is not None:

            self._editable_save_path.parent.mkdir(parents=True,
            exist_ok=True)

            if before_image_path and
            Path(before_image_path).exists():
                with Image.open(before_image_path) as img:
                    self._before_image_pil = img.convert("RGBA")

            loaded_after = None
            if after_image_path and
            Path(after_image_path).exists():
                with Image.open(after_image_path) as img:
                    loaded_after = img.convert("RGBA")

            if loaded_after is not None:
                self._editable_after_image_pil = loaded_after
            elif self._before_image_pil is not None:
                self._editable_after_image_pil =
                self._before_image_pil.copy()
            else:
                self._editable_after_image_pil = None

            if self._editable_after_image_pil is not None:
                self._editable_rgba =
                np.array(self._editable_after_image_pil)
            else:
                self._editable_rgba = None
```

```
        if self._before_image_pil is not None:
            self._restore_source_rgba =
np.array(self._before_image_pil)
            elif self._editable_after_image_pil is not None:
                self._restore_source_rgba =
np.array(self._editable_after_image_pil)
            else:
                self._restore_source_rgba = None

        valid_modes = {"before", "after"}
        chosen_view_mode = preferred_view_mode if
preferred_view_mode in valid_modes else
default_mode

        if chosen_view_mode == "before" and
self._before_image_pil is not None:
            self._current_mode = "before"
        elif self._editable_after_image_pil is not None:
            self._current_mode = "after"
        else:
            self._current_mode = "before"

        if preferred_interaction_mode == "edit" and
self._editable_after_image_pil is not None:
            self._interaction_mode = "edit"
            self._current_mode = "after"
        else:
            self._interaction_mode = "preview"

        self._zoom_factor = 1.0
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def set_preview_mode(self):
        self._interaction_mode = "preview"
        if self._editable_after_image_pil is not None:
            self._current_mode = "after"
        else:
            self._current_mode = "before"
        self._load_current_mode_pixmap()
```

```
        self._update_mode_buttons()

    def set_edit_mode(self):
        if self._editable_after_image_pil is None:
            return
        self._interaction_mode = "edit"
        self._current_mode = "after"
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def show_before(self):
        if self._before_image_pil is None:
            return
        self._current_mode = "before"
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def show_after(self):
        if self._editable_after_image_pil is None:
            return
        self._current_mode = "after"
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def zoom_in(self):
        if self._current_pixmap is None:
            return
        self._zoom_factor = min(self._zoom_factor * 1.25,
8.0)
        self._render_current_pixmap()

    def zoom_out(self):
        if self._current_pixmap is None:
            return
        self._zoom_factor = max(self._zoom_factor / 1.25,
0.1)
        self._render_current_pixmap()

    def _update_mode_buttons(self):
        if self._interaction_mode == "preview":
```

```
        self.preview_mode_btn.setText("Preview Mode ✓")
        self.edit_mode_btn.setText("Edit Mode")
    else:
        self.preview_mode_btn.setText("Preview Mode")
        self.edit_mode_btn.setText("Edit Mode ✓")

    if self._current_mode == "before":
        self.before_btn.setText("Before ✓")
        self.after_btn.setText("After")
    else:
        self.before_btn.setText("Before")
        self.after_btn.setText("After ✓")

    has_edit_target = self._editable_after_image_pil is
not None
    self.edit_mode_btn.setEnabled(has_edit_target)
    self.after_btn.setEnabled(has_edit_target)

    self.undo_btn.setEnabled(bool(self._undo_stack))
    self.redo_btn.setEnabled(bool(self._redo_stack))

def _get_current_image(self):
    if self._current_mode == "after":
        return self._editable_after_image_pil
    return self._before_image_pil

def _load_current_mode_pixmap(self):
    image = self._get_current_image()

    if image is None:
        self.image_label.clear()
        self.image_label.resize(0, 0)
        self.image_label.hide()
        self._current_pixmap = None
        self._update_mode_buttons()
        return

    buffer = BytesIO()
    image.save(buffer, format="PNG")
```

```
        pixmap = QPixmap()
        pixmap.loadFromData(buffer.getvalue())
        self._current_pixmap = pixmap

    def _render_current_pixmap(self):
        if self._current_pixmap is None:
            return

        viewport_size = self.scroll_area.viewport().size()
        if not viewport_size.isValid():
            viewport_size = QSize(400, 400)

        available_width = max(1, viewport_size.width() -
20)
        available_height = max(1, viewport_size.height() -
20)

        pixmap_width = max(1,
self._current_pixmap.width())
        pixmap_height = max(1,
self._current_pixmap.height())

        fit_scale = min(
            available_width / pixmap_width,
            available_height / pixmap_height,
        )

        self._base_fit_scale = min(fit_scale, 1.0)
        final_scale = max(0.01, self._base_fit_scale *
self._zoom_factor)

        scaled_width = max(1, int(pixmap_width *
final_scale))
        scaled_height = max(1, int(pixmap_height *
final_scale))

        scaled_pixmap = self._current_pixmap.scaled(
```

```
        scaled_width,  
        scaled_height,  
        Qt.AspectRatioMode.KeepAspectRatio,  
        Qt.TransformationMode.SmoothTransformation,  
    )  
  
    self.image_label.setPixmap(scaled_pixmap)  
    self.image_label.resize(scaled_pixmap.size())  
    self.image_label.show()  
    self._update_mode_buttons()  
  
def handle_mouse_press(self, event):  
    if event.button() != Qt.MouseButton.LeftButton:  
        return  
  
    if self._interaction_mode != "edit":  
        return  
  
    if self._editable_rgba is None:  
        return  
  
    pos = event.position().toPoint()  
    image_pos = self._widget_pos_to_image_pos(pos)  
  
    if image_pos is None:  
        return  
  
    x, y = image_pos  
  
    self._mouse_down = True  
    self._last_image_point = (x, y)  
    self._drag_edit_active = False  
  
    if self._tool_name in ("Erase", "Restore"):  
  
self._undo_stack.append(self._editable_rgba.copy())  
        self._redo_stack.clear()  
  
        changed = self._apply_brush_tool(x, y)  
        if changed:
```

```
        self._drag_edit_active = True
        self._finish_edit_update()
    else:
        self._undo_stack.pop()
        self._update_mode_buttons()
    return

self._undo_stack.append(self._editable_rgba.copy())
self._redo_stack.clear()

changed = False

if self._tool_name == "Magic Erase":
    changed = self._magic_erase(x, y)
elif self._tool_name == "Background Erase":
    changed = self._background_erase(x, y)

if not changed:
    self._undo_stack.pop()
    self._update_mode_buttons()
    return

self._finish_edit_update()

def handle_mouse_move(self, event):
    if not self._mouse_down:
        return

    if self._interaction_mode != "edit":
        return

    if self._editable_rgba is None:
        return

    if self._tool_name not in ("Erase", "Restore"):
        return

    pos = event.position().toPoint()
    image_pos = self._widget_pos_to_image_pos(pos)
```



```
        if image_pos is None:
            return

        x, y = image_pos
        current_point = (x, y)

        if self._last_image_point is None:
            self._last_image_point = current_point

        changed =
self._apply_brush_stroke(self._last_image_point,
current_point)

        if changed:
            self._drag_edit_active = True
            self._finish_edit_update()

        self._last_image_point = current_point

def handle_mouse_release(self, event):
    self._mouse_down = False
    self._last_image_point = None
    self._drag_edit_active = False

def _widget_pos_to_image_pos(self, pos: QPoint):
    pixmap = self.image_label.pixmap()

    if pixmap is None or self._editable_rgba is None:
        return None

    w = pixmap.width()
    h = pixmap.height()

    if w == 0 or h == 0:
        return None

    x = pos.x()
    y = pos.y()
```

```
img_h, img_w = self._editable_rgba.shape[:2]

img_x = int(x * img_w / w)
img_y = int(y * img_h / h)

if img_x < 0 or img_y < 0 or img_x >= img_w or
img_y >= img_h:
    return None

return img_x, img_y

def _apply_brush_tool(self, x: int, y: int) -> bool:
    if self._tool_name == "Erase":
        return self._brush_erase(x, y)
    if self._tool_name == "Restore":
        return self._brush_restore(x, y)
    return False

def _apply_brush_stroke(self, start_point: tuple[int,
int], end_point: tuple[int, int]) -> bool:
    x1, y1 = start_point
    x2, y2 = end_point

    dx = x2 - x1
    dy = y2 - y1
    steps = max(abs(dx), abs(dy), 1)

    changed = False

    for i in range(steps + 1):
        px = int(round(x1 + (dx * i / steps)))
        py = int(round(y1 + (dy * i / steps)))
        if self._apply_brush_tool(px, py):
            changed = True

    return changed

def _brush_mask(self, center_x: int, center_y: int):
    if self._editable_rgba is None:
        return None
```

```
img_h, img_w = self._editable_rgba.shape[:2]
radius = max(1, int(self._brush_size / 2))

y_grid, x_grid = np.ogrid[:img_h, :img_w]
distance_sq = (x_grid - center_x) ** 2 + (y_grid -
center_y) ** 2
return distance_sq <= radius ** 2

def _brush_erase(self, x: int, y: int) -> bool:
    if self._editable_rgba is None:
        return False

    mask = self._brush_mask(x, y)
    if mask is None:
        return False

    before_alpha = self._editable_rgba[:, :, 3].copy()
    self._editable_rgba[mask, 3] = 0
    return not np.array_equal(before_alpha,
self._editable_rgba[:, :, 3])

def _brush_restore(self, x: int, y: int) -> bool:
    if self._editable_rgba is None or
self._restore_source_rgba is None:
        return False

    mask = self._brush_mask(x, y)
    if mask is None:
        return False

    before_rgba = self._editable_rgba.copy()
    self._editable_rgba[mask] =
self._restore_source_rgba[mask]
    return not np.array_equal(before_rgba,
self._editable_rgba)

def _magic_erase(self, x: int, y: int) -> bool:
    before_alpha = self._editable_rgba[:, :, 3].copy()
```

```
        rgb = self._editable_rgba[:, :, :3].astype(np.int16)
        alpha = self._editable_rgba[:, :,
3].astype(np.float32)

        seed_color = rgb[y, x]

        diff = np.sqrt(np.sum((rgb - seed_color) ** 2,
axis=2))
        mask = diff <= self._tolerance

        if self._magic_mode == "Connected Region":
            height, width = mask.shape
            visited = np.zeros_like(mask, dtype=bool)
            stack = [(x, y)]

            while stack:
                px, py = stack.pop()

                if px < 0 or py < 0 or px >= width or py >=
height:
                    continue

                if visited[py, px]:
                    continue

                visited[py, px] = True

                if not mask[py, px]:
                    continue

                alpha[py, px] = 0

                stack.append((px + 1, py))
                stack.append((px - 1, py))
                stack.append((px, py + 1))
                stack.append((px, py - 1))
            else:
                alpha[mask] = 0

        self._editable_rgba[:, :, 3] = alpha.astype(np.uint8)
```

```
        return not np.array_equal(before_alpha,
self._editable_rgba[:, :, 3])

    def _background_erase(self, x: int, y: int) -> bool:
        before_alpha = self._editable_rgba[:, :, 3].copy()

        rgb = self._editable_rgba[:, :, :3].astype(np.int16)
        alpha = self._editable_rgba[:, :, 3].copy()

        seed_color = rgb[y, x]

        diff = np.sqrt(np.sum((rgb - seed_color) ** 2,
axis=2))
        mask = diff <= self._tolerance

        alpha[mask] = 0

        self._editable_rgba[:, :, 3] = alpha
        return not np.array_equal(before_alpha,
self._editable_rgba[:, :, 3])

    def _sync_pil_from_rgba(self):
        self._editable_after_image_pil =
Image.fromarray(self._editable_rgba, "RGBA")

    def _save_edited_preview(self):
        if self._editable_after_image_pil is None or
self._editable_save_path is None:
            return

        self._editable_after_image_pil.save(self._editable_save
_path, format="PNG")

        self.image_edited.emit(str(self._editable_save_path))

    def _finish_edit_update(self):
        self._sync_pil_from_rgba()
        self._save_edited_preview()
        self._current_mode = "after"
```

```
        self._load_current_mode_pixmap()

    def undo(self):
        if not self._undo_stack or self._editable_rgba is
None:
            return

        self._redo_stack.append(self._editable_rgba.copy())
        self._editable_rgba = self._undo_stack.pop()

        self._sync_pil_from_rgba()
        self._save_edited_preview()
        self._current_mode = "after"
        self._interaction_mode = "edit"
        self._load_current_mode_pixmap()

    def redo(self):
        if not self._redo_stack or self._editable_rgba is
None:
            return

        self._undo_stack.append(self._editable_rgba.copy())
        self._editable_rgba = self._redo_stack.pop()

        self._sync_pil_from_rgba()
        self._save_edited_preview()
        self._current_mode = "after"
        self._interaction_mode = "edit"
        self._load_current_mode_pixmap()
```

Thought for 38s >

Changed file:

- `ui/preview_canvas.py`

This change makes **Undo** able to go all the way back to the **original image** baseline, not just a few recent edit states.

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)

        self._current_pixmap = None
```


What to test

Undo to original

- load an image
- make several erase strokes
- click **Undo** repeatedly
- confirm it now keeps going all the way back to the original image state

Redo after full undo

- after undoing all the way back to original
- click **Redo**
- confirm the edits come back forward step by step

Processed image case

- process an image first
- edit it
- undo repeatedly

- confirm undo still goes all the way back to the original image baseline

Restore tool still works

- erase some area
- use **Restore**
- confirm it still restores from the original image

No regression on editing

- edit with drag strokes
- confirm edits still save
- confirm preview still updates
- confirm switching between Before/After still works



ok, all is good, what is next

Thought for a second

Next step should be **Phase 26: export polish and safety checks.**

You now have the main workflow working well:

- load images
- preview/edit
- undo/redo
- process/export split
- export without forced processing for pre-process brush edits
- ZIP-only export
- file panel original/working toggle
- desktop launcher

So the best next move is not another UI reshuffle. It is to harden the export behavior and edge cases.

Best next targets

1. Add an export summary dialog or log block

At the end of export, show a cleaner summary such as:

- total selected
- exported
- failed
- ZIP created or not
- output folder path

This gives a cleaner finish to batch jobs.

2. Prevent accidental re-export confusion

Right now repeated exports of the same file create:

- `_u2netp_nobg.png`
- `_u2netp_nobg_1.png`
- etc.

That is safe, but can get messy.

A useful next improvement would be either:

- warn when exporting duplicates into the same folder, or
- add an option like `Overwrite existing exports`

3. Validate export behavior across all cases

This is probably the most practical next testing phase:

- export with no edits and no process
- export with edits before process
- process, then edit, then export
- batch export selected
- batch export all
- ZIP on and off
- each rembg model at least once

That will likely reveal the next real bug.

4. Clean up long filename handling

Your log shows very long source filenames. That can become annoying in:

- output names
- ZIP contents
- file panel text

A good next step would be:

- optionally limit output filename length while keeping it unique

5. Add a safer "Reset edits" action

You already have undo and restore, but a dedicated reset action for the current image would be useful:

- reset working image back to original
- clear edited preview cache for that image
- keep the file loaded

That would be a strong quality-of-life feature.

Best immediate next step

The strongest next coding step is:

Add a "Reset Edits" action for the current image.

Why this is the best next step:

- low risk
- very useful
- fits your current workflow
- makes experimentation safer
- pairs well with the new undo/export behavior

After that

Then I would do:

duplicate export / overwrite handling

because that is the next likely annoyance in real use.

If you want, the next thing I can help you build is the **Reset Edits** button and behavior.



yes

Thought for 33s >

Changed files:

- `ui/preview_canvas.py`
- `ui/main_window.py`

This adds a **Reset Edits** button for the current image.

Behavior:

- if the image has **not** been processed, Reset Edits returns it to the **original**
- if the image **has** been processed, Reset Edits returns it to the **processed working image**
- it clears the edit state for that image in `main_window.py`

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)
```


ui/main_window.py **changes**

1) In `_connect_signals()`

Before

Python

Run

```
self.preview_canvas.image_edited.connect(self._on_preview_image_ed
```

After

Python

Run

```
self.preview_canvas.image_edited.connect(self._on_preview_image_ed
self.preview_canvas.edits_reset.connect(self._on_preview_edits_res
```

2) Add this new function inside `MainWindow`

Add it directly **below** `_on_preview_image_edited()` .

Python

Run

```
def _on_preview_edits_reset(self):
    selected_index = self.state.selected_index
    if selected_index is None:
        return

    item = self.state.get_item(selected_index)
    if item is None:
        return

    if item.edited_preview_path is not None:
        try:
            edited_path = Path(item.edited_preview_path)
            if edited_path.exists():
                edited_path.unlink()
        except Exception as e:
            self.log(f"Could not remove edited preview cache: {e}")

    item.edited_preview_path = None
    item.extra.pop("preprocess_edit_preview_path", None)
    item.extra.pop("preprocess_edit_base_preview_path", None)

    if item.processed_preview_path is not None and Path(item.processed_preview_path).exists():
        item.status = "processed"
    else:
        item.status = "pending"

    self.rebuild_file_panel(reload_preview=False)
    self.select_item(selected_index)
    self.log(f"Reset edits: {item.filename}")
```

What to test

Reset Edits on unprocessed image

- load an image
- edit it without processing
- click **Reset Edits**
- confirm it goes back to the original image
- confirm the item no longer behaves like it has manual pre-process edits

Reset Edits on processed image

- process an image

- edit it
- click **Reset Edits**
- confirm it returns to the processed working image, not the manually edited one

Reset clears edit state

- after using **Reset Edits**
- export the image
- confirm it behaves as a normal non-edited export for that current state

Undo/redo after reset

- make edits
- click **Reset Edits**
- confirm undo stack is cleared
- make a new edit after reset
- confirm undo/redo starts fresh from there

File panel / preview refresh

- after **Reset Edits**
- confirm the file panel thumbnail updates correctly
- confirm the preview remains on the same selected image



when i process image, use brush, click undo, and then press reset edits, it resets to the processed image. is it supposed to do that? other than that, whats next

Thought for a couple of seconds

Yes. That is the intended behavior with the current design.

Current reset logic is:

- **unprocessed image** → Reset Edits goes back to the **original**

- **processed image** → Reset Edits goes back to the **processed image**

So in your example:

1. process image
2. brush edit
3. undo
4. reset edits

Reset Edits returning to the **processed image** is correct, because once that image has a processed working baseline, reset uses that baseline, not the original.

If you want different behavior, there are two possible versions:

- **current behavior:** Reset Edits = reset only manual brush edits
- **alternate behavior:** Reset Edits = always go all the way back to original, even after processing

Right now you have the first version.

What is next

The best next step is:

Add an Export Overwrite option

Why this is the best next step:

- it solves a real workflow annoyance
- it is smaller and safer than a large UI change
- it prevents folders filling up with:
 - _u2netp_nobg.png
 - _u2netp_nobg_1.png
 - _u2netp_nobg_2.png

A good version would be:

- add a checkbox in settings:
 - Overwrite existing exports
- when checked:
 - export reuses the same output filename instead of making numbered duplicates

- when unchecked:
 - keep the current safe numbered behavior

After that, the next likely step would be:

- optional shorter output filenames for very long source names

If you want, I'll do the **Overwrite existing exports** step next.



sounds good but before you do that, what other checkbox options could be there

Useful checkbox options for this app would be:

Export behavior

- **Overwrite existing exports**
- **Export successful results as ZIP**
- **Open output folder when export finishes**
- **Clear exported items after successful batch**
- **Skip already exported items in batch**
- **Keep cache files after export**
- **Delete temporary cache files after export**

Process behavior

- **Skip already processed items**
- **Auto-select next item after process/export**
- **Keep preview on current item during batch**
- **Auto-scroll to active item during batch**

Editing behavior

- **Stay in Edit Mode after undo/redo**
- **Show original thumbnails in file panel**
- **Auto-save edits immediately**

- **Confirm before Reset Edits**

File loading / panel behavior

- **Include subfolders on drag/drop**
- **Check newly loaded items automatically**
- **Preserve checks when filters change**
- **Preserve scroll position when rebuilding file list**

Safety / convenience

- **Confirm before Clear All**
- **Confirm before removing selected item**
- **Log detailed worker messages**
- **Show batch summary popup when finished**

Best ones to add first

The highest-value ones are probably:

- **Overwrite existing exports**
- **Skip already exported items in batch**
- **Open output folder when export finishes**
- **Check newly loaded items automatically**
- **Show batch summary popup when finished**

Ones I would postpone

These are lower priority for now:

- **Keep cache files after export**
- **Delete temporary cache files after export**
- **Stay in Edit Mode after undo/redo** because that already mostly behaves well
- **Show original thumbnails in file panel** because you already have a direct toggle button for that

The strongest near-term checkbox set would be:

- **Overwrite existing exports**
- **Open output folder when export finishes**
- **Skip already exported items in batch**
- **Show batch summary popup when finished**

If you want, I can help you choose a small final set before we implement them.



ok add checkboxes for overwrite existing exports, skip already exported items, open output folder wen export finishes, skip already processed items,

on file panel, remove "retry failed" and add "undo all" that resets all selected images to their originals

I have the scope. This will touch the settings panel, file panel, and batch-routing logic in `main_window.py` , plus one export-path change so overwrite works correctly.

Thought for 22s >

Changed files:

- `ui/settings_panel.py`
- `ui/file_panel.py`
- `processing/export_utils.py`

`ui/main_window.py` changes are below as **before/after**.

Full updated file: `ui/settings_panel.py`

Python

Run


```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


Full updated file: `ui/file_panel.py`

Python

Run


```
from PySide6.QtCore import Qt, QSize, Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
    QSizePolicy,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()
    thumbnail_mode_changed = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.current_view_mode = "thumbnail"
        self.current_thumbnail_mode = "working"

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Files")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.include_subfolders_check = QCheckBox("Include subfolders on d")
        layout.addWidget(self.include_subfolders_check)

        controls_row = QHBoxLayout()
```


Full updated file: `processing/export_utils.py`

Python

Run


```
from pathlib import Path
from PIL import Image
import zipfile
```

```
FORMAT_MAP = {
    "PNG": ".png",
    "JPG": ".jpg",
    "WEBP": ".webp",
}
```

```
BACKGROUND_COLOR_MAP = {
    "White": (255, 255, 255),
    "Black": (0, 0, 0),
}
```

```
def sanitize_output_stem(stem: str) -> str:
    bad_chars = '<>:"/\_\|?*'
    cleaned = "".join("_" if ch in bad_chars else ch for ch in stem)
    return cleaned.strip().rstrip(".")
```

```
def build_output_path(
    source_path: Path,
    output_dir: Path,
    suffix: str,
    output_format: str,
    overwrite_existing: bool = False,
) -> Path:
    extension = FORMAT_MAP.get(output_format.upper(), ".png")
    safe_stem = sanitize_output_stem(source_path.stem)
    base_name = f"{safe_stem}{suffix}"

    candidate = output_dir / f"{base_name}{extension}"

    if overwrite_existing:
        return candidate

    counter = 1
    while candidate.exists():
        candidate = output_dir / f"{base_name}_{counter}{extension}"
        counter += 1

    return candidate
```


ui/main_window.py changes

1) In _connect_signals()

Before

Python

Run

```
def _connect_signals(self):
    self.file_panel.item_selected.connect(self.select_item_from_panel)
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
    self.file_panel.status_filter_changed.connect(self.rebuild_file_pa
    self.file_panel.type_filter_changed.connect(self.rebuild_file_pane
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)
    self.file_panel.thumbnail_mode_changed.connect(self.rebuild_file_p

    self.file_panel.remove_selected_btn.clicked.connect(self.remove_se
    self.file_panel.clear_completed_btn.clicked.connect(self.clear_com
    self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items
    self.file_panel.retry_failed_btn.clicked.connect(self.retry_failed
    self.file_panel.open_output_btn.clicked.connect(self.open_output_f

    self.settings_panel.run_action_btn.clicked.connect(self.run_select
    self.settings_panel.cancel_btn.clicked.connect(self.request_cancel
```

After

Python

Run

```
def _connect_signals(self):
    self.file_panel.item_selected.connect(self.select_item_from_panel)
    self.file_panel.sort_changed.connect(self.rebuild_file_panel)
    self.file_panel.status_filter_changed.connect(self.rebuild_file_pa
    self.file_panel.type_filter_changed.connect(self.rebuild_file_pane
    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)
    self.file_panel.thumbnail_mode_changed.connect(self.rebuild_file_p

    self.file_panel.remove_selected_btn.clicked.connect(self.remove_se
    self.file_panel.clear_completed_btn.clicked.connect(self.clear_com
    self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items
    self.file_panel.undo_all_btn.clicked.connect(self.undo_all_selecte
    self.file_panel.open_output_btn.clicked.connect(self.open_output_f

    self.settings_panel.run_action_btn.clicked.connect(self.run_select
    self.settings_panel.cancel_btn.clicked.connect(self.request_cancel
```

2) Add this new function inside MainWindow

Add it directly below `clear_completed_items()` .

Python

Run

```
def undo_all_selected_to_originals(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "Wait for processi
        return

    checked = self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing Selected", "No selected
        return

    reset_count = 0
    current_index = self.state.selected_index

    for index in checked:
        item = self.state.get_item(index)
        if item is None:
            continue

        if item.edited_preview_path is not None:
            try:
                edited_path = Path(item.edited_preview_path)
                if edited_path.exists():
                    edited_path.unlink()
            except Exception as e:
                self.log(f"Could not remove edited preview cache for {

        item.edited_preview_path = None
        item.processed_preview_path = None
        item.output_path = ""
        item.extra.pop("preprocess_edit_preview_path", None)
        item.extra.pop("preprocess_edit_base_preview_path", None)
        item.status = "pending"
        reset_count += 1

    self.rebuild_file_panel(reload_preview=False)

    if current_index is not None:
        self.select_item(current_index)

    self.log(f"Reset selected images to original: {reset_count}")
```

3) In process_checked_items()

Before

Python

Run

```
def process_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A processing job is in progress. Please wait until it is completed."
                                )
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal will be added later. Use AI Removal for now."
        )
        return

    checked = self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing Selected", "No selected items found."
                                )
        return

    self.batch_queue = [
        index for index in checked
        if 0 <= index < len(self.state.items)
        and (
            (self.state.items[index].preview_path is not None and Path(self.state.items[index].preview_path).exists())
            or self._has_edited_preview(self.state.items[index])
        )
    ]
```

After

Python

Run

```
def process_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A processing job is in progress. Please wait until it is completed."
                                )
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal will be added later. Use AI Removal for now."
        )
        return

    checked = self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing Selected", "No selected items found."
                                )
        return

    skip_processed = self.settings_panel.skip_processed_check.isChecked()

    self.batch_queue = [
        index for index in checked
        if 0 <= index < len(self.state.items)
        and (
            (self.state.items[index].preview_path is not None and Path(
                self.state.items[index].preview_path).exists()
             or self._has_edited_preview(self.state.items[index]))
            and not (
                skip_processed
                and self._has_processed_preview(self.state.items[index])
            )
        )
    ]
```

4) In process_all_items()

Before

Python

Run


```
def process_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A processing job is already running. Please wait until it completes."
        )
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images", "Load images first."
        )
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal will be added later. Use AI Removal for now."
        )
        return

    self.batch_queue = [
        index
        for index in range(len(self.state.items))
        if (
            (self.state.items[index].preview_path is not None and Path(
                self.state.items[index].preview_path).exists())
            or self._has_edited_preview(self.state.items[index])
        )
    ]
```

After

Python

Run

```
def process_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A processing job .
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images", "Load images first.
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Rem
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal will be added later. Use AI Removal for now
        )
        return

    skip_processed = self.settings_panel.skip_processed_check.isChecked

    self.batch_queue = [
        index
        for index in range(len(self.state.items))
        if (
            (self.state.items[index].preview_path is not None and Path
            or self._has_edited_preview(self.state.items[index]))
        )
        and not (
            skip_processed
            and self._has_processed_preview(self.state.items[index])
        )
    ]
```

5) In export_checked_items()

Before

Python

Run

```
def export_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A job is already in progress. Please wait until it is completed before exporting.", QMessageBox.Ok)
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before exporting.",
            QMessageBox.Ok
        )
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use AI Removal for now.",
            QMessageBox.Ok
        )
        return

    checked = self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing Selected", "No selected items found.", QMessageBox.Ok)
        return

    self.batch_queue = [index for index in checked if 0 <= index < len(self.images)]
```

After

Python

Run

```

def export_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A job is already in progress.",
                                QMessageBox.Ok)
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before exporting.",
            QMessageBox.Ok)
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use AI Removal for now.",
            QMessageBox.Ok)
        return

    checked = self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing Selected", "No selected items.",
                                QMessageBox.Ok)
        return

    skip_exported = self.settings_panel.skip_exported_check.isChecked()

    self.batch_queue = [
        index
        for index in checked
        if 0 <= index < len(self.state.items)
        and not (
            skip_exported
            and self.state.items[index].status == "exported"
        )
    ]

```

6) In export_all_items()

Before

Python

Run

```
def export_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A job is already in progress.",
                                QMessageBox.Ok)
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images", "Load images first.",
                                QMessageBox.Ok)
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before exporting.",
            QMessageBox.Ok)
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use AI Removal for now.",
            QMessageBox.Ok)
        return

    self.batch_queue = list(range(len(self.state.items)))
```

After

Python

Run

```

def export_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A job is already in progress.",
                                QMessageBox.Ok)
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images", "Load images first.",
                                QMessageBox.Ok)
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before exporting.",
            QMessageBox.Ok)
        return

    if self.settings_panel.removal_mode_combo.currentText() != "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use AI Removal for now.",
            QMessageBox.Ok)
        return

    skip_exported = self.settings_panel.skip_exported_check.isChecked()

    self.batch_queue = [
        index
        for index in range(len(self.state.items))
        if not (
            skip_exported
            and self.state.items[index].status == "exported"
        )
    ]

```

7) In _start_full_export_item()

Find the `build_output_path(...)` call inside the worker creation path and pass the `overwrite` setting through the worker constructor.

Before

Python

Run

```
self.export_worker = FullExportWorker(  
    item_index=item_index,  
    source_path=item.source_path,  
    processed_cache_path=fullres_cache_path,  
    output_dir=output_dir,  
    output_format=output_format,  
    suffix=suffix,  
    model_name=model_name,  
    background_mode=background_mode,  
    background_color_name=background_color,  
    base_preview_mask_path=base_preview_mask_path,  
    edited_preview_mask_path=edited_preview_mask_path,  
    apply_preview_edits=(  
        resolved_apply_preview_edits  
        and base_preview_mask_path is not None  
        and edited_preview_mask_path is not None  
    ),  
    skip_background_removal=skip_background_removal,  
)
```

After

Python

Run

```
self.export_worker = FullExportWorker(
    item_index=item_index,
    source_path=item.source_path,
    processed_cache_path=fullres_cache_path,
    output_dir=output_dir,
    output_format=output_format,
    suffix=suffix,
    model_name=model_name,
    background_mode=background_mode,
    background_color_name=background_color,
    base_preview_mask_path=base_preview_mask_path,
    edited_preview_mask_path=edited_preview_mask_path,
    apply_preview_edits=(
        resolved_apply_preview_edits
        and base_preview_mask_path is not None
        and edited_preview_mask_path is not None
    ),
    skip_background_removal=skip_background_removal,
    overwrite_existing=self.settings_panel.overwrite_exports_check
)
```

8) In `_finish_processing_session()`

Add automatic open-folder behavior after successful export.

Before

Python

Run


```

def _finish_processing_session(self, final_progress: int | None = None
    if self.batch_action == "export" and self.settings_panel.export_zi
        output_dir = self._get_output_dir()
        if output_dir is not None:
            ok, zip_path, err = create_batch_zip(self.batch_exported_f
            if ok:
                self.log(f"ZIP export created: {zip_path}")

            for exported_file in self.batch_exported_files:
                exported_path = Path(exported_file)
                try:
                    if exported_path.exists() and exported_path.is
                        exported_path.unlink()
                        self.log(f"Removed loose export after ZIP
                except Exception as e:
                    self.log(f"Could not remove loose export {expo
            else:
                self.log(f"ZIP export failed: {err}")

```

After

Python

Run

```

def _finish_processing_session(self, final_progress: int | None = None
    should_open_output_folder = False

    if self.batch_action == "export" and self.batch_success_count > 0:
        should_open_output_folder = self.settings_panel.open_output_af

    if self.batch_action == "export" and self.settings_panel.export_zi
        output_dir = self._get_output_dir()
        if output_dir is not None:
            ok, zip_path, err = create_batch_zip(self.batch_exported_f
            if ok:
                self.log(f"ZIP export created: {zip_path}")

            for exported_file in self.batch_exported_files:
                exported_path = Path(exported_file)
                try:
                    if exported_path.exists() and exported_path.is
                        exported_path.unlink()
                        self.log(f"Removed loose export after ZIP
                except Exception as e:
                    self.log(f"Could not remove loose export {expo
            else:
                self.log(f"ZIP export failed: {err}")

```

Then, near the end of the same function, right before these lines:

Before

Python

Run

```
self.progress_bar.setRange(0, 100)
if final_progress is not None:
    self.progress_bar.setValue(final_progress)
```

Insert this:

After add this block

Python

Run

```
if should_open_output_folder:
    output_dir = self._get_output_dir()
    if output_dir is not None and output_dir.exists():
        QDesktopServices.openUrl(QUrl.fromLocalFile(str(output_dir)
        self.log(f"Opened output folder after export: {output_dir}")
```

So that final section becomes:

Python

Run

```
self._set_processing_controls_enabled(True)

if should_open_output_folder:
    output_dir = self._get_output_dir()
    if output_dir is not None and output_dir.exists():
        QDesktopServices.openUrl(QUrl.fromLocalFile(str(output_dir)
        self.log(f"Opened output folder after export: {output_dir}")

self.progress_bar.setRange(0, 100)
if final_progress is not None:
    self.progress_bar.setValue(final_progress)
```

One more required change: workers/
full_export_worker.py

Because the worker now needs the overwrite setting, make these edits in that file.

In `__init__()`

Before

Python

Run

```
def __init__(
    self,
    item_index: int,
    source_path: Path,
    processed_cache_path: Path,
    output_dir: Path,
    output_format: str,
    suffix: str,
    model_name: str,
    background_mode: str,
    background_color_name: str,
    base_preview_mask_path: Path | None = None,
    edited_preview_mask_path: Path | None = None,
    apply_preview_edits: bool = False,
    skip_background_removal: bool = False,
):
```

After

Python

Run

```
def __init__(
    self,
    item_index: int,
    source_path: Path,
    processed_cache_path: Path,
    output_dir: Path,
    output_format: str,
    suffix: str,
    model_name: str,
    background_mode: str,
    background_color_name: str,
    base_preview_mask_path: Path | None = None,
    edited_preview_mask_path: Path | None = None,
    apply_preview_edits: bool = False,
    skip_background_removal: bool = False,
    overwrite_existing: bool = False,
):
```

Still in `__init__()`

Before

Python

Run

```
self.apply_preview_edits = apply_preview_edits
self.skip_background_removal = skip_background_removal
```

After

Python

Run

```
self.apply_preview_edits = apply_preview_edits
self.skip_background_removal = skip_background_removal
self.overwrite_existing = overwrite_existing
```

In `run()`, at the `build_output_path(...)` call

Before

Python

Run

```
output_path = build_output_path(  
    source_path=self.source_path,  
    output_dir=self.output_dir,  
    suffix=self.suffix,  
    output_format=self.output_format,  
)
```

After

Python

Run

```
output_path = build_output_path(  
    source_path=self.source_path,  
    output_dir=self.output_dir,  
    suffix=self.suffix,  
    output_format=self.output_format,  
    overwrite_existing=self.overwrite_existing,  
)
```

What to test

New checkboxes in settings

Confirm these now appear:

- Overwrite existing exports
- Skip already exported items
- Open output folder when export finishes
- Skip already processed items

Overwrite existing exports

- enable overwrite existing exports
- export the same image twice
- confirm it reuses the same filename
- disable it
- export the same image twice
- confirm numbered duplicates are created again

Skip already exported items

- export some images so they become exported
- enable Skip already exported items
- run batch export again
- confirm already exported items are skipped
- disable it
- confirm they are included again

Open output folder when export finishes

- enable Open output folder when export finishes
- run a successful export
- confirm the output folder opens automatically
- disable it
- confirm it no longer opens automatically

Skip already processed items

- process some images
- enable Skip already processed items
- run batch process again
- confirm already processed items are skipped
- disable it
- confirm they are included again

Undo All Selected

- check several images
- click Undo All Selected
- confirm each checked image is reset to its original state
- confirm processed preview state is cleared
- confirm edited preview state is cleared
- confirm status returns to pending

File panel change

- confirm Retry Failed is gone
- confirm Undo All Selected is present

Current selected image refresh

- select one of the checked images
- click Undo All Selected
- confirm the preview refreshes correctly for that currently selected image



i want to move the checkboxes to underneath "Actions"
and i want to rename "actions" to "Process/Export"

Changed file:

- `ui/settings_panel.py`

No `main_window.py` change is needed for this step.

Full updated file: `ui/settings_panel.py`

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially better",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


What to test

- confirm the section title now says `Process/Export`
- confirm these checkboxes are now below that section instead of near the output settings:
 - `Overwrite existing exports`
 - `Skip already exported items`
 - `Open output folder when export finishes`
 - `Skip already processed items`
- confirm the `Run` and `Cancel` controls still behave normally
- confirm export ZIP checkbox stayed where it was and still works



i want the zip checkbox to be with the other checkboxes. the export option isn't working. and on file panel "clear exported" and "remove selected" don't work right

Thought for 30s >

Changed file now:

- `ui/settings_panel.py`

I moved the ZIP checkbox into the **Process/Export** section with the other process/export checkboxes.

For the other two problems:

- **"export option isn't working"**
- **"clear exported" and "remove selected" don't work right**

I need the **exact wrong behavior** or the current `ui/main_window.py` again before I change that file, because "don't work right" could mean a few different things and I do not want to break the working batch logic by guessing.

Examples of what would help:

- what happens when you choose `Export` and click `Run`
- whether there is an error, no action, or the wrong action
- whether `Remove Selected` removes the wrong item, does nothing, or removes checked items instead
- whether `Clear Exported` removes too much, too little, or nothing

Full updated file: `ui/settings_panel.py`

Python

Run


```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially better",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


What to test

- confirm Export successful results as ZIP is now under Process/Export
- confirm the Process/Export section now contains:
 - Run
 - Overwrite existing exports
 - Skip already exported items
 - Open output folder when export finishes
 - Skip already processed items
 - Export successful results as ZIP
 - Cancel
- confirm export still runs
- confirm ZIP export still works
- confirm the other checkboxes still behave the same



exported images arent showing up in the folder

Thought for 7s

Most likely cause: Export successful results as ZIP **is checked**.

With the current behavior, when ZIP export is enabled:

- the app exports the images
- creates the ZIP
- then deletes the loose exported image files
- so the folder ends up with **the ZIP only**, not separate image files

That was the change you asked for earlier to avoid duplicates.

What to check

Look in the output folder for:

batch_export.zip

or

batch_export_1.zip

batch_export_2.zip

Expected behavior right now

- **ZIP checked** → only the ZIP remains
- **ZIP unchecked** → individual exported images remain in the folder

Test

- uncheck **Export successful results as ZIP**
- export one image
- confirm the exported image file appears in the folder

If the ZIP checkbox is already off and files still are not appearing, send your current:

- `ui/main_window.py`
- `workers/full_export_worker.py`

because then the issue is somewhere else in the export flow.



still isnt exporting

full export worker:

```
from pathlib import Path
```

```
from PySide6.QtCore import QObject, Signal, Slot
```

```
from processing.rembg_engine import RembgEngine
```

```
from processing.export_utils import (
```

```
    build_output_path,
```

```
    export_processed_image,
```

```
    prepare_base_export_image,
```

```
)
```

```
from processing.mask_ops import
```

```
apply_preview_edit_delta_to_fullres
```

```
class FullExportWorker(QObject):
```

```
    finished = Signal(int, str, str)
```

```
    error = Signal(int, str)
```

```
    status = Signal(str)
```

```
    def __init__(
```

```
        self,
```

```
        item_index: int,
```

```
        source_path: Path,
```

```
        processed_cache_path: Path,
```

```
        output_dir: Path,
```

```
        output_format: str,
        suffix: str,
        model_name: str,
        background_mode: str,
        background_color_name: str,
        base_preview_mask_path: Path | None = None,
        edited_preview_mask_path: Path | None = None,
        apply_preview_edits: bool = False,
        skip_background_removal: bool = False,
        overwrite_existing: bool = False,
    ):
        super().__init__()

        self.item_index = item_index
        self.source_path = Path(source_path)
        self.processed_cache_path =
Path(processed_cache_path)
        self.output_dir = Path(output_dir)

        self.output_format = output_format
        self.suffix = suffix
        self.model_name = model_name

        self.background_mode = background_mode
        self.background_color_name =
background_color_name

        self.base_preview_mask_path =
Path(base_preview_mask_path) if
base_preview_mask_path else None
        self.edited_preview_mask_path =
Path(edited_preview_mask_path) if
edited_preview_mask_path else None
        self.apply_preview_edits = apply_preview_edits
        self.skip_background_removal =
skip_background_removal
        self.overwrite_existing = overwrite_existing

    @Slot()
    def run(self):
```

```
try:
    self.status.emit(f"Full export worker started for
item index {self.item_index}")
    self.status.emit(f"Source image:
{self.source_path}")
    self.status.emit(f"Model: {self.model_name}")
    self.status.emit(f"Output format:
{self.output_format}")
    self.status.emit(f"Background mode:
{self.background_mode}")
    self.status.emit(f"Background color:
{self.background_color_name}")
    self.status.emit(f"Full-res cache path:
{self.processed_cache_path}")

    if self.skip_background_removal:
        self.status.emit("Skipping AI background
removal for export.")
        self.status.emit("Preparing full-resolution
base image from original source.")

        ok, error_message =
prepare_base_export_image(
            source_path=self.source_path,

destination_path=self.processed_cache_path,
        )

        if not ok:
            self.error.emit(
                self.item_index,
                error_message or "Preparing base full-
resolution export image failed.",
            )
            return
        else:
            self.status.emit(f"Generating full-resolution
cutout from: {self.source_path}")

            engine = RembgEngine()
```

```
        ok, error_message =
engine.remove_background_fullres(
    input_path=self.source_path,
    output_path=self.processed_cache_path,
    model_name=self.model_name,
)

if not ok:
    self.error.emit(
        self.item_index,
        error_message or "Full-resolution
background removal failed."
    )
    return

if not self.processed_cache_path.exists():
    self.error.emit(
        self.item_index,
        "Full-resolution processed image was not
created."
    )
    return

self.status.emit(f"Full-res working image ready:
{self.processed_cache_path}")

if (
    self.apply_preview_edits
    and self.base_preview_mask_path is not None
    and self.edited_preview_mask_path is not
None
):
    self.status.emit(
        f"Applying preview edit delta to full-res
export: {self.edited_preview_mask_path}"
    )

    ok, error_message =
    apply_preview_edit_delta_to_fullres(
```

```
fullres_image_path=self.processed_cache_path,

base_preview_mask_image_path=self.base_preview_mask_path,

edited_preview_mask_image_path=self.edited_preview_mask_path,
    )

    if not ok:
        self.error.emit(
            self.item_index,
            error_message or "Applying preview edits
to full-resolution export failed."
        )
        return

    self.status.emit("Preview edit delta applied to
full-resolution export.")

    output_path = build_output_path(
        source_path=self.source_path,
        output_dir=self.output_dir,
        suffix=self.suffix,
        output_format=self.output_format,
        overwrite_existing=self.overwrite_existing,
    )

    self.status.emit(f"Preparing final export:
{output_path}")

    ok, error_message = export_processed_image(

processed_image_path=self.processed_cache_path,
        output_path=output_path,
        output_format=self.output_format,
        background_mode=self.background_mode,

background_color_name=self.background_color_name,
```

```
)

if not ok:
    self.error.emit(
        self.item_index,
        error_message or "Final export failed."
    )
    return

if not Path(output_path).exists():
    self.error.emit(
        self.item_index,
        "Export finished but output file was not
created."
    )
    return

self.status.emit(f"Full export complete:
{output_path}")

self.finished.emit(
    self.item_index,
    str(self.processed_cache_path),
    str(output_path),
)

except Exception as e:
    self.error.emit(self.item_index, str(e))
```

main window:

```
from pathlib import Path
import hashlib
```

```
from PySide6.QtCore import Qt, QThread, QUrl,
QSettings
from PySide6.QtGui import QDesktopServices
from PySide6.QtWidgets import (
    QMainWindow,
    QWidget,
```

```
        QVBoxLayout,  
        QProgressBar,  
        QMessageBox,  
        QSplitter,  
        QFrame,  
        QLabel,  
        QScrollArea,  
        QTextEdit,  
    )
```

```
    from ui.file_panel import FilePanel  
    from ui.preview_canvas import PreviewCanvas  
    from ui.settings_panel import SettingsPanel
```

```
    from core.state_manager import StateManager  
    from core.models import ImageItem
```

```
    from utils.file_utils import scan_paths  
    from utils.image_utils import get_image_metadata,  
    create_thumbnail, create_preview_image  
    from workers.process_worker import ProcessWorker  
    from workers.full_export_worker import  
    FullExportWorker  
    from workers.watch_folder_worker import  
    WatchFolderWorker  
    from processing.export_utils import create_batch_zip
```

```
class MainWindow(QMainWindow):
```

```
    def _finalize_preview_thread(self):  
        if self.preview_worker_thread is not None and  
self.preview_worker_thread.isRunning():  
            self.preview_worker_thread.quit()  
            self.preview_worker_thread.wait(10000)  
  
        if self.preview_worker is not None:  
            self.preview_worker.deleteLater()  
            self.preview_worker = None
```



```
        if self.preview_worker_thread is not None:
            self.preview_worker_thread.deleteLater()
            self.preview_worker_thread = None

    def _finalize_export_thread(self):
        if self.export_worker_thread is not None and
self.export_worker_thread.isRunning():
            self.export_worker_thread.quit()
            self.export_worker_thread.wait(10000)

        if self.export_worker is not None:
            self.export_worker.deleteLater()
            self.export_worker = None

        if self.export_worker_thread is not None:
            self.export_worker_thread.deleteLater()
            self.export_worker_thread = None

    def _shutdown_running_threads(self):
        if self.preview_worker_thread is not None and
self.preview_worker_thread.isRunning():
            self.preview_worker_thread.quit()
            self.preview_worker_thread.wait(3000)

        if self.export_worker_thread is not None and
self.export_worker_thread.isRunning():
            self.export_worker_thread.quit()
            self.export_worker_thread.wait(3000)

        if self.watch_worker is not None:
            self.watch_worker.stop()

        if self.watch_worker_thread is not None and
self.watch_worker_thread.isRunning():
            self.watch_worker_thread.quit()
            self.watch_worker_thread.wait(3000)

    def __init__(self):
        super().__init__()
        self.setWindowTitle("MyBGRemover")
```

```
self.resize(1500, 900)
self.setMinimumSize(1080, 700)

self.settings_store = QSettings("MyBGRemover",
"MyBGRemover")

self.state = StateManager()

self.preview_worker_thread = None
self.preview_worker = None
self.export_worker_thread = None
self.export_worker = None

self.watch_worker_thread = None
self.watch_worker = None

self.processing_active = False
self.current_processing_index = None

self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_mode = False
self.batch_action = None
self.cancel_requested = False
self.batch_exported_files = []

self.visible_indices = []
self._next_date_added = 0

self.cache_root = Path("cache")
self.thumb_cache_dir = self.cache_root / "thumbs"
self.preview_cache_dir = self.cache_root /
"previews"
self.processed_preview_cache_dir =
self.cache_root / "processed"
self.edited_preview_cache_dir = self.cache_root /
"edited_previews"
self.fullres_export_cache_dir = self.cache_root /
"fullres_exports"
```

```
        self.thumb_cache_dir.mkdir(parents=True,
exist_ok=True)
        self.preview_cache_dir.mkdir(parents=True,
exist_ok=True)

self.processed_preview_cache_dir.mkdir(parents=True,
exist_ok=True)
        self.edited_preview_cache_dir.mkdir(parents=True,
exist_ok=True)
        self.fullres_export_cache_dir.mkdir(parents=True,
exist_ok=True)

self.setAcceptDrops(True)

central = QWidget()
self.setCentralWidget(central)

outer_layout = QVBoxLayout(central)
outer_layout.setContentsMargins(8, 8, 8, 8)
outer_layout.setSpacing(8)

main_vertical_splitter =
QSplitter(Qt.Orientation.Vertical)
    content_splitter =
QSplitter(Qt.Orientation.Horizontal)

self.file_panel = FilePanel()
self.file_panel.setMinimumWidth(360)
self.file_panel.setMaximumWidth(520)

self.preview_canvas = PreviewCanvas()
self.preview_canvas.setMinimumWidth(300)

self.settings_panel = SettingsPanel()
self.settings_scroll = QScrollArea()
self.settings_scroll.setWidget(self.settings_panel)
self.settings_scroll.setWidgetResizable(True)
self.settings_scroll.setMinimumWidth(320)
self.settings_scroll.setMaximumWidth(380)
```

```
self.settings_scroll.setFrameShape(QFrame.Shape.NoFrame)
```

```
content_splitter.addWidget(self.file_panel)
content_splitter.addWidget(self.preview_canvas)
content_splitter.addWidget(self.settings_scroll)
content_splitter.setSizes([430, 780, 340])
content_splitter.setStretchFactor(0, 0)
content_splitter.setStretchFactor(1, 1)
content_splitter.setStretchFactor(2, 0)
```

```
content_container = QWidget()
content_layout = QVBoxLayout(content_container)
content_layout.setContentsMargins(0, 0, 0, 0)
content_layout.setSpacing(8)
```

```
self.progress_bar = QProgressBar()
self.progress_bar.setValue(0)
self.progress_bar.setMinimumHeight(22)
self.progress_bar.setMaximumHeight(24)
```

```
content_layout.addWidget(content_splitter, 1)
content_layout.addWidget(self.progress_bar, 0)
```

```
self.log_frame = QFrame()
self.log_frame.setObjectName("logFrame")
```

```
self.log_frame.setFrameShape(QFrame.Shape.StyledPanel)
```

```
self.log_frame.setMinimumHeight(100)
```

```
log_layout = QVBoxLayout(self.log_frame)
log_layout.setContentsMargins(10, 10, 10, 10)
log_layout.setSpacing(8)
```

```
self.log_title = QLabel("Log Panel")
self.log_title.setObjectName("panelTitle")
```

```
self.log_panel = QTextEdit()
```

```
        self.log_panel.setReadOnly(True)
        self.log_panel.setPlaceholderText("Status / log
activity will appear here...")

        log_layout.addWidget(self.log_title)
        log_layout.addWidget(self.log_panel)

    main_vertical_splitter.addWidget(content_container)
    main_vertical_splitter.addWidget(self.log_frame)
    main_vertical_splitter.setSizes([680, 170])
    main_vertical_splitter.setStretchFactor(0, 1)
    main_vertical_splitter.setStretchFactor(1, 0)

    outer_layout.addWidget(main_vertical_splitter, 1)

    self._apply_basic_styles()
    self._connect_signals()
    self._restore_persistent_settings()
    self._sync_preview_tool_settings()
    self._update_watch_status_label(False, "")
    self.preview_canvas.clear_preview()

    self.log("MyBGRemover started.")
    self.log("Drag and drop files or folders into the
window.")
    self.log("Process = preview/background removal
only.")
    self.log("Export = final full-resolution file output.")
    self.log("Preview processing uses preview-size
images for speed.")
    self.log("Final export uses the original full-
resolution source image.")
    self.log("Output folder is preserved between
sessions.")
    self.log("ZIP export is available for batch runs.")

    def _apply_basic_styles(self):
        self.setStyleSheet("""
            QWidget {
```

```
        background: #2b2b2b;
        color: #f0f0f0;
    }
    QFrame#filePanel, QFrame#previewCanvas,
    QFrame#settingsPanel, QFrame#logFrame {
        border: 2px solid #808080;
        border-radius: 6px;
        background: #353535;
    }
    QLabel#panelTitle {
        font-size: 16px;
        font-weight: bold;
        color: #ffffff;
        padding: 2px 0 6px 0;
    }
    QListWidget, QTextEdit, QLineEdit,
    QComboBox, QScrollArea {
        background: #1f1f1f;
        color: #f0f0f0;
        border: 1px solid #666;
    }
    QPushButton {
        background: #4a4a4a;
        color: #ffffff;
        border: 1px solid #777;
        padding: 6px;
    }
    QPushButton:hover {
        background: #5a5a5a;
    }
    QPushButton:disabled {
        background: #3a3a3a;
        color: #999999;
    }
    QCheckBox, QLabel {
        color: #f0f0f0;
    }
    QProgressBar {
        border: 1px solid #666;
        background: #1f1f1f;
```

```
        color: #ffffff;
        text-align: center;
    }
    """

def _connect_signals(self):

    self.file_panel.item_selected.connect(self.select_item_from_panel)

    self.file_panel.sort_changed.connect(self.rebuild_file_panel)

    self.file_panel.status_filter_changed.connect(self.rebuild_file_panel)

    self.file_panel.type_filter_changed.connect(self.rebuild_file_panel)

    self.file_panel.view_mode_changed.connect(self.rebuild_file_panel)

    self.file_panel.thumbnail_mode_changed.connect(self.rebuild_file_panel)

    self.file_panel.remove_selected_btn.clicked.connect(self.remove_selected_item)

    self.file_panel.clear_completed_btn.clicked.connect(self.clear_completed_items)

    self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items)

    self.file_panel.undo_all_btn.clicked.connect(self.undo_all_selected_to_originals)

    self.file_panel.open_output_btn.clicked.connect(self.open_output_folder)
```

```
self.settings_panel.run_action_btn.clicked.connect(self.  
run_selected_action)
```

```
self.settings_panel.cancel_btn.clicked.connect(self.requ  
est_cancel)
```

```
self.settings_panel.tool_combo.currentTextChanged.co  
nnect(self._sync_preview_tool_settings)
```

```
self.settings_panel.apply_mode_combo.currentTextCha  
nged.connect(self._sync_preview_tool_settings)
```

```
self.settings_panel.brush_size_slider.valueChanged.con  
nect(self._sync_preview_tool_settings)
```

```
self.settings_panel.softness_slider.valueChanged.conne  
ct(self._sync_preview_tool_settings)
```

```
self.settings_panel.opacity_slider.valueChanged.conne  
ct(self._sync_preview_tool_settings)
```

```
self.settings_panel.spacing_slider.valueChanged.conne  
ct(self._sync_preview_tool_settings)
```

```
self.settings_panel.tolerance_slider.valueChanged.conn  
ect(self._sync_preview_tool_settings)
```

```
self.settings_panel.watch_enable_check.toggled.conne  
ct(self._on_watch_enable_toggled)
```

```
self.settings_panel.output_dir_edit.textChanged.conne  
ct(self._persist_output_dir)
```

```
self.settings_panel.naming_edit.textChanged.connect(s  
elf._persist_naming_suffix)
```



```
self.preview_canvas.image_edited.connect(self._on_preview_image_edited)
```

```
self.preview_canvas.edits_reset.connect(self._on_preview_edits_reset)
```

```
def _restore_persistent_settings(self):
    output_dir = self.settings_store.value("output_dir",
    "", type=str)
    naming_suffix =
self.settings_store.value("naming_suffix", "_nobg",
type=str)
```

```
    if output_dir:
```

```
self.settings_panel.output_dir_edit.setText(output_dir)
    if naming_suffix:
```

```
self.settings_panel.naming_edit.setText(naming_suffix)
```

```
def _persist_output_dir(self, text: str):
    self.settings_store.setValue("output_dir", text)
```

```
def _persist_naming_suffix(self, text: str):
    self.settings_store.setValue("naming_suffix", text)
```

```
def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(
```

```
tool_name=self.settings_panel.tool_combo.currentText
()),
```

```
apply_mode=self.settings_panel.apply_mode_combo.cu
rrentText(),
```

```
brush_size=self.settings_panel.brush_size_slider.value(
),
```

```
softness=self.settings_panel.softness_slider.value(),
```

```
opacity=self.settings_panel.opacity_slider.value(),

spacing=self.settings_panel.spacing_slider.value(),

tolerance=self.settings_panel.tolerance_slider.value(),
    )

    def run_selected_action(self):
        action =
self.settings_panel.action_combo.currentText()
        target =
self.settings_panel.target_combo.currentText()

        if action == "Process":
            if target == "Selected":
                self.process_checked_items()
            else:
                self.process_all_items()
        else:
            if target == "Selected":
                self.export_checked_items()
            else:
                self.export_all_items()

    def _short_text(self, text: str, max_len: int = 48) ->
str:
        return text if len(text) <= max_len else
text[:max_len - 3] + "..."

    def _cache_token(self, path: Path) -> str:
        return
hashlib.sha1(str(path.resolve()).encode("utf-8")).hexdigest()[:16]

    def _get_output_dir(self) -> Path | None:
        output_dir_text =
self.settings_panel.output_dir_edit.text().strip()
        if not output_dir_text:
            return None
```

```
        output_dir = Path(output_dir_text)
    try:
        output_dir.mkdir(parents=True, exist_ok=True)
    except Exception:
        return None

    return output_dir if output_dir.exists() and
output_dir.is_dir() else None

def _build_export_suffix(self) -> str:
    model_name =
self.settings_panel.model_combo.currentText().strip()
    user_suffix =
self.settings_panel.naming_edit.text().strip()

    if not user_suffix:
        user_suffix = "_nobg"

    if not user_suffix.startswith("_"):
        user_suffix = f"_{user_suffix}"

    if model_name:
        return f"_{model_name}{user_suffix}"

    return user_suffix

def _get_checked_or_all_indices(self) -> list[int]:
    checked =
self.file_panel.get_checked_user_indices()
    if checked:
        return checked
    return list(range(len(self.state.items)))

def _has_processed_preview(self, item: ImageItem) -
> bool:
    return item.processed_preview_path is not None
and Path(item.processed_preview_path).exists()

def _has_edited_preview(self, item: ImageItem) ->
```

```
bool:
    return item.edited_preview_path is not None and
    Path(item.edited_preview_path).exists()

    def _update_watch_status_label(self, enabled: bool,
    folder_path: str):
        if enabled and folder_path:

self.settings_panel.watch_status_label.setText(f"Watch
status: On ({folder_path})")
        else:

self.settings_panel.watch_status_label.setText("Watch
status: Off")

    def _start_watch_folder(self):
        folder_path =
self.settings_panel.watch_folder_edit.text().strip()
        include_subfolders =
self.settings_panel.watch_include_subfolders_check.is
Checked()

        if not folder_path:
            self.log("Watch folder could not start: no folder
selected.")

self.settings_panel.watch_enable_check.blockSignals(T
rue)

self.settings_panel.watch_enable_check.setChecked(Fal
se)

self.settings_panel.watch_enable_check.blockSignals(F
alse)
        self._update_watch_status_label(False, "")
        return

    folder = Path(folder_path)
    if not folder.exists() or not folder.is_dir():
        self.log(f"Watch folder could not start: invalid
```

```
directory: {folder_path}")

self.settings_panel.watch_enable_check.blockSignals(True)

self.settings_panel.watch_enable_check.setChecked(False)

self.settings_panel.watch_enable_check.blockSignals(False)
    self._update_watch_status_label(False, "")
    return

    if self.watch_worker_thread is not None or
self.watch_worker is not None:
        self._stop_watch_folder()

        self.watch_worker_thread = QThread(self)
        self.watch_worker = WatchFolderWorker(
            folder_path=folder_path,
            include_subfolders=include_subfolders,
            poll_interval_seconds=2.0,
        )

self.watch_worker.moveToThread(self.watch_worker_thread)

self.watch_worker_thread.started.connect(self.watch_worker.run)
    self.watch_worker.status.connect(self.log)

self.watch_worker.new_files_detected.connect(self._on_watch_new_files_detected)

self.watch_worker.error.connect(self._on_watch_error)

self.watch_worker.finished.connect(self.watch_worker_thread.quit)
```

```
self.watch_worker_thread.finished.connect(self._cleanup_watch_worker)

        self.watch_worker_thread.start()
        self._update_watch_status_label(True,
folder_path)

    def _stop_watch_folder(self):
        if self.watch_worker is not None:
            self.watch_worker.stop()

        self._update_watch_status_label(False, "")

    def _cleanup_watch_worker(self):
        if self.watch_worker is not None:
            self.watch_worker.deleteLater()
            self.watch_worker = None

        if self.watch_worker_thread is not None:
            if self.watch_worker_thread.isRunning():
                self.watch_worker_thread.quit()
                self.watch_worker_thread.wait(3000)
            self.watch_worker_thread.deleteLater()
            self.watch_worker_thread = None

    def _on_watch_enable_toggled(self, checked: bool):
        if checked:
            self._start_watch_folder()
        else:
            self._stop_watch_folder()

    def _on_watch_error(self, message: str):
        self.log(f"Watch folder error: {message}")

self.settings_panel.watch_enable_check.blockSignals(True)

self.settings_panel.watch_enable_check.setChecked(False)
```

```
self.settings_panel.watch_enable_check.blockSignals(False)
self._update_watch_status_label(False, "")

def _on_watch_new_files_detected(self, file_paths: list):
    if not file_paths:
        return

    existing_paths = {str(item.source_path.resolve())
for item in self.state.items}
    added_any = False

    for file_path_str in file_paths:
        path = Path(file_path_str)
        resolved = str(path.resolve())
        if resolved in existing_paths:
            continue

        self._add_image_item(path)
        existing_paths.add(resolved)
        added_any = True

    if added_any:
        self._refresh_type_filter_options()
        self.rebuild_file_panel()

def log(self, message: str):
    self.log_panel.append(message)

def dragEnterEvent(self, event):
    if event.mimeData().hasUrls():
        event.acceptProposedAction()
    else:
        event.ignore()

def dropEvent(self, event):
    if not event.mimeData().hasUrls():
        event.ignore()
    return
```

```
        if self.processing_active:
            self.log("Drop ignored while processing is
active.")
            event.ignore()
            return

        urls = event.mimeData().urls()
        dropped_paths = [url.toLocalFile() for url in urls if
url.isLocalFile()]
        include_subfolders =
self.file_panel.include_subfolders_check.isChecked()
        files, skipped = scan_paths(dropped_paths,
include_subfolders=include_subfolders)

        existing_paths = {str(item.source_path.resolve())
for item in self.state.items}
        duplicate_files = [f for f in files if str(f.resolve()) in
existing_paths]
        new_files = [f for f in files if str(f.resolve()) not in
existing_paths]

        if duplicate_files:
            reply = QMessageBox.question(
                self,
                "Duplicates Detected",
                f"{len(duplicate_files)} duplicate file(s) were
found.\nAdd duplicates anyway?",
                QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No,
                QMessageBox.StandardButton.No,
            )
            if reply == QMessageBox.StandardButton.Yes:
                new_files.extend(duplicate_files)

        for path in new_files:
            self._add_image_item(path)

        for path in skipped:
            self.log(f"Skipped unsupported file: {path}")
```



```
self._refresh_type_filter_options()
self.rebuild_file_panel()

event.acceptProposedAction()

def _add_image_item(self, path: Path):
    metadata = get_image_metadata(path)
    token = self._cache_token(path)
    thumb_path = self.thumb_cache_dir / f"{token}
_thumb.png"
    preview_path = self.preview_cache_dir / f"{token}
_preview.png"

    thumb_ok = create_thumbnail(path, thumb_path)
    preview_ok = create_preview_image(path,
preview_path)

    item = ImageItem(
        source_path=path,
        filename=path.name,
        file_type=path.suffix.lower(),
        dimensions=metadata["dimensions"],
        dpi=metadata["dpi"],
        thumb_path=thumb_path if thumb_ok else
None,
        preview_path=preview_path if preview_ok else
None,
        date_added=self._next_date_added,
    )
    self._next_date_added += 1
    self.state.add_item(item)

    if not preview_ok:
        self.log(f"Could not create preview: {path}")
    if not thumb_ok:
        self.log(f"Could not create thumbnail: {path}")

    self.log(f"Loaded: {path}")
```

```
def _refresh_type_filter_options(self):
    file_types = sorted({item.file_type for item in
self.state.items})
    self.file_panel.set_type_filter_options(file_types)

def _item_matches_filter(self, item: ImageItem,
status_filter: str, type_filter: str) -> bool:
    effective_status = "All" if status_filter in ("Filter
status", "") else status_filter
    effective_type = "All Types" if type_filter in ("Filter
type", "") else type_filter

    status_ok = effective_status == "All" or
item.status.lower() == effective_status.lower()
    type_ok = effective_type == "All Types" or
item.file_type == effective_type
    return status_ok and type_ok

def _sorted_indices(self) -> list[int]:
    sort_name =
self.file_panel.sort_combo.currentText()
    effective_sort = "Date Added" if sort_name ==
"Sort by" else sort_name

    indices = list(range(len(self.state.items)))

    if effective_sort == "Date Added":
        indices.sort(key=lambda i:
self.state.items[i].date_added)
    elif effective_sort == "Name":
        indices.sort(key=lambda i:
self.state.items[i].filename.lower())
    elif effective_sort == "Type":
        indices.sort(key=lambda i:
(self.state.items[i].file_type.lower(),
self.state.items[i].filename.lower()))
    elif effective_sort == "Status":
        indices.sort(key=lambda i:
(self.state.items[i].status.lower(),
self.state.items[i].filename.lower()))
```

```
        return indices

    def _build_panel_text(self, item: ImageItem) -> str:
        if self.file_panel.current_view_mode ==
            "thumbnail":
                return item.filename

        return (
            f"{self._short_text(item.filename, 42)}\n"
            f"{item.file_type} | {item.dimensions} | DPI:
{item.dpi} | {item.status}"
        )

    def rebuild_file_panel(self, *_args, reload_preview:
bool = True):
        selected_index = self.state.selected_index
        checked_indices =
set(self.file_panel.get_checked_user_indices())
        status_filter =
self.file_panel.status_filter_combo.currentText()
        type_filter =
self.file_panel.type_filter_combo.currentText()

        scroll_bar =
self.file_panel.file_view.verticalScrollBar()
        previous_scroll_value = scroll_bar.value() if
scroll_bar is not None else 0

        self.file_panel.file_view.blockSignals(True)
        self.file_panel.clear_items()
        self.visible_indices = []

        for item_index in self._sorted_indices():
            item = self.state.items[item_index]
            if not self._item_matches_filter(item,
status_filter, type_filter):
                continue

        if getattr(self.file_panel,
```

```
"current_thumbnail_mode", "working") == "original":
    thumb_path = item.thumb_path
else:
    thumb_path = item.edited_preview_path or
item.processed_preview_path or item.thumb_path

self.file_panel.add_item(
    self._build_panel_text(item),
    str(thumb_path) if thumb_path else None,
    item_index,
    checked=item_index in checked_indices,
)
self.visible_indices.append(item_index)

selected_to_show = None

if selected_index is not None and selected_index in
self.visible_indices:
    visible_row =
self.visible_indices.index(selected_index)

self.file_panel.set_current_visible_row(visible_row)
    selected_to_show = selected_index
elif self.visible_indices:
    selected_to_show = self.visible_indices[0]
    self.state.set_selected_index(selected_to_show)
    self.file_panel.set_current_visible_row(0)
else:
    self.state.set_selected_index(None)
    self.preview_canvas.clear_preview()

self.file_panel.file_view.blockSignals(False)

if scroll_bar is not None:
    scroll_bar.setValue(min(previous_scroll_value,
scroll_bar.maximum()))

if hasattr(self.file_panel, "_update_counts"):
    self.file_panel._update_counts()
```

```
        if selected_to_show is not None and
reload_preview:
            self.select_item(selected_to_show)

    def _refresh_item_display(self, _index: int):
        self.rebuild_file_panel()

    def select_item_from_panel(self, _visible_row: int):
        real_index =
self.file_panel.get_selected_user_index()
        if real_index >= 0:
            self.select_item(real_index, source="panel")

    def select_item(self, index: int, source: str | None =
None):
        item = self.state.get_item(index)
        if item is None:
            return

        self.state.set_selected_index(index)

        if source != "panel" and index in self.visible_indices:
            self.file_panel.file_view.blockSignals(True)

self.file_panel.set_current_visible_row(self.visible_indices.index(index))
            self.file_panel.file_view.blockSignals(False)

        before_path = item.preview_path
        after_path = item.edited_preview_path or
item.processed_preview_path
        editable_save_path =
self.edited_preview_cache_dir /
f"{self._cache_token(item.source_path)}_edited.png"

        preferred_interaction_mode =
getattr(self.preview_canvas, "_interaction_mode",
"preview")
        preferred_view_mode =
getattr(self.preview_canvas, "_current_mode", "after")
```

```
        if before_path and Path(before_path).exists():
            title_text = (
                f"{self._short_text(item.filename, 60)} | "
                f"{item.dimensions} | DPI: {item.dpi} | "
                f"{item.status}"
            )

            self.preview_canvas.set_preview_images(
                before_image_path=str(before_path),
                after_image_path=str(after_path) if
after_path and Path(after_path).exists() else None,
                title_text=title_text,
                default_mode="after",
                editable_save_path=str(editable_save_path),

                preferred_interaction_mode=preferred_interaction_mode,
                preferred_view_mode=preferred_view_mode,
            )
        else:
            self.preview_canvas.clear_preview()
            self.log(f"Could not load preview: {item.source_path}")

    def _on_preview_image_edited(self, save_path_str: str):
        selected_index = self.state.selected_index
        if selected_index is None:
            return

        item = self.state.get_item(selected_index)
        if item is None:
            return

        item.edited_preview_path = Path(save_path_str)

        if item.processed_preview_path is None:
            item.extra["preprocess_edit_preview_path"] = save_path_str
```

```
item.extra["preprocess_edit_base_preview_path"] = (
    str(item.preview_path) if item.preview_path is
not None else ""
)

if item.status != "processing":
    item.status = "edited"

self.rebuild_file_panel(reload_preview=False)

def _on_preview_edits_reset(self):
    selected_index = self.state.selected_index
    if selected_index is None:
        return

    item = self.state.get_item(selected_index)
    if item is None:
        return

    if item.edited_preview_path is not None:
        try:
            edited_path = Path(item.edited_preview_path)
            if edited_path.exists():
                edited_path.unlink()
        except Exception as e:
            self.log(f"Could not remove edited preview
cache: {e}")

        item.edited_preview_path = None
        item.extra.pop("preprocess_edit_preview_path",
None)

    item.extra.pop("preprocess_edit_base_preview_path",
None)

    if item.processed_preview_path is not None and
Path(item.processed_preview_path).exists():
        item.status = "processed"
    else:
```

```
        item.status = "pending"

    self.rebuild_file_panel(reload_preview=False)
    self.select_item(selected_index)
    self.log(f"Reset edits: {item.filename}")

    def remove_selected_item(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
                "Wait for processing to finish first.")
            return

        selected_index = self.state.selected_index
        if selected_index is None:
            QMessageBox.information(self, "No Selection",
                "Select an image first.")
            return

        item = self.state.get_item(selected_index)
        if item is None:
            return

        self.state.items.pop(selected_index)

        for idx, state_item in enumerate(self.state.items):
            state_item.date_added = idx
        self._next_date_added = len(self.state.items)

        self.state.set_selected_index(None)
        self._refresh_type_filter_options()
        self.rebuild_file_panel()
        self.preview_canvas.clear_preview()

        self.log(f"Removed: {item.source_path}")

    def remove_checked_items(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
                "Wait for processing to finish first.")
            return
```



```
        checked =
set(self.file_panel.get_checked_user_indices())
        if not checked:
            QMessageBox.information(self, "Nothing
Checked", "No checked files to remove.")
            return

        self.state.items = [item for idx, item in
enumerate(self.state.items) if idx not in checked]

        for idx, state_item in enumerate(self.state.items):
            state_item.date_added = idx
            self._next_date_added = len(self.state.items)

        self.state.set_selected_index(None)
        self._refresh_type_filter_options()
        self.rebuild_file_panel()
        self.preview_canvas.clear_preview()
        self.log(f"Removed {len(checked)} checked
item(s).")

    def clear_completed_items(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
"Wait for processing to finish first.")
            return

        before_count = len(self.state.items)
        self.state.items = [item for item in self.state.items
if item.status != "exported"]

        for idx, state_item in enumerate(self.state.items):
            state_item.date_added = idx
            self._next_date_added = len(self.state.items)

        removed_count = before_count -
len(self.state.items)
        self.state.set_selected_index(None)
        self._refresh_type_filter_options()
```

```
self.rebuild_file_panel()
self.preview_canvas.clear_preview()

self.log(f"Cleared exported items:
{removed_count}")

def undo_all_selected_to_originals(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing",
"Wait for processing to finish first.")
        return

    checked =
self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing
Selected", "No selected files to reset.")
        return

    reset_count = 0
    current_index = self.state.selected_index

    for index in checked:
        item = self.state.get_item(index)
        if item is None:
            continue

        if item.edited_preview_path is not None:
            try:
                edited_path =
Path(item.edited_preview_path)
                if edited_path.exists():
                    edited_path.unlink()
            except Exception as e:
                self.log(f"Could not remove edited preview
cache for {item.filename}: {e}")

        item.edited_preview_path = None
        item.processed_preview_path = None
        item.output_path = ""
```

```
        item.extra.pop("preprocess_edit_preview_path",
None)

    item.extra.pop("preprocess_edit_base_preview_path",
None)
    item.status = "pending"
    reset_count += 1

    self.rebuild_file_panel(reload_preview=False)

    if current_index is not None:
        self.select_item(current_index)

    self.log(f"Reset selected images to original:
{reset_count}")

    def uncheck_completed_items(self):
        if not hasattr(self.file_panel, "file_view"):
            return

        self.file_panel.file_view.blockSignals(True)
        for i in range(self.file_panel.file_view.count()):
            item = self.file_panel.file_view.item(i)
            if item is None:
                continue

            index = item.data(Qt.ItemDataRole.UserRole)
            if index is None:
                continue

            state_item = self.state.get_item(index)
            if state_item is not None and state_item.status
== "exported":

        item.setCheckState(Qt.CheckState.Unchecked)

        self.file_panel.file_view.blockSignals(False)

        if hasattr(self.file_panel,
"_sync_master_checkbox"):
```

```
        self.file_panel._sync_master_checkbox()
    if hasattr(self.file_panel, "_update_counts"):
        self.file_panel._update_counts()

    self.log("Unchecked exported items.")

    def retry_failed_items(self):
        failed_indices = [idx for idx, item in
            enumerate(self.state.items) if item.status == "error"]
        if not failed_indices:
            self.log("No failed items to retry.")
            return

        for idx in failed_indices:
            item = self.state.items[idx]
            item.status = "pending"
            item.processed_preview_path = None
            item.edited_preview_path = None
            item.output_path = ""
            item.extra.pop("preprocess_edit_preview_path",
                None)

            item.extra.pop("preprocess_edit_base_preview_path",
                None)

            self.rebuild_file_panel()
            self.log(f"Reset failed items to pending:
{len(failed_indices)}")

        def open_output_folder(self):
            output_dir = self._get_output_dir()
            if output_dir is not None and output_dir.exists():

                QDesktopServices.openUrl(QUrl.fromLocalFile(str(output_dir)))
                self.log(f"Opened output folder: {output_dir}")
                return

            selected_index = self.state.selected_index
            if selected_index is not None:
```

```
        item = self.state.get_item(selected_index)
        if item is not None and
item.source_path.parent.exists():

QDesktopServices.openUrl(QUrl.fromLocalFile(str(item
.source_path.parent)))
        self.log(f"Opened source folder for selected
image: {item.source_path.parent}")
        return

    QMessageBox.information(
        self,
        "Folder Not Available",
        "No valid output folder is set, and no selected
image folder could be opened.",
    )

    def process_selected_item(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing",
            "An image is already being processed.")
            return

        selected_index = self.state.selected_index
        if selected_index is None:
            QMessageBox.information(self, "No Selection",
            "Select an image first.")
            return

        item = self.state.get_item(selected_index)
        if item is None:
            QMessageBox.information(self, "No Selection",
            "Select an image first.")
            return

        if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
            QMessageBox.information(
                self,
```

```
        "Not Yet Available",
        "Color Removal will be added later. Use AI
Removal for now.",
    )
    return
```

```
self.batch_mode = False
self.batch_action = "process"
self.batch_queue = []
self.batch_total = 1
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.batch_exported_files = []
```

```
self._start_preview_processing_item(selected_index,
force_reprocess=True)
```

```
def process_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
processing job is already active.")
        return
```

```
    if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal will be added later. Use AI
Removal for now.",
        )
    return
```

```
    checked =
self.file_panel.get_checked_user_indices()
    if not checked:
```

```
        QMessageBox.information(self, "Nothing  
Selected", "No selected files to process.")  
        return  
  
        skip_processed =  
self.settings_panel.skip_processed_check.isChecked()  
  
        self.batch_queue = [  
            index for index in checked  
            if 0 <= index < len(self.state.items)  
            and (  
                (self.state.items[index].preview_path is not  
None and  
Path(self.state.items[index].preview_path).exists())  
                or  
self._has_edited_preview(self.state.items[index])  
            )  
            and not (  
                skip_processed  
                and  
self._has_processed_preview(self.state.items[index])  
            )  
        ]  
  
        if not self.batch_queue:  
            QMessageBox.information(  
                self,  
                "No Valid Images",  
                "The selected files do not have valid preview  
data to process.",  
            )  
            return  
  
        self.batch_total = len(self.batch_queue)  
        self.batch_processed_count = 0  
        self.batch_success_count = 0  
        self.batch_error_count = 0  
        self.batch_mode = True  
        self.batch_action = "process"  
        self.cancel_requested = False
```

```
self.batch_exported_files = []

self.log(f"Starting batch process for
{self.batch_total} selected image(s).")
self._start_next_batch_item()

def process_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
processing job is already active.")
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images",
"Load images first.")
        return

    if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal will be added later. Use AI
Removal for now.",
        )
        return

    skip_processed =
self.settings_panel.skip_processed_check.isChecked()

    self.batch_queue = [
        index
        for index in range(len(self.state.items))
        if (
            (self.state.items[index].preview_path is not
None and
Path(self.state.items[index].preview_path).exists())
            or
self._has_edited_preview(self.state.items[index])
```



```
        )
        and not (
            skip_processed
            and
self._has_processed_preview(self.state.items[index])
        )
    ]

    if not self.batch_queue:
        QMessageBox.information(self, "No Valid
Images", "There are no valid images to process.")
        return

    self.batch_mode = True
    self.batch_action = "process"
    self.batch_total = len(self.batch_queue)
    self.batch_processed_count = 0
    self.batch_success_count = 0
    self.batch_error_count = 0
    self.cancel_requested = False
    self.batch_exported_files = []

    self.log(f"Starting batch processing for
{self.batch_total} image(s).")
    self._start_next_batch_item()

    def export_selected_item(self):
        if self.processing_active:
            QMessageBox.information(self, "Processing", "A
job is already active.")
            return

        selected_index = self.state.selected_index
        if selected_index is None:
            QMessageBox.information(self, "No Selection",
"Select an image first.")
            return

        item = self.state.get_item(selected_index)
        if item is None:
```

```
        QMessageBox.information(self, "No Selection",
                                "Select an image first.")
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before
exporting.",
        )
        return

    if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use
AI Removal for now.",
        )
        return

    self.batch_mode = False
    self.batch_action = "export"
    self.batch_queue = []
    self.batch_total = 1
    self.batch_processed_count = 0
    self.batch_success_count = 0
    self.batch_error_count = 0
    self.cancel_requested = False
    self.batch_exported_files = []

    self._start_export_item(selected_index)

def export_checked_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
job is already active.")
```

```
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before
exporting.",
        )
        return

    if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
        QMessageBox.information(
            self,
            "Not Yet Available",
            "Color Removal export will be added later. Use
AI Removal for now.",
        )
        return

    checked =
self.file_panel.get_checked_user_indices()
    if not checked:
        QMessageBox.information(self, "Nothing
Selected", "No selected files to export.")
        return

    skip_exported =
self.settings_panel.skip_exported_check.isChecked()

    self.batch_queue = [
        index
        for index in checked
        if 0 <= index < len(self.state.items)
        and not (
            skip_exported
            and self.state.items[index].status ==
"exported"
```

```
    )
]

if not self.batch_queue:
    QMessageBox.information(self, "No Valid
Images", "The selected files are not valid for export.")
    return

self.batch_total = len(self.batch_queue)
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.batch_mode = True
self.batch_action = "export"
self.cancel_requested = False
self.batch_exported_files = []

self.log(f"Starting batch export for
{self.batch_total} selected image(s).")
self._start_next_batch_item()

def export_all_items(self):
    if self.processing_active:
        QMessageBox.information(self, "Processing", "A
job is already active.")
        return

    if not self.state.items:
        QMessageBox.information(self, "No Images",
"Load images first.")
        return

    if self._get_output_dir() is None:
        QMessageBox.information(
            self,
            "Output Folder Required",
            "Choose a valid output folder before
exporting.",
        )
        return
```

```
        if
self.settings_panel.removal_mode_combo.currentText()
!= "AI Removal":
    QMessageBox.information(
        self,
        "Not Yet Available",
        "Color Removal export will be added later. Use
AI Removal for now.",
    )
    return
```

```
        skip_exported =
self.settings_panel.skip_exported_check.isChecked()
```

```
        self.batch_queue = [
            index
            for index in range(len(self.state.items))
            if not (
                skip_exported
                and self.state.items[index].status ==
"exported"
            )
        ]
        self.batch_mode = True
        self.batch_action = "export"
        self.batch_total = len(self.batch_queue)
        self.batch_processed_count = 0
        self.batch_success_count = 0
        self.batch_error_count = 0
        self.cancel_requested = False
        self.batch_exported_files = []
```

```
        self.log(f"Starting batch export for
{self.batch_total} image(s).")
        self._start_next_batch_item()
```

```
def request_cancel(self):
    if not self.processing_active:
        self.log("No active job to cancel.")
```

```
        return

        self.cancel_requested = True
        self.log("Cancel requested. The current item will
finish, then the batch will stop.")

    def _start_next_batch_item(self):
        if self.cancel_requested:
            self.log("Batch canceled by user.")
            self.log(
                f"Batch summary: completed
{self.batch_processed_count}/{self.batch_total}, "
                f"succeeded {self.batch_success_count},
failed {self.batch_error_count}."
            )
            self._finish_processing_session()
            return

        if not self.batch_queue:
            if self.batch_action == "export":
                self.log("Batch export complete.")
            else:
                self.log("Batch processing complete.")

            self.log(
                f"Batch summary: completed
{self.batch_processed_count}/{self.batch_total}, "
                f"succeeded {self.batch_success_count},
failed {self.batch_error_count}."
            )

        self._finish_processing_session(final_progress=100)
        return

        next_index = self.batch_queue.pop(0)

        if self.batch_action == "export":
            self._start_export_item(next_index)
        else:
            self._start_preview_processing_item(next_index,
```

```
force_reprocess=True)

def _set_processing_controls_enabled(self, enabled:
bool):

self.settings_panel.run_action_btn.setEnabled(enabled)
    self.settings_panel.cancel_btn.setVisible(not
enabled)
    self.settings_panel.cancel_btn.setEnabled(not
enabled)

    if not enabled:
        self._scroll_settings_to_bottom()

    if hasattr(self.file_panel, "process_checked_btn"):

self.file_panel.process_checked_btn.setEnabled(enable
d)

def _scroll_settings_to_bottom(self):
    if hasattr(self, "settings_scroll") and
self.settings_scroll is not None:
        bar = self.settings_scroll.verticalScrollBar()
        if bar is not None:
            bar.setValue(bar.maximum())

def _scroll_settings_to_bottom(self):
    if hasattr(self, "settings_scroll") and
self.settings_scroll is not None:
        bar = self.settings_scroll.verticalScrollBar()
        if bar is not None:
            bar.setValue(bar.maximum())

def _update_batch_progress_busy(self):
    completed_percent =
int((self.batch_processed_count / self.batch_total) *
100) if self.batch_total > 0 else 0
    self.progress_bar.setRange(0, 100)
    self.progress_bar.setValue(completed_percent)
```

```
def _update_batch_progress(self):
    percent = int((self.batch_processed_count /
self.batch_total) * 100) if self.batch_total > 0 else 0
    self.progress_bar.setRange(0, 100)
    self.progress_bar.setValue(percent)

def _start_preview_processing_item(self, item_index:
int, force_reprocess: bool = False):
    item = self.state.get_item(item_index)
    if item is None:
        if self.batch_mode:
            self._start_next_batch_item()
        return

    preview_input_path = item.edited_preview_path or
item.preview_path

    if item.edited_preview_path is not None and
Path(item.edited_preview_path).exists():
        item.extra["preprocess_edit_preview_path"] =
str(item.edited_preview_path)

    item.extra["preprocess_edit_base_preview_path"] = (
        str(item.preview_path) if item.preview_path is
not None else ""
    )
    else:
        item.extra.pop("preprocess_edit_preview_path",
None)

    item.extra.pop("preprocess_edit_base_preview_path",
None)

    if preview_input_path is None or not
Path(preview_input_path).exists():
        self.log(f"Cannot process preview for:
{item.source_path}")
        item.status = "error"
        self._refresh_item_display(item_index)
```



```
        if self.batch_mode:
            self.batch_processed_count += 1
            self._update_batch_progress()
            self._start_next_batch_item()
        return

    if self.processing_active:
        self.log("Processing is already active; new start
request ignored.")
        return

    if self._has_processed_preview(item) and not
force_reprocess:
        item.status = "processed"
        self._refresh_item_display(item_index)
        self.select_item(item_index)

    if self.batch_mode:
        self.batch_processed_count += 1
        self._update_batch_progress()
        self._start_next_batch_item()
    return

    model_name =
self.settings_panel.model_combo.currentText()
    token = self._cache_token(item.source_path)
    processed_preview_path =
self.processed_preview_cache_dir / f"{token}
_processed.png"

    self.current_processing_index = item_index
    item.status = "processing"
    self._refresh_item_display(item_index)
    self.select_item(item_index)

    self.processing_active = True
    self._set_processing_controls_enabled(False)

    if self.batch_mode:
        self._update_batch_progress_busy()
```

```
        else:
            self.progress_bar.setRange(0, 0)

            item_number = self.batch_processed_count + 1 if
self.batch_mode else 1
            item_total = self.batch_total if self.batch_mode
else 1

            self.log(f"[{item_number}/{item_total}] Processing
preview: {item.filename}")
            self.log(f"Model: {model_name}")
            self.log(f"Input preview path:
{preview_input_path}")
            self.log(f"Output preview path:
{processed_preview_path}")

            self.preview_worker_thread = QThread(self)
            self.preview_worker = ProcessWorker(
                item_index=item_index,
                input_path=Path(preview_input_path),
                output_path=processed_preview_path,
                model_name=model_name,
            )

            self.preview_worker.moveToThread(self.preview_worke
r_thread)

            self.preview_worker_thread.started.connect(self.previe
w_worker.run)
            self.preview_worker.status.connect(self.log)

            self.preview_worker.finished.connect(self._on_preview
_process_finished)

            self.preview_worker.error.connect(self._on_preview_pr
ocess_error)

            self.preview_worker.finished.connect(self.preview_wor
ker_thread.quit)
```

```
self.preview_worker.error.connect(self.preview_worker  
_thread.quit)
```

```
self.preview_worker_thread.finished.connect(self._clea  
nup_preview_worker)
```

```
self.preview_worker_thread.start()
```

```
def _start_export_item(self, item_index: int):  
    item = self.state.get_item(item_index)  
    if item is None:  
        if self.batch_mode:  
            self._start_next_batch_item()  
        return
```

```
self.current_processing_index = item_index  
item.status = "processing"  
self._refresh_item_display(item_index)  
self.select_item(item_index)
```

```
self.processing_active = True  
self._set_processing_controls_enabled(False)
```

```
if self.batch_mode:  
    self._update_batch_progress_busy()  
else:  
    self.progress_bar.setRange(0, 0)
```

```
self._start_full_export_item(item_index,  
apply_preview_edits=True)
```

```
def _cleanup_preview_worker(self):  
    if self.preview_worker is not None:  
        self.preview_worker.deleteLater()  
        self.preview_worker = None
```

```
if self.preview_worker_thread is not None and not  
self.preview_worker_thread.isRunning():  
    self.preview_worker_thread.deleteLater()
```

```
        self.preview_worker_thread = None

    def _start_full_export_item(self, item_index: int,
                                apply_preview_edits: bool = False):
        item = self.state.get_item(item_index)
        if item is None:
            self._handle_post_item_completion()
            return

        output_dir = self._get_output_dir()
        if output_dir is None:
            item.status = "error"
            self.log("Export failed: output directory is not
available.")
            self._refresh_item_display(item_index)
            self._handle_post_item_completion(error=True)
            return

        model_name =
self.settings_panel.model_combo.currentText()
        suffix = self._build_export_suffix()
        output_format =
self.settings_panel.output_format_combo.currentText()
        .upper()
        background_mode =
self.settings_panel.background_mode_combo.currentTe
xt()
        background_color =
self.settings_panel.background_color_combo.currentTe
xt()

        token = self._cache_token(item.source_path)
        fullres_cache_path = self.fullres_export_cache_dir
/ f"{token}_fullres.png"

        base_preview_mask_path = None
        edited_preview_mask_path = None
        resolved_apply_preview_edits = False
        skip_background_removal = False
```

```
        if self._has_processed_preview(item) and
self._has_edited_preview(item):
            base_preview_mask_path =
item.processed_preview_path
            edited_preview_mask_path =
item.edited_preview_path
            resolved_apply_preview_edits = True
        else:
            preprocess_edit_preview =
item.extra.get("preprocess_edit_preview_path")
            preprocess_base_preview =
item.extra.get("preprocess_edit_base_preview_path")

            if preprocess_edit_preview and
preprocess_base_preview:
                preprocess_edit_preview_path =
Path(preprocess_edit_preview)
                preprocess_base_preview_path =
Path(preprocess_base_preview)

                if preprocess_edit_preview_path.exists() and
preprocess_base_preview_path.exists():
                    base_preview_mask_path =
preprocess_base_preview_path
                    edited_preview_mask_path =
preprocess_edit_preview_path
                    resolved_apply_preview_edits = True
                    skip_background_removal = True

            item_number = self.batch_processed_count + 1 if
self.batch_mode else 1
            item_total = self.batch_total if self.batch_mode
else 1

            self.log(f"Starting full-resolution export for:
{item.filename}")
            self.log(f"Source image: {item.source_path}")
            self.log(f"Full-res temp path:
{fullres_cache_path}")
            self.log(f"Filename suffix: {suffix}")
```

```
        self.log(f"Apply preview edits: {'yes' if
resolved_apply_preview_edits and
edited_preview_mask_path else 'no'}")
        self.log(f"Skip AI background removal: {'yes' if
skip_background_removal else 'no'}")

        if base_preview_mask_path is not None:
            self.log(f"Base preview mask path:
{base_preview_mask_path}")
            if edited_preview_mask_path is not None:
                self.log(f"Edited preview mask path:
{edited_preview_mask_path}")

        self.export_worker_thread = QThread(self)
        self.export_worker = FullExportWorker(
            item_index=item_index,
            source_path=item.source_path,
            processed_cache_path=fullres_cache_path,
            output_dir=output_dir,
            output_format=output_format,
            suffix=suffix,
            model_name=model_name,
            background_mode=background_mode,
            background_color_name=background_color,

            base_preview_mask_path=base_preview_mask_path,

            edited_preview_mask_path=edited_preview_mask_path
        ,
            apply_preview_edits=(
                resolved_apply_preview_edits
                and base_preview_mask_path is not None
                and edited_preview_mask_path is not None
            ),

            skip_background_removal=skip_background_removal,

            overwrite_existing=self.settings_panel.overwrite_exports_check.isChecked(),
        )
```

```
self.export_worker.moveToThread(self.export_worker_
thread)
```

```
self.export_worker_thread.started.connect(self.export_
worker.run)
    self.export_worker.status.connect(self.log)
```

```
self.export_worker.finished.connect(self._on_full_expo
rt_finished)
```

```
self.export_worker.error.connect(self._on_full_export_
error)
```

```
self.export_worker.finished.connect(self.export_worker
_thread.quit)
```

```
self.export_worker.error.connect(self.export_worker_th
read.quit)
```

```
self.export_worker_thread.finished.connect(self._clean
up_export_worker)
```

```
    self.export_worker_thread.start()
```

```
def _cleanup_export_worker(self):
    if self.export_worker is not None:
        self.export_worker.deleteLater()
        self.export_worker = None
```

```
    if self.export_worker_thread is not None and not
self.export_worker_thread.isRunning():
        self.export_worker_thread.deleteLater()
        self.export_worker_thread = None
```

```
def _on_preview_process_finished(self, item_index:
int, processed_preview_path_str: str):
    item = self.state.get_item(item_index)
    if item is None:
```

```
        self._finalize_preview_thread()
        self._handle_post_item_completion(error=True)
        return

        processed_preview_path =
        Path(processed_preview_path_str)
        item.processed_preview_path =
        processed_preview_path
        item.edited_preview_path = None
        item.status = "processed"

        self.batch_success_count += 1
        self.log(f"Process success: {item.filename}")
        self.log(f"Preview processing complete:
        {processed_preview_path}")
        if processed_preview_path.exists():
            self.log(f"Processed preview size:
            {processed_preview_path.stat().st_size} bytes")

        self._refresh_item_display(item_index)
        self.select_item(item_index)
        self._finalize_preview_thread()
        self._handle_post_item_completion()

    def _on_preview_process_error(self, item_index: int,
    error_message: str):
        item = self.state.get_item(item_index)
        if item is not None:
            item.status = "error"
            item.processed_preview_path = None
            item.edited_preview_path = None
            self._refresh_item_display(item_index)
            self.log(f"Process failed: {item.filename}")
        else:
            self.log(f"Preview processing failed for item
            index {item_index}")

        self.batch_error_count += 1
        self.log(f"Error: {error_message}")
        self._finalize_preview_thread()
```



```
        self._handle_post_item_completion(error=True)

    def _on_full_export_finished(self, item_index: int,
                                _fullres_cache_path: str, output_path_str: str):
        item = self.state.get_item(item_index)
        if item is not None:
            item.output_path = output_path_str
            item.status = "exported"
            self.batch_success_count += 1
            self.log(f"Export success: {item.filename}")
            self.log(f"Exported: {output_path_str}")
            self._refresh_item_display(item_index)
            self.select_item(item_index)

        if self.batch_mode:

            self.batch_exported_files.append(output_path_str)

            self._finalize_export_thread()
            self._handle_post_item_completion()

    def _on_full_export_error(self, item_index: int,
                              error_message: str):
        item = self.state.get_item(item_index)
        if item is not None:
            item.status = "error"
            self._refresh_item_display(item_index)
            self.log(f"Export failed: {item.filename}")
        else:
            self.log(f"Full export failed for item index
{item_index}")

        self.batch_error_count += 1
        self.log(f"Error: {error_message}")
        self._finalize_export_thread()
        self._handle_post_item_completion(error=True)

    def _handle_post_item_completion(self, error: bool =
False):
        self.current_processing_index = None
```

```
        if self.batch_mode:
            self.batch_processed_count += 1
            self._update_batch_progress()
        else:
            self.progress_bar.setRange(0, 100)
            self.progress_bar.setValue(0 if error else 100)

    self.processing_active = False

    if self.batch_mode:
        self._start_next_batch_item()
    else:
        self._set_processing_controls_enabled(True)

    def _finish_processing_session(self, final_progress:
int | None = None):
        should_open_output_folder = False

        if self.batch_action == "export" and
self.batch_success_count > 0:
            should_open_output_folder =
self.settings_panel.open_output_after_export_check.is
Checked()

        if self.batch_action == "export" and
self.settings_panel.export_zip_check.isChecked() and
self.batch_exported_files:
            output_dir = self._get_output_dir()
            if output_dir is not None:
                ok, zip_path, err =
create_batch_zip(self.batch_exported_files, output_dir)
                if ok:
                    self.log(f"ZIP export created: {zip_path}")

        for exported_file in
self.batch_exported_files:
            exported_path = Path(exported_file)
            try:
                if exported_path.exists() and
```

```
exported_path.is_file():
    exported_path.unlink()
    self.log(f"Removed loose export
after ZIP creation: {exported_path.name}")
except Exception as e:
    self.log(f"Could not remove loose
export {exported_path.name}: {e}")
else:
    self.log(f"ZIP export failed: {err}")

self.batch_mode = False
self.batch_action = None
self.batch_queue = []
self.batch_total = 0
self.batch_processed_count = 0
self.batch_success_count = 0
self.batch_error_count = 0
self.cancel_requested = False
self.current_processing_index = None
self.batch_exported_files = []

self._set_processing_controls_enabled(True)

self._set_processing_controls_enabled(True)

if should_open_output_folder:
    output_dir = self._get_output_dir()
    if output_dir is not None and output_dir.exists():

QDesktopServices.openUrl(QUrl.fromLocalFile(str(outp
ut_dir)))
    self.log(f"Opened output folder after export:
{output_dir}")

self.progress_bar.setRange(0, 100)
if final_progress is not None:
    self.progress_bar.setValue(final_progress)

def clear_all_items(self):
    if self.processing_active:
```

```
        QMessageBox.information(
            self,
            "Processing",
            "Wait for the current processing job to finish
first.",
        )
        return

    reply = QMessageBox.question(
        self,
        "Clear All",
        "Remove all loaded items from MyBGRemover?",
        QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No,
    )
    if reply != QMessageBox.StandardButton.Yes:
        return

    self.state.clear_all()
    self.file_panel.clear_items()
    self.preview_canvas.clear_preview()
    self.progress_bar.setRange(0, 100)
    self.progress_bar.setValue(0)
    self.batch_queue = []
    self.batch_total = 0
    self.batch_processed_count = 0
    self.batch_success_count = 0
    self.batch_error_count = 0
    self.batch_mode = False
    self.batch_action = None
    self.cancel_requested = False
    self.batch_exported_files = []
    self.visible_indices = []
    self._refresh_type_filter_options()

    self.log("Cleared all items.")

def closeEvent(self, event):
    self.cancel_requested = True
```

```
self._stop_watch_folder()
self._shutdown_running_threads()
self.settings_store.sync()
super().closeEvent(event)
```

Thought for 1m 13s >

From the two files you pasted, I do **not** see a direct break in `FullExportWorker`. That part should still write the file.

The most likely causes now are:

1. Export successful results as ZIP **is still checked**
 - batch export writes files
 - then zips them
 - then deletes the loose files
2. Skip already exported items **is checked**
 - items already marked exported will be skipped in later runs
3. The app is exporting to a different output folder than the one you're checking
4. Remove Selected **and** Clear Exported **are behaving differently than you expect, but not necessarily broken**
 - Remove Selected removes the **highlighted item**, not all checked items
 - Clear Exported removes **all loaded items whose status is** exported

What I need to fix this correctly is:

- the **latest log lines** from one export attempt where it "doesn't export"
- your current `ui/settings_panel.py`

That will let me verify:

- whether ZIP is still enabled
- whether skip-exported is enabled
- whether the output folder path is the one you think it is
- whether the export action UI is wired correctly after the checkbox move

Also, for the file panel buttons, tell me which behavior you want:

- Remove Selected should remove the **highlighted** item

or

- Remove Selected should remove the **checked** items

and

- Clear Exported should clear **all exported items**

or

- Clear Exported should clear only **visible exported items**

Send:

- ui/settings_panel.py
- the latest export log from one failed attempt

and I'll give you the exact fix.



Model: u2netp

Output format: PNG

Background mode: Solid Color

Background color: White

Full-res cache path: cache/

fullres_exports/90ceff2d0a1923bb_fullres.png

Generating full-resolution cutout from: /home/yma/

Downloads/amazing_tattoo_design_A_playfu(1).jpeg

Full-res working image ready: cache/

fullres_exports/90ceff2d0a1923bb_fullres.png

Export failed: amazing_tattoo_design_A_playfu(1).jpeg

Error: build_output_path() got an unexpected keyword argument 'overwrite_existing'

Starting full-resolution export for:

amazing_tattoo_design_A_playfu.jpeg

Source image: /home/yma/Downloads/

amazing_tattoo_design_A_playfu.jpeg

Full-res temp path: cache/

fullres_exports/77c0ca9ef14405ae_fullres.png

Filename suffix: _u2netp_nobg

Apply preview edits: no

Skip AI background removal: no

Full export worker started for item index 9

```
Source image: /home/yma/Downloads/
amazing_tattoo_design_A_playfu.jpeg
Model: u2netp
Output format: PNG
Background mode: Solid Color
Background color: White
Full-res cache path: cache/
fullres_exports/77c0ca9ef14405ae_fullres.png
Generating full-resolution cutout from: /home/yma/
Downloads/amazing_tattoo_design_A_playfu.jpeg
Full-res working image ready: cache/
fullres_exports/77c0ca9ef14405ae_fullres.png
Export failed: amazing_tattoo_design_A_playfu.jpeg
Error: build_output_path() got an unexpected keyword
argument 'overwrite_existing'
Batch export complete.
Batch summary: completed 10/10, succeeded 0, failed
10
```

Thought for 8s

The error is clear:

```
build_output_path() got an unexpected keyword argument 'overwrite_exi
```

That means your app is still using an **older** `processing/export_utils.py` file that does **not** include the new `overwrite_existing` parameter.

So the export failure is not in `main_window.py` or `full_export_worker.py`.

It is this file:

- `processing/export_utils.py`

Replace that file with this full version.

Full updated file: `processing/export_utils.py`

Python

Run


```
from pathlib import Path
from PIL import Image
import zipfile

FORMAT_MAP = {
    "PNG": ".png",
    "JPG": ".jpg",
    "WEBP": ".webp",
}

BACKGROUND_COLOR_MAP = {
    "White": (255, 255, 255),
    "Black": (0, 0, 0),
}

def sanitize_output_stem(stem: str) -> str:
    bad_chars = '<>:"/\\|?*'
    cleaned = "".join("_" if ch in bad_chars else ch for ch in stem)
    return cleaned.strip().rstrip(".")

def build_output_path(
    source_path: Path,
    output_dir: Path,
    suffix: str,
    output_format: str,
    overwrite_existing: bool = False,
) -> Path:
    extension = FORMAT_MAP.get(output_format.upper(), ".png")
    safe_stem = sanitize_output_stem(source_path.stem)
    base_name = f"{safe_stem}{suffix}"

    candidate = output_dir / f"{base_name}{extension}"

    if overwrite_existing:
        return candidate

    counter = 1
    while candidate.exists():
        candidate = output_dir / f"{base_name}_{counter}{extension}"
        counter += 1

    return candidate
```


What to test

- run one normal export
- confirm the log no longer shows:
 - unexpected keyword argument 'overwrite_existing'
- test with **Overwrite existing exports** unchecked
- confirm repeated exports create numbered files
- test with **Overwrite existing exports** checked
- confirm repeated exports reuse the same filename

For the file panel buttons, based on your current code:

- Remove Selected removes the **highlighted** item only
- Clear Exported removes **all loaded items with status** exported

If that is not the behavior you want, tell me the exact behavior you want for each and I will change it.



ok it works now.. whats next

Thought for a couple of seconds

Next best step is **make the file panel actions behave exactly the way you want.**

That is the last area you flagged as feeling wrong:

- Remove Selected
- Clear Exported

Right now, based on your current code:

- Remove Selected removes the **highlighted** item
- Clear Exported removes **all loaded items** whose status is exported

A good cleanup step would be to make those labels and behaviors more explicit.

Strongest options

Option 1: keep current behavior, rename buttons

This is the safest:

- Remove Selected → Remove Highlighted
- Clear Exported → Remove Exported

That makes the UI match what the code already does.

Option 2: change behavior to checked-item workflow

This may fit your app better now that checked items are central:

- Remove Selected removes **checked** items
- Clear Exported removes only **checked exported** items, or only **visible exported** items

That would make the file panel more consistent with your Selected terminology.

My recommendation

I would do this:

- Remove Selected → remove **checked** items
- Clear Exported → remove **all exported**
- rename Remove Selected to Remove Selected
- rename Clear Exported to Clear All Exported

That gives:

- one checked-item removal action
- one bulk exported cleanup action
- less mismatch between label and behavior

After that, the next likely step would be:

- save checkbox settings with `QSettings` , so your export/process preferences persist between launches

If you want, I would do the file-panel button behavior cleanup next.



id rather redo the file panel buttons entirely. make a list of useful buttons

Useful file panel buttons for this app:

Most useful core buttons

- **Remove Selected**
Remove checked items from the file list.
- **Reset Selected to Original**
Reset checked items back to original state.
- **Clear All Exported**
Remove all items with `exported` status from the file list.
- **Clear All**
Remove everything from the file list.
- **Open Output Folder**
Open the current export folder.

Very useful optional buttons

- **Check All Visible**
You already have this as a checkbox, which is good.
- **Uncheck All Visible**
Useful if a lot of items are selected.
- **Check Exported**
Quickly select exported items.
- **Check Unprocessed**

Quickly select items still needing work.

- **Check Edited**

Quickly select items with manual edits.

- **Check Errors**

Quickly select failed items.

- **Invert Selection**

Flip checked vs unchecked for visible items.

Batch workflow buttons that may be useful

These are only worth adding if you want faster panel-based actions instead of relying on the settings panel:

- **Process Selected**
- **Export Selected**

But since you already have the `Process/Export` section on the right, I would probably **not** duplicate those in file panel.

Buttons I would avoid

These tend to create clutter or duplicate other UI:

- `Retry Failed` as a permanent button
- separate `Process Checked`
- separate `Export Checked`
- too many one-off buttons for every status

Best compact file panel button set

If the goal is clean and useful, I would recommend:

Main row

- **Remove Selected**
- **Reset Selected**
- **Open Output Folder**

Secondary row

- **Clear All Exported**
- **Clear All**

Optional small utility row

- Uncheck All Visible
- Invert Selection

My recommendation

The strongest redesign would be:

Row 1

- Remove Selected
- Reset Selected
- Open Output Folder

Row 2

- Clear All Exported
- Clear All

That keeps the file panel focused on:

- managing loaded items
- resetting work
- opening output
- cleanup

while leaving process/export execution in the settings panel where it already belongs.

If you want, I can turn that into the actual new file panel layout next.



lets do the most useful core buttons, some that are there, and i want to have a brief description of each button when the mouse hovers over it
after this is want to implement and polish the advanced brush tools

Thought for 4s

Changed files:

- `ui/file_panel.py`
- `ui/main_window.py`

This redesign keeps the file panel focused on the most useful core actions and adds hover descriptions to each button.

New file panel button set

- Remove Selected
- Reset Selected
- Clear All Exported
- Clear All
- Open Output Folder

Full updated file: `ui/file_panel.py`

Python

Run

```
from PySide6.QtCore import Qt, QSize, Signal
from PySide6.QtGui import QIcon, QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QListWidget,
    QListWidgetItem,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QCheckBox,
    QAbstractItemView,
    QSizePolicy,
)
```

```
class FilePanel(QFrame):
    item_selected = Signal(int)
    sort_changed = Signal(str)
    status_filter_changed = Signal(str)
    type_filter_changed = Signal(str)
    view_mode_changed = Signal(str)
    checked_items_changed = Signal()
    thumbnail_mode_changed = Signal(str)

    def __init__(self):
        super().__init__()

        self.setObjectName("filePanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.current_view_mode = "thumbnail"
        self.current_thumbnail_mode = "working"

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Files")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.include_subfolders_check = QCheckBox("Include subfolders on d
        self.include_subfolders_check.setToolTip(
            "When dropping folders, also load supported images from subfol
        )
```


ui/main_window.py changes

1) In `_connect_signals()`

Replace this block:

Python

Run

```
self.file_panel.remove_selected_btn.clicked.connect(self.remove_selected)  
self.file_panel.clear_completed_btn.clicked.connect(self.clear_completed)  
self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items)  
self.file_panel.undo_all_btn.clicked.connect(self.undo_all_selections)  
self.file_panel.open_output_btn.clicked.connect(self.open_output_folder)
```

With this:

Python

Run

```
self.file_panel.remove_selected_btn.clicked.connect(self.remove_selected)  
self.file_panel.reset_selected_btn.clicked.connect(self.undo_all_selections)  
self.file_panel.clear_completed_btn.clicked.connect(self.clear_completed)  
self.file_panel.clear_all_btn.clicked.connect(self.clear_all_items)  
self.file_panel.open_output_btn.clicked.connect(self.open_output_folder)
```

What changed

- Remove Selected now uses the checked-item workflow
- Reset Selected is the renamed reset button
- tooltips were added to the important file panel buttons
- Clear All Exported remains a bulk cleanup button
- Open Output Folder remains in the file panel

What to test

- hover over each file panel button and confirm a brief description appears
- check a few items, click Remove Selected , and confirm the checked items are removed
- check a few items, click Reset Selected , and confirm they reset to original state
- click Clear All Exported and confirm all exported items are removed from the file list
- click Clear All and confirm all items are removed
- click Open Output Folder and confirm it opens the correct folder

After this, the next step is a good time to implement and polish the advanced brush tools.



ok ready

Thought for 13s >

Next step should be **advanced brush tools, in this order:**

Phase 27A – make the existing brush controls actually work

Implement the controls you already have but currently hide/ignore:

- **Softness**
 - hard edge vs feathered edge brush

- **Opacity**
 - partial erase / partial restore instead of full on/off
- **Spacing**
 - smoother stroke behavior and fewer gaps while dragging

This is the best first step because the UI already has those concepts, and it upgrades the tools you already use every time.

Phase 27B – improve click tools

After that:

- **Magic Erase**
 - better region selection behavior
 - clearer connected vs global behavior
- **Background Erase**
 - smarter color tolerance behavior
 - better edge cleanup

Phase 27C – polish

Then:

- show/hide the advanced sliders only when relevant
- add tooltips explaining what softness/opacity/spacing do
- improve default values so the brush feels good immediately

Best immediate implementation target

Start with:

1. **Softness**
2. **Opacity**
3. **Spacing**

That gives the biggest real improvement with the least UI disruption.

What I need to do it cleanly

Send the current full contents of:

- `ui/preview_canvas.py`
- `ui/settings_panel.py`

I already know the general structure, but for this step I want the current exact versions so I can wire the advanced brush controls in cleanly and give you the correct full updated files.



preview canvas:

```
from io import BytesIO
from pathlib import Path
```

```
import numpy as np
from PIL import Image
```

```
from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)
```

```
class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner
```

```
def mousePressEvent(self, event):
    self.owner.handle_mouse_press(event)
    super().mousePressEvent(event)

def mouseMoveEvent(self, event):
    self.owner.handle_mouse_move(event)
    super().mouseMoveEvent(event)

def mouseReleaseEvent(self, event):
    self.owner.handle_mouse_release(event)
    super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)

        self._current_pixmap = None
        self._base_fit_scale = 1.0
        self._zoom_factor = 1.0

        self._before_image_pil = None
        self._editable_after_image_pil = None
        self._editable_rgba = None
        self._restore_source_rgba = None
        self._history_baseline_rgba = None
        self._edit_reset_baseline_rgba = None

        self._interaction_mode = "preview"
        self._current_mode = "after"

        self._tool_name = "Erase"
        self._apply_mode = "Brush"
```

```
self._brush_size = 40
self._tolerance = 35

self._magic_mode = "Connected Region"

self._mouse_down = False
self._last_image_point = None
self._drag_edit_active = False

self._undo_stack = []
self._redo_stack = []

self._editable_save_path = None
self._title_text = "Preview"

layout = QVBoxLayout(self)

self.title_label = QLabel("Preview")

toolbar = QHBoxLayout()

self.preview_mode_btn = QPushButton("Preview
Mode ✓")
self.edit_mode_btn = QPushButton("Edit Mode")

self.before_btn = QPushButton("Before")
self.after_btn = QPushButton("After ✓")

self.undo_btn = QPushButton("Undo")
self.redo_btn = QPushButton("Redo")
self.reset_edits_btn = QPushButton("Reset Edits")

self.zoom_in_btn = QPushButton("Zoom +")
self.zoom_out_btn = QPushButton("Zoom -")

toolbar.addWidget(self.preview_mode_btn)
toolbar.addWidget(self.edit_mode_btn)
toolbar.addWidget(self.before_btn)
toolbar.addWidget(self.after_btn)
```

```
        toolbar.addWidget(self.undo_btn)
        toolbar.addWidget(self.redo_btn)
        toolbar.addWidget(self.reset_edits_btn)
        toolbar.addStretch()
        toolbar.addWidget(self.zoom_in_btn)
        toolbar.addWidget(self.zoom_out_btn)

        self.image_label = _CanvasLabel(self)

        self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)

        self.image_label.setSizePolicy(QSizePolicy.Policy.Fixed,
                                       QSizePolicy.Policy.Fixed)
        self.image_label.setStyleSheet("background:
transparent;")
        self.image_label.setMouseTracking(True)
        self.image_label.hide()

        self.image_container = QWidget()
        self.image_container.setStyleSheet("background:
transparent;")
        self.image_container_layout =
QVBoxLayout(self.image_container)

        self.image_container_layout.setContentsMargins(0, 0,
0, 0)
        self.image_container_layout.addStretch()
        self.image_container_layout.addWidget(
            self.image_label,
            0,
            Qt.AlignmentFlag.AlignCenter,
        )
        self.image_container_layout.addStretch()

        self.scroll_area = QScrollArea()
        self.scroll_area.setWidget(self.image_container)
        self.scroll_area.setWidgetResizable(True)

        self.scroll_area.setAlignment(Qt.AlignmentFlag.AlignCe
```



```

        nter)
        self.scroll_area.setStyleSheet("background:
transparent; border: 0;")

self.scroll_area.viewport().setStyleSheet("background:
transparent;")

        layout.addWidget(self.title_label)
        layout.addLayout(toolbar)
        layout.addWidget(self.scroll_area)

self.preview_mode_btn.clicked.connect(self.set_previe
w_mode)

self.edit_mode_btn.clicked.connect(self.set_edit_mode)
    self.before_btn.clicked.connect(self.show_before)
    self.after_btn.clicked.connect(self.show_after)
    self.undo_btn.clicked.connect(self.undo)
    self.redo_btn.clicked.connect(self.redo)

self.reset_edits_btn.clicked.connect(self.reset_edits)
    self.zoom_in_btn.clicked.connect(self.zoom_in)
    self.zoom_out_btn.clicked.connect(self.zoom_out)

    self._update_mode_buttons()

def resizeEvent(self, event):
    super().resizeEvent(event)
    if self._current_pixmap is not None:
        self._render_current_pixmap()

def clear_preview(self, reset_title: bool = True):
    self._current_pixmap = None
    self._base_fit_scale = 1.0
    self._zoom_factor = 1.0

    self._before_image_pil = None
    self._editable_after_image_pil = None
    self._editable_rgba = None
```

```
self._restore_source_rgba = None
self._history_baseline_rgba = None
self._edit_reset_baseline_rgba = None

self._interaction_mode = "preview"
self._current_mode = "after"

self._mouse_down = False
self._last_image_point = None
self._drag_edit_active = False

self._undo_stack = []
self._redo_stack = []

self._editable_save_path = None

if reset_title:
    self._title_text = "Preview"
    self.title_label.setText("Preview")

self.image_label.clear()
self.image_label.resize(0, 0)
self.image_label.hide()
self._update_mode_buttons()

def set_magic_mode(self, mode: str):
    self._magic_mode = mode

def set_tool_settings(
    self,
    tool_name,
    apply_mode,
    brush_size,
    softness,
    opacity,
    spacing,
    tolerance,
):
    self._tool_name = tool_name
    self._apply_mode = apply_mode
```

```
        self._brush_size = brush_size
        self._tolerance = tolerance

    def set_preview_images(
        self,
        before_image_path,
        after_image_path=None,
        title_text="Preview",
        default_mode="after",
        editable_save_path=None,
        preferred_interaction_mode="preview",
        preferred_view_mode="after",
        **_,
    ):
        self.clear_preview(reset_title=False)

        self._title_text = title_text or "Preview"
        self.title_label.setText(self._title_text)

        self._editable_save_path =
        Path(editable_save_path) if editable_save_path else
        None
        if self._editable_save_path is not None:

        self._editable_save_path.parent.mkdir(parents=True,
        exist_ok=True)

        if before_image_path and
        Path(before_image_path).exists():
            with Image.open(before_image_path) as img:
                self._before_image_pil = img.convert("RGBA")

        loaded_after = None
        if after_image_path and
        Path(after_image_path).exists():
            with Image.open(after_image_path) as img:
                loaded_after = img.convert("RGBA")

        if loaded_after is not None:
            self._editable_after_image_pil = loaded_after
```

```
        elif self._before_image_pil is not None:
            self._editable_after_image_pil =
self._before_image_pil.copy()
        else:
            self._editable_after_image_pil = None

        if self._editable_after_image_pil is not None:
            self._editable_rgba =
np.array(self._editable_after_image_pil)
        else:
            self._editable_rgba = None

        if self._before_image_pil is not None:
            self._restore_source_rgba =
np.array(self._before_image_pil)
            elif self._editable_after_image_pil is not None:
                self._restore_source_rgba =
np.array(self._editable_after_image_pil)
            else:
                self._restore_source_rgba = None

        if self._restore_source_rgba is not None:
            self._history_baseline_rgba =
self._restore_source_rgba.copy()
            elif self._editable_rgba is not None:
                self._history_baseline_rgba =
self._editable_rgba.copy()
            else:
                self._history_baseline_rgba = None

        if loaded_after is not None:
            self._edit_reset_baseline_rgba =
np.array(loaded_after)
            elif self._before_image_pil is not None:
                self._edit_reset_baseline_rgba =
np.array(self._before_image_pil)
            elif self._editable_rgba is not None:
                self._edit_reset_baseline_rgba =
self._editable_rgba.copy()
            else:
```

```
        self._edit_reset_baseline_rgba = None

        valid_modes = {"before", "after"}
        chosen_view_mode = preferred_view_mode if
preferred_view_mode in valid_modes else
default_mode

        if chosen_view_mode == "before" and
self._before_image_pil is not None:
            self._current_mode = "before"
        elif self._editable_after_image_pil is not None:
            self._current_mode = "after"
        else:
            self._current_mode = "before"

        if preferred_interaction_mode == "edit" and
self._editable_after_image_pil is not None:
            self._interaction_mode = "edit"
            self._current_mode = "after"
        else:
            self._interaction_mode = "preview"

        self._zoom_factor = 1.0
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def set_preview_mode(self):
        self._interaction_mode = "preview"
        if self._editable_after_image_pil is not None:
            self._current_mode = "after"
        else:
            self._current_mode = "before"
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def set_edit_mode(self):
        if self._editable_after_image_pil is None:
            return
        self._interaction_mode = "edit"
        self._current_mode = "after"
```

```
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def show_before(self):
        if self._before_image_pil is None:
            return
        self._current_mode = "before"
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def show_after(self):
        if self._editable_after_image_pil is None:
            return
        self._current_mode = "after"
        self._load_current_mode_pixmap()
        self._update_mode_buttons()

    def zoom_in(self):
        if self._current_pixmap is None:
            return
        self._zoom_factor = min(self._zoom_factor * 1.25,
8.0)
        self._render_current_pixmap()

    def zoom_out(self):
        if self._current_pixmap is None:
            return
        self._zoom_factor = max(self._zoom_factor / 1.25,
0.1)
        self._render_current_pixmap()

    def _can_undo(self) -> bool:
        if self._undo_stack:
            return True

        if self._editable_rgba is None or
self._history_baseline_rgba is None:
            return False

        return not np.array_equal(self._editable_rgba,
```

```
self._history_baseline_rgba)

def _update_mode_buttons(self):
    if self._interaction_mode == "preview":
        self.preview_mode_btn.setText("Preview Mode ✓")
        self.edit_mode_btn.setText("Edit Mode")
    else:
        self.preview_mode_btn.setText("Preview Mode")
        self.edit_mode_btn.setText("Edit Mode ✓")

    if self._current_mode == "before":
        self.before_btn.setText("Before ✓")
        self.after_btn.setText("After")
    else:
        self.before_btn.setText("Before")
        self.after_btn.setText("After ✓")

    has_edit_target = self._editable_after_image_pil is
not None
    self.edit_mode_btn.setEnabled(has_edit_target)
    self.after_btn.setEnabled(has_edit_target)
    self.reset_edits_btn.setEnabled(has_edit_target
and self._edit_reset_baseline_rgba is not None)

    self.undo_btn.setEnabled(self._can_undo())
    self.redo_btn.setEnabled(bool(self._redo_stack))

def _get_current_image(self):
    if self._current_mode == "after":
        return self._editable_after_image_pil
    return self._before_image_pil

def _load_current_mode_pixmap(self):
    image = self._get_current_image()

    if image is None:
        self.image_label.clear()
        self.image_label.resize(0, 0)
        self.image_label.hide()
```

```
        self._current_pixmap = None
        self._update_mode_buttons()
        return

    buffer = BytesIO()
    image.save(buffer, format="PNG")

    pixmap = QPixmap()
    pixmap.loadFromData(buffer.getvalue())
    self._current_pixmap = pixmap

    self._render_current_pixmap()

    def _render_current_pixmap(self):
        if self._current_pixmap is None:
            return

        viewport_size = self.scroll_area.viewport().size()
        if not viewport_size.isValid():
            viewport_size = QSize(400, 400)

        available_width = max(1, viewport_size.width() -
20)
        available_height = max(1, viewport_size.height() -
20)

        pixmap_width = max(1,
self._current_pixmap.width())
        pixmap_height = max(1,
self._current_pixmap.height())

        fit_scale = min(
            available_width / pixmap_width,
            available_height / pixmap_height,
        )

        self._base_fit_scale = min(fit_scale, 1.0)
        final_scale = max(0.01, self._base_fit_scale *
self._zoom_factor)
```



```
scaled_width = max(1, int(pixmap_width *
final_scale))
scaled_height = max(1, int(pixmap_height *
final_scale))

scaled_pixmap = self._current_pixmap.scaled(
    scaled_width,
    scaled_height,
    Qt.AspectRatioMode.KeepAspectRatio,
    Qt.TransformationMode.SmoothTransformation,
)

self.image_label.setPixmap(scaled_pixmap)
self.image_label.resize(scaled_pixmap.size())
self.image_label.show()
self._update_mode_buttons()

def handle_mouse_press(self, event):
    if event.button() != Qt.MouseButton.LeftButton:
        return

    if self._interaction_mode != "edit":
        return

    if self._editable_rgba is None:
        return

    pos = event.position().toPoint()
    image_pos = self._widget_pos_to_image_pos(pos)

    if image_pos is None:
        return

    x, y = image_pos

    self._mouse_down = True
    self._last_image_point = (x, y)
    self._drag_edit_active = False

    if self._tool_name in ("Erase", "Restore"):
```

```
self._undo_stack.append(self._editable_rgba.copy())
    self._redo_stack.clear()

    changed = self._apply_brush_tool(x, y)
    if changed:
        self._drag_edit_active = True
        self._finish_edit_update()
    else:
        self._undo_stack.pop()
        self._update_mode_buttons()
    return

self._undo_stack.append(self._editable_rgba.copy())
self._redo_stack.clear()

changed = False

if self._tool_name == "Magic Erase":
    changed = self._magic_erase(x, y)
elif self._tool_name == "Background Erase":
    changed = self._background_erase(x, y)

if not changed:
    self._undo_stack.pop()
    self._update_mode_buttons()
    return

self._finish_edit_update()

def handle_mouse_move(self, event):
    if not self._mouse_down:
        return

    if self._interaction_mode != "edit":
        return

    if self._editable_rgba is None:
        return
```

```
        if self._tool_name not in ("Erase", "Restore"):
            return

        pos = event.position().toPoint()
        image_pos = self._widget_pos_to_image_pos(pos)

        if image_pos is None:
            return

        x, y = image_pos
        current_point = (x, y)

        if self._last_image_point is None:
            self._last_image_point = current_point

        changed =
self._apply_brush_stroke(self._last_image_point,
current_point)

        if changed:
            self._drag_edit_active = True
            self._finish_edit_update()

        self._last_image_point = current_point

def handle_mouse_release(self, event):
    self._mouse_down = False
    self._last_image_point = None
    self._drag_edit_active = False

def _widget_pos_to_image_pos(self, pos: QPoint):
    pixmap = self.image_label.pixmap()

    if pixmap is None or self._editable_rgba is None:
        return None

    w = pixmap.width()
    h = pixmap.height()
```

```
        if w == 0 or h == 0:
            return None

        x = pos.x()
        y = pos.y()

        img_h, img_w = self._editable_rgba.shape[:2]

        img_x = int(x * img_w / w)
        img_y = int(y * img_h / h)

        if img_x < 0 or img_y < 0 or img_x >= img_w or
img_y >= img_h:
            return None

        return img_x, img_y

def _apply_brush_tool(self, x: int, y: int) -> bool:
    if self._tool_name == "Erase":
        return self._brush_erase(x, y)
    if self._tool_name == "Restore":
        return self._brush_restore(x, y)
    return False

def _apply_brush_stroke(self, start_point: tuple[int,
int], end_point: tuple[int, int]) -> bool:
    x1, y1 = start_point
    x2, y2 = end_point

    dx = x2 - x1
    dy = y2 - y1
    steps = max(abs(dx), abs(dy), 1)

    changed = False

    for i in range(steps + 1):
        px = int(round(x1 + (dx * i / steps)))
        py = int(round(y1 + (dy * i / steps)))
        if self._apply_brush_tool(px, py):
            changed = True
```

```
        return changed

    def _brush_mask(self, center_x: int, center_y: int):
        if self._editable_rgba is None:
            return None

        img_h, img_w = self._editable_rgba.shape[:2]
        radius = max(1, int(self._brush_size / 2))

        y_grid, x_grid = np.ogrid[:img_h, :img_w]
        distance_sq = (x_grid - center_x) ** 2 + (y_grid -
center_y) ** 2
        return distance_sq <= radius ** 2

    def _brush_erase(self, x: int, y: int) -> bool:
        if self._editable_rgba is None:
            return False

        mask = self._brush_mask(x, y)
        if mask is None:
            return False

        before_alpha = self._editable_rgba[:, :, 3].copy()
        self._editable_rgba[mask, 3] = 0
        return not np.array_equal(before_alpha,
self._editable_rgba[:, :, 3])

    def _brush_restore(self, x: int, y: int) -> bool:
        if self._editable_rgba is None or
self._restore_source_rgba is None:
            return False

        mask = self._brush_mask(x, y)
        if mask is None:
            return False

        before_rgba = self._editable_rgba.copy()
        self._editable_rgba[mask] =
self._restore_source_rgba[mask]
```

```
        return not np.array_equal(before_rgba,
self._editable_rgba)

def _magic_erase(self, x: int, y: int) -> bool:
    before_alpha = self._editable_rgba[:, :, 3].copy()

    rgb = self._editable_rgba[:, :, :3].astype(np.int16)
    alpha = self._editable_rgba[:, :,
3].astype(np.float32)

    seed_color = rgb[y, x]

    diff = np.sqrt(np.sum((rgb - seed_color) ** 2,
axis=2))
    mask = diff <= self._tolerance

    if self._magic_mode == "Connected Region":
        height, width = mask.shape
        visited = np.zeros_like(mask, dtype=bool)
        stack = [(x, y)]

        while stack:
            px, py = stack.pop()

            if px < 0 or py < 0 or px >= width or py >=
height:
                continue

            if visited[py, px]:
                continue

            visited[py, px] = True

            if not mask[py, px]:
                continue

            alpha[py, px] = 0

            stack.append((px + 1, py))
            stack.append((px - 1, py))
```

```
        stack.append((px, py + 1))
        stack.append((px, py - 1))
    else:
        alpha[mask] = 0

    self._editable_rgba[:, :, 3] = alpha.astype(np.uint8)
    return not np.array_equal(before_alpha,
self._editable_rgba[:, :, 3])

def _background_erase(self, x: int, y: int) -> bool:
    before_alpha = self._editable_rgba[:, :, 3].copy()

    rgb = self._editable_rgba[:, :, :3].astype(np.int16)
    alpha = self._editable_rgba[:, :, 3].copy()

    seed_color = rgb[y, x]

    diff = np.sqrt(np.sum((rgb - seed_color) ** 2,
axis=2))
    mask = diff <= self._tolerance

    alpha[mask] = 0

    self._editable_rgba[:, :, 3] = alpha
    return not np.array_equal(before_alpha,
self._editable_rgba[:, :, 3])

def _sync_pil_from_rgba(self):
    self._editable_after_image_pil =
Image.fromarray(self._editable_rgba, "RGBA")

def _save_edited_preview(self):
    if self._editable_after_image_pil is None or
self._editable_save_path is None:
        return

self._editable_after_image_pil.save(self._editable_save
_path, format="PNG")
```

```
self.image_edited.emit(str(self._editable_save_path))
```

```
def _finish_edit_update(self):  
    self._sync_pil_from_rgba()  
    self._save_edited_preview()  
    self._current_mode = "after"  
    self._load_current_mode_pixmap()
```

```
def reset_edits(self):  
    if self._edit_reset_baseline_rgba is None:  
        return
```

```
        self._editable_rgba =  
self._edit_reset_baseline_rgba.copy()  
        self._undo_stack.clear()  
        self._redo_stack.clear()
```

```
        self._sync_pil_from_rgba()  
        self._save_edited_preview()  
        self._current_mode = "after"  
        self._interaction_mode = "edit"  
        self._load_current_mode_pixmap()  
        self.edits_reset.emit()
```

```
def undo(self):  
    if self._editable_rgba is None:  
        return
```

```
        if self._undo_stack:
```

```
            self._redo_stack.append(self._editable_rgba.copy())  
            self._editable_rgba = self._undo_stack.pop()  
            elif self._history_baseline_rgba is not None and  
not np.array_equal(self._editable_rgba,  
self._history_baseline_rgba):
```

```
            self._redo_stack.append(self._editable_rgba.copy())  
            self._editable_rgba =  
self._history_baseline_rgba.copy()  
            else:
```



```
        return

        self._sync_pil_from_rgba()
        self._save_edited_preview()
        self._current_mode = "after"
        self._interaction_mode = "edit"
        self._load_current_mode_pixmap()

    def redo(self):
        if not self._redo_stack or self._editable_rgba is
None:
            return

        self._undo_stack.append(self._editable_rgba.copy())
        self._editable_rgba = self._redo_stack.pop()

        self._sync_pil_from_rgba()
        self._save_edited_preview()
        self._current_mode = "after"
        self._interaction_mode = "edit"
        self._load_current_mode_pixmap()

settings panel:

from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt
```

```
class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality but can be heavier/slower.",
            "isnet-anime": "Best suited for anime / illustrated characters.",
            "birefnet-general-lite": "Lighter BiRefNet model for general images.",
            "birefnet-general": "General BiRefNet model, potentially better quality but heavier.",
            "birefnet-portrait": "BiRefNet model intended for portraits / people.",
            "bria-rmbg": "BRIA background removal model. May vary in speed/stability depending on setup.",
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
```

```
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI
Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be
removed.\n"
            "AI Removal is the working mode.\n"
            "Color Removal is still unfinished."
        )
        layout.addWidget(self.removal_mode_label)
        layout.addWidget(self.removal_mode_combo)

        self.model_label = QLabel("Model")
        self.model_combo = QComboBox()
        self.model_combo.addItem([
            "u2net",
            "u2netp",
            "u2net_human_seg",
            "isnet-general-use",
            "isnet-anime",
            "birefnet-general-lite",
            "birefnet-general",
            "birefnet-portrait",
            "bria-rmbg",
        ])
        self.model_combo.setCurrentText("u2netp")
        layout.addWidget(self.model_label)
        layout.addWidget(self.model_combo)

        self._apply_model_item_tooltips()

        self.model_hint_label = QLabel("")
        self.model_hint_label.setWordWrap(True)
        self.model_hint_label.setStyleSheet("color:
#cfcfcf; font-size: 11px;")
        layout.addWidget(self.model_hint_label)

        self.output_format_label = QLabel("Output
Format")
        self.output_format_combo = QComboBox()
```

```
        self.output_format_combo.addItem(["PNG",
        "JPG", "WEBP"])
        self.output_format_combo.setCurrentText("PNG")
        self.output_format_combo.setToolTip("Choose the
        final exported file format.")
        layout.addWidget(self.output_format_label)
        layout.addWidget(self.output_format_combo)

        self.background_mode_label =
        QLabel("Background Mode")
        self.background_mode_combo = QComboBox()

        self.background_mode_combo.addItem(["Transparent"
        , "Solid Color"])

        self.background_mode_combo.setCurrentText("Transpa
        rent")
        self.background_mode_combo.setToolTip(
            "Transparent keeps the cutout background
            transparent.\n"
            "Solid Color fills the background with a chosen
            color."
        )
        layout.addWidget(self.background_mode_label)
        layout.addWidget(self.background_mode_combo)

        self.background_color_label = QLabel("Solid
        Background Color")
        self.background_color_combo = QComboBox()
        self.background_color_combo.addItem(["White",
        "Black"])
        self.background_color_combo.setToolTip("Used
        only when Background Mode is set to Solid Color.")
        layout.addWidget(self.background_color_label)
        layout.addWidget(self.background_color_combo)

        self.output_dir_label = QLabel("Output Directory")
        self.output_dir_edit = QLineEdit()
        self.output_dir_edit.setPlaceholderText("Choose a
        folder for exported files")
```

```
        self.output_dir_btn = QPushButton("Choose  
Output Folder")  
        self.open_output_btn = QPushButton("Open  
Output Folder")  
  
        output_btn_row = QHBoxLayout()  
        output_btn_row.addWidget(self.output_dir_btn)  
        output_btn_row.addWidget(self.open_output_btn)  
  
        layout.addWidget(self.output_dir_label)  
        layout.addWidget(self.output_dir_edit)  
        layout.addLayout(output_btn_row)  
  
        self.naming_label = QLabel("Filename Suffix")  
        self.naming_edit = QLineEdit("_nobg")  
        self.naming_edit.setToolTip(  
            "This text is added to the exported filename.\n"  
            "The selected model name is also included  
automatically."  
        )  
        layout.addWidget(self.naming_label)  
        layout.addWidget(self.naming_edit)  
  
        self.tool_section_label = QLabel("Editing Tools")  
  
        self.tool_section_label.setObjectName("panelTitle")  
        layout.addWidget(self.tool_section_label)  
  
        self.tool_label = QLabel("Tool")  
        self.tool_combo = QComboBox()  
        self.tool_combo.addItem(["Erase", "Restore",  
"Magic Erase", "Background Erase"])  
        self.tool_combo.setToolTip(  
            "Erase: remove parts of the working image.\n"  
            "Restore: restore from the original image.\n"  
            "Magic Erase / Background Erase are still  
limited."  
        )  
        layout.addWidget(self.tool_label)  
        layout.addWidget(self.tool_combo)
```

```
        self.apply_mode_label = QLabel("Apply Mode")
        self.apply_mode_combo = QComboBox()
        self.apply_mode_combo.addItem("Brush",
"Click"])
        self.apply_mode_combo.setCurrentText("Brush")
        layout.addWidget(self.apply_mode_label)
        layout.addWidget(self.apply_mode_combo)

        self.brush_size_label = QLabel("Brush Size")
        self.brush_size_slider =
QSlider(Qt.Orientation.Horizontal)
        self.brush_size_slider.setRange(1, 200)
        self.brush_size_slider.setValue(40)
        self.brush_size_slider.setToolTip("Brush size for
Erase and Restore.")
        layout.addWidget(self.brush_size_label)
        layout.addWidget(self.brush_size_slider)

        self.tolerance_label = QLabel("Tolerance")
        self.tolerance_slider =
QSlider(Qt.Orientation.Horizontal)
        self.tolerance_slider.setRange(0, 255)
        self.tolerance_slider.setValue(35)
        self.tolerance_slider.setToolTip("Tolerance for
click-based tools.")
        layout.addWidget(self.tolerance_label)
        layout.addWidget(self.tolerance_slider)

        self.phase_note = QLabel(
            "Only the main erase/restore workflow is fully
ready right now. "
            "Advanced controls are hidden until they are
implemented."
        )
        self.phase_note.setWordWrap(True)
        layout.addWidget(self.phase_note)

        self.softness_label = QLabel("Softness")
        self.softness_slider =
```

```
QSlider(Qt.Orientation.Horizontal)
    self.softness_slider.setRange(0, 100)
    self.softness_slider.setValue(35)
    layout.addWidget(self.softness_label)
    layout.addWidget(self.softness_slider)

    self.opacity_label = QLabel("Opacity")
    self.opacity_slider =
QSlider(Qt.Orientation.Horizontal)
    self.opacity_slider.setRange(1, 100)
    self.opacity_slider.setValue(100)
    layout.addWidget(self.opacity_label)
    layout.addWidget(self.opacity_slider)

    self.spacing_label = QLabel("Spacing")
    self.spacing_slider =
QSlider(Qt.Orientation.Horizontal)
    self.spacing_slider.setRange(1, 100)
    self.spacing_slider.setValue(20)
    layout.addWidget(self.spacing_label)
    layout.addWidget(self.spacing_slider)

    self.threshold_label = QLabel("Color Threshold")
    self.threshold_slider =
QSlider(Qt.Orientation.Horizontal)
    self.threshold_slider.setRange(0, 255)
    self.threshold_slider.setValue(30)
    self.pick_color_btn = QPushButton("Pick Color")

    layout.addWidget(self.threshold_label)
    layout.addWidget(self.threshold_slider)
    layout.addWidget(self.pick_color_btn)

    self.watch_section_label = QLabel("Watch Folder")

    self.watch_section_label.setObjectName("panelTitle")
    layout.addWidget(self.watch_section_label)

    self.watch_folder_path_label = QLabel("Watch
Folder Path")
```

```
        self.watch_folder_edit = QLineEdit()

        self.watch_folder_edit.setPlaceholderText("Choose a
        folder to monitor")
        self.watch_folder_btn = QPushButton("Choose
        Watch Folder")
        self.watch_include_subfolders_check =
        QCheckBox("Include subfolders in watch folder")
        self.watch_enable_check = QCheckBox("Enable
        watch folder automation")
        self.watch_status_label = QLabel("Watch status:
        Off")

        layout.addWidget(self.watch_folder_path_label)
        layout.addWidget(self.watch_folder_edit)
        layout.addWidget(self.watch_folder_btn)

        layout.addWidget(self.watch_include_subfolders_check
        )
        layout.addWidget(self.watch_enable_check)
        layout.addWidget(self.watch_status_label)

        self.action_section_label = QLabel("Process/
        Export")

        self.action_section_label.setObjectName("panelTitle")
        layout.addWidget(self.action_section_label)

        action_row = QHBoxLayout()

        self.action_combo = QComboBox()
        self.action_combo.addItem(["Process", "Export"])
        self.action_combo.setToolTip(
            "Process = preview/background removal only.\n"
            "Export = final full-resolution output."
        )

        self.target_combo = QComboBox()
        self.target_combo.addItem(["Selected", "All"])
        self.target_combo.setToolTip("Selected means the
```



```
items with checkmarks.")
```

```
        self.run_action_btn = QPushButton("Run")
        self.run_action_btn.setToolTip("Run the selected
        Action on the selected Target.")
```

```
        action_row.addWidget(self.action_combo, 1)
        action_row.addWidget(self.target_combo, 1)
        action_row.addWidget(self.run_action_btn)
```

```
        layout.addLayout(action_row)
```

```
        self.export_zip_check = QCheckBox("Export
        successful results as ZIP")
        self.export_zip_check.setToolTip("For batch
        export, also create a ZIP containing the exported files.")
        layout.addWidget(self.export_zip_check)
```

```
        self.overwrite_exports_check =
        QCheckBox("Overwrite existing exports")
        self.overwrite_exports_check.setToolTip("Reuse
        the same output filename instead of creating numbered
        duplicates.")
        layout.addWidget(self.overwrite_exports_check)
```

```
        self.skip_exported_check = QCheckBox("Skip
        already exported items")
        self.skip_exported_check.setToolTip("During batch
        export, skip items already marked as exported.")
        layout.addWidget(self.skip_exported_check)
```

```
        self.open_output_after_export_check =
        QCheckBox("Open output folder when export finishes")
```

```
        self.open_output_after_export_check.setToolTip("Open
        the export folder automatically after a successful export
        batch.")
```

```
        layout.addWidget(self.open_output_after_export_chec
        k)
```

```
        self.skip_processed_check = QCheckBox("Skip  
already processed items")  
        self.skip_processed_check.setToolTip("During  
batch process, skip items that already have a processed  
preview.")  
        layout.addWidget(self.skip_processed_check)  
  
        self.cancel_btn = QPushButton("Cancel")  
        self.cancel_btn.hide()  
        self.cancel_btn.setToolTip("Stop after the current  
item finishes.")  
        layout.addWidget(self.cancel_btn)  
  
        layout.addStretch()
```

```
self.output_dir_btn.clicked.connect(self.choose_output  
_dir)
```

```
self.watch_folder_btn.clicked.connect(self.choose_watc  
h_folder)
```

```
self.removal_mode_combo.currentTextChanged.connect  
(self.update_mode_visibility)
```

```
self.background_mode_combo.currentTextChanged.con  
nect(self.update_background_visibility)
```

```
self.tool_combo.currentTextChanged.connect(self.upda  
te_tool_visibility)
```

```
self.model_combo.currentTextChanged.connect(self.up  
date_model_hint)
```

```
self.update_model_hint()  
self.update_mode_visibility()  
self.update_background_visibility()  
self.update_tool_visibility()
```

```
def _apply_model_item_tooltips(self):
    for i in range(self.model_combo.count()):
        model_name = self.model_combo.itemText(i)
        description =
self.model_descriptions.get(model_name, "")
        self.model_combo.setItemData(i, description,
Qt.ItemDataRole.ToolTipRole)

def choose_output_dir(self):
    folder = QFileDialog.getExistingDirectory(self,
"Choose Output Folder")
    if folder:
        self.output_dir_edit.setText(folder)

def choose_watch_folder(self):
    folder = QFileDialog.getExistingDirectory(self,
"Choose Watch Folder")
    if folder:
        self.watch_folder_edit.setText(folder)

def update_model_hint(self):
    model_name = self.model_combo.currentText()
    description =
self.model_descriptions.get(model_name, "")
    self.model_combo.setToolTip(description)
    self.model_hint_label.setText(description)

def update_mode_visibility(self):
    is_color_mode =
self.removal_mode_combo.currentText() == "Color
Removal"

    self.threshold_label.setVisible(is_color_mode)
    self.threshold_slider.setVisible(is_color_mode)
    self.pick_color_btn.setVisible(is_color_mode)

    self.model_label.setEnabled(not is_color_mode)
    self.model_combo.setEnabled(not is_color_mode)
    self.model_hint_label.setEnabled(not
is_color_mode)
```

```
def update_background_visibility(self):
    is_solid =
self.background_mode_combo.currentText() == "Solid
Color"
    self.background_color_label.setVisible(is_solid)
    self.background_color_combo.setVisible(is_solid)

def update_tool_visibility(self):
    tool_name = self.tool_combo.currentText()
    is_click_tool = tool_name in ("Magic Erase",
"Background Erase")

    self.tolerance_label.setVisible(is_click_tool)
    self.tolerance_slider.setVisible(is_click_tool)

    self.brush_size_label.setVisible(not is_click_tool)
    self.brush_size_slider.setVisible(not is_click_tool)

    self.apply_mode_label.setVisible(True)
    self.apply_mode_combo.setVisible(True)

    self.phase_note.setVisible(True)

    self.softness_label.setVisible(False)
    self.softness_slider.setVisible(False)
    self.opacity_label.setVisible(False)
    self.opacity_slider.setVisible(False)
    self.spacing_label.setVisible(False)
    self.spacing_slider.setVisible(False)

    if is_click_tool:
        self.apply_mode_combo.setCurrentText("Click")
        self.apply_mode_combo.setEnabled(False)
    else:

self.apply_mode_combo.setCurrentText("Brush")
    self.apply_mode_combo.setEnabled(False)

    self.watch_section_label.setVisible(False)
```

```
self.watch_folder_path_label.setVisible(False)
self.watch_folder_edit.setVisible(False)
self.watch_folder_btn.setVisible(False)

self.watch_include_subfolders_check.setVisible(False)
self.watch_enable_check.setVisible(False)
self.watch_status_label.setVisible(False)
```

I have enough to wire the advanced brush controls into the existing tools without changing the overall workflow. This step will make softness, opacity, and spacing actually affect Erase/Restore, and make the settings panel show those controls when they matter.

Thought for 20s >

Changed files:

- `ui/preview_canvas.py`
- `ui/settings_panel.py`

This step implements:

- **Softness** for Erase/Restore
- **Opacity** for Erase/Restore
- **Spacing** for drag strokes
- better visibility for the advanced brush controls in the settings panel

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)
```


Full updated file: ui/settings_panel.py

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


What changed

Preview canvas

- **Softness** now feathers the brush edge instead of using a hard circle only
- **Opacity** now controls brush strength
 - erase can partially reduce alpha
 - restore can partially blend back toward the original
- **Spacing** now controls how close brush stamps are during drag strokes

Settings panel

- advanced brush sliders are now shown for:
 - Erase
 - Restore
- tolerance is shown for:
 - Magic Erase
 - Background Erase
- tooltips were updated to explain what the advanced sliders do

What to test

Softness

- choose Erase
- set low softness
- make a stroke
- confirm the edge is hard/sharp
- set high softness
- make another stroke
- confirm the edge is more feathered

Opacity

- choose Erase
- set opacity to 100
- make one stroke
- confirm it erases strongly
- set opacity much lower, like 20
- make one stroke
- confirm it only partially erases

Restore with opacity

- erase part of the image
- switch to Restore
- set opacity low

- brush over erased area
- confirm it restores gradually
- set opacity high
- confirm it restores much more strongly

Spacing

- choose Erase
- set spacing low
- drag a long stroke
- confirm it looks dense/smooth
- set spacing high
- drag again
- confirm the stroke uses more separated stamps

Existing behavior still works

- undo/redo
- reset edits
- before/after view buttons
- process then edit
- edit before process
- export after edits

After this, the best next step is to improve the click tools:

- better Magic Erase behavior
- better Background Erase behavior
- clearer user control over connected vs broader selection



ok now make all options able to choose between "click"
and "brush"

Thought for 20s >

Changed files:

- `ui/preview_canvas.py`
- `ui/settings_panel.py`

This makes **all tools** able to use either:

- Click
- Brush

New behavior

Erase / Restore

- Click = one single stamp per click
- Brush = drag stroke

Magic Erase / Background Erase

- Click = one click-based action
- Brush = repeats the click-based action along the drag path

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)
```


Full updated file: ui/settings_panel.py

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


What to test

Apply Mode on every tool

For each of these tools:

- Erase
- Restore
- Magic Erase
- Background Erase

confirm you can choose both:

- Brush
- Click

Erase / Restore

- set Erase to Click
- click once on image
- confirm it applies one stamp only
- set Erase to Brush
- drag across image
- confirm it paints along the stroke
- repeat the same for Restore

Magic Erase / Background Erase

- set Magic Erase to Click
- click once
- confirm one click action happens
- set Magic Erase to Brush
- drag across image
- confirm it repeats along the drag path
- repeat the same for Background Erase

Settings panel visibility

- with Erase or Restore + Brush , confirm brush controls are visible
- with Erase or Restore + Click , confirm brush controls hide
- with Magic Erase or Background Erase , confirm tolerance remains visible

No regressions

- undo / redo
- reset edits
- before / after
- process then edit
- export after edits

The next strong step is polishing the click tools themselves so Magic Erase and Background Erase feel more predictable and useful.



ok lets go back to the way click/brush settings showed before, unless its possible to use magic eraser and background erase as a brush

Thought for 7s

Yes, it is possible.

Right now:

- **Magic Erase + Brush** = repeats the magic-erase action along the drag path
- **Background Erase + Brush** = repeats the background-erase action along the drag path

So based on what you said, there is **no need to revert** just because those tools need brush support. They already can use brush.

The one important caveat is this:

- brush mode for those two tools is more aggressive and less predictable than normal Erase/Restore
- each stamp is doing a tolerance-based erase operation
- so it can feel heavier or jumpier than a standard brush

So the best path is:

- **keep the current click/brush capability for all tools**
- then **polish Magic Erase and Background Erase brush behavior**

The next improvement should be:

- make Magic Erase brush feel more controlled
- make Background Erase brush feel smoother and less harsh
- tune how often those actions stamp while dragging

If you want, I'll do that next and keep the current UI.



ok

Thought for 14s >

Changed file:

- `ui/preview_canvas.py`

This keeps **Click/Brush available for all tools**, but makes **Magic Erase** and **Background Erase** behave much better in **Brush** mode.

What changed

Magic Erase

- **Click mode** stays broad, like before
- **Brush mode** is now limited to the brush area, so it is much more controlled

Background Erase

- **Click mode** stays broad, like before
- **Brush mode** now only affects pixels inside the brush area that match the tolerance

This should make those two tools feel much less wild when dragging.

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)
```


What to test

Magic Erase brush

- choose Magic Erase
- set Apply Mode to Brush
- drag over a small area
- confirm it mainly affects the brush region instead of wiping broad unrelated areas

Magic Erase click

- keep Magic Erase
- set Apply Mode to Click
- click once
- confirm it still behaves like the broader click action

Background Erase brush

- choose Background Erase
- set Apply Mode to Brush
- drag over a small area
- confirm it only erases matching pixels inside the brush area

Background Erase click

- set Apply Mode to Click
- click once
- confirm it still behaves like the broader click version

No regression

- Erase / Restore click
- Erase / Restore brush
- undo / redo
- reset edits

The next strong polish step is adding a **magic mode selector** in the UI so you can choose between:

- Connected Region
- Global Match

That would make Magic Erase much more understandable and controllable.



ok, i want to speed this up so can we go over the next several fine tuning steps for the editor and and update the codes with several of the u

I'll bundle several editor polish steps into one pass so you do not have to do them one at a time. The practical set is: add a Magic Erase mode selector, make click/brush controls show more sensibly, cap undo history, and make magic/background brush strokes more controllable.

Thought for 25s >

Yes. The fastest way is to bundle several editor polish steps into one update.

Bundle for this pass

I'm grouping these together:

1. Magic Erase mode selector in the UI

- Connected Region
- Global Match

2. Better control visibility

- only show the controls that matter for the current tool/mode

3. Safer undo history

- cap undo stack size so editing stays responsive

4. Softer, more controllable Magic/Background brush mode

- brush mode uses the brush area and brush opacity instead of harsh full wipes every stamp

5. Cleaner tool descriptions

- clearer hover text for editor controls

Changed files:

- `ui/preview_canvas.py`
- `ui/settings_panel.py`
- `ui/main_window.py`

Full updated file: `ui/preview_canvas.py`

Python

Run


```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
        super().__init__()

        self.setObjectName("previewCanvas")
        self setFrameShape(QFrame.StyledPanel)
```


Full updated file: ui/settings_panel.py

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)

        title = QLabel("Settings Panel")
        title.setObjectName("panelTitle")
        layout.addWidget(title)

        self.removal_mode_label = QLabel("Removal Mode")
        self.removal_mode_combo = QComboBox()
        self.removal_mode_combo.addItem("AI Removal", "Color Removal")
        self.removal_mode_combo.setToolTip(
            "Choose how the background should be removed.\n"
```


ui/main_window.py changes

In `_connect_signals()`

Add this line after the existing tool/apply/tolerance connections.

Before

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_tool)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_apply)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_brush_size)
self.settings_panel.softness_slider.valueChanged.connect(self._sync_softness)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_opacity)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_spacing)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_tolerance)
```

After

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_tool)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_apply)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_brush_size)
self.settings_panel.softness_slider.valueChanged.connect(self._sync_softness)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_opacity)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_spacing)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_tolerance)
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._sync_magic_mode)
```

In `_sync_preview_tool_settings()`

Replace this function:

Before

Python

Run

```
def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(
        tool_name=self.settings_panel.tool_combo.currentText(),
        apply_mode=self.settings_panel.apply_mode_combo.currentText(),
        brush_size=self.settings_panel.brush_size_slider.value(),
        softness=self.settings_panel.softness_slider.value(),
        opacity=self.settings_panel.opacity_slider.value(),
        spacing=self.settings_panel.spacing_slider.value(),
        tolerance=self.settings_panel.tolerance_slider.value(),
    )
```

After

Python

Run

```
def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(
        tool_name=self.settings_panel.tool_combo.currentText(),
        apply_mode=self.settings_panel.apply_mode_combo.currentText(),
        brush_size=self.settings_panel.brush_size_slider.value(),
        softness=self.settings_panel.softness_slider.value(),
        opacity=self.settings_panel.opacity_slider.value(),
        spacing=self.settings_panel.spacing_slider.value(),
        tolerance=self.settings_panel.tolerance_slider.value(),
    )
    self.preview_canvas.set_magic_mode(
        self.settings_panel.magic_mode_combo.currentText()
    )
```

What to test

Magic Mode UI

- choose Magic Erase
- confirm a Magic Mode dropdown appears
- switch between:
 - Connected Region
 - Global Match

Magic Erase click

- set Apply Mode to Click
- try both magic modes
- confirm Connected Region is more local
- confirm Global Match is broader

Magic Erase brush

- set Apply Mode to Brush
- use both magic modes
- confirm brush strokes are more controlled than before
- confirm opacity affects strength

Background Erase brush

- set Background Erase to Brush
- drag over a small region
- confirm it fades matching pixels inside the brush area instead of wiping too aggressively

Undo responsiveness

- do many edits
- confirm undo still works
- confirm app stays responsive

The next strong bundle after this would be:

- add a small live brush preview/cursor overlay
- add an "edge protect" style option for click tools
- tune default values for each tool automatically when switching tools



so in connect signals, is this right

```
self.settings_panel.tool_combo.currentTextChanged.co
```

```
        connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.softness_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.opacity_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.spacing_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.softness_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.opacity_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.spacing_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
        self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

```
ect(self._sync_preview_tool_settings)
```

```
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
```

No. That block has the same connections duplicated.

Use this once only:

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.softness_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
```

What to remove

Delete this first repeated block:

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.softness_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_preview_tool_settings)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_preview_tool_settings)
```

Why

If you leave both copies in:

- `_sync_preview_tool_settings()` will run twice for those controls
- that can cause unnecessary refreshes and confusing behavior

So the correct version is:

- one set of connections
- plus the new `magic_mode_combo` line

If you want, paste your current `_sync_preview_tool_settings()` too and I'll verify that part.



ok whats the next several parts

Thought for a second

Next several parts should be these, in order:

1. Live brush preview

Add a visible brush cursor on the image so you can see:

- brush size
- where the stroke will land
- current tool area

This is one of the biggest usability upgrades.

2. Better tool-specific defaults

When switching tools, auto-set sensible defaults, for example:

- Erase : medium size, medium softness, full opacity
- Restore : medium size, softer edge
- Magic Erase : click mode, medium tolerance
- Background Erase : click mode, slightly lower tolerance

That makes the tools feel better immediately.

3. Magic/Background polish

Refine:

- tolerance sensitivity

- connected vs global behavior
- brush mode smoothness
- opacity behavior for click tools if needed

4. Edge-safe editing

Add protection to reduce ugly halos and accidental damage near subject edges.
This could be a simple first version, not a full AI edge detector.

5. Better undo batching

Right now undo is functional, but a later polish step is:

- one full drag stroke = one undo step
instead of many micro-updates feeling noisy

That is a very good quality improvement.

6. Reset options cleanup

Add clearer reset choices:

- Reset Current Image
- Reset Selected
- maybe later Reset Processed State separately

7. Editor UI polish

Clean up labels and visibility:

- show only relevant controls for current tool/mode
- improve tooltips
- possibly group controls into:
 - Brush controls
 - Click controls

8. Performance pass

Once the editor features are stable:

- reduce unnecessary preview saves
- reduce redraw overhead
- keep large images responsive

Best immediate next bundle

The strongest next bundle is:

- **live brush preview**
- **tool-specific defaults**
- **better undo batching per drag stroke**

That would make the editor feel much more polished without changing the export/process logic again.

After that

Then do:

- **Magic/Background polish**
- **edge-safe editing**
- **editor UI cleanup**

Suggested phase order

- **Phase 28** — live brush preview + tool defaults
- **Phase 29** — undo batching per stroke + editor responsiveness
- **Phase 30** — Magic Erase / Background Erase polish
- **Phase 31** — edge-safe editing and final editor cleanup

The best next coding step is **Phase 28: live brush preview + tool defaults**.



ok

I'm packaging the next editor pass into one practical upgrade: visible brush/click cursor preview, better per-tool defaults, and cleaner control behavior so switching tools feels less fiddly.

Thought for 29s >

Changed files:

- `ui/preview_canvas.py`
- `ui/settings_panel.py`

No `main_window.py` change is needed for this pass.

This pass adds:

- a **live tool preview** on the image
- better **tool-specific defaults**
- cleaner **tool control behavior** when switching tools

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap, QPainter, QPen, QColor
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

    def leaveEvent(self, event):
        self.owner.handle_mouse_leave(event)
        super().leaveEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
```


Full updated file: `ui/settings_panel.py`

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially better",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed."
        }

        self.tool_defaults = {
            "Erase": {
                "apply_mode": "Brush",
                "brush_size": 42,
                "softness": 35,
                "opacity": 100,
                "spacing": 18,
                "tolerance": 35,
            },
            "Restore": {
                "apply_mode": "Brush",
                "brush_size": 42,
                "softness": 45,
```


ui/main_window.py changes

In `_connect_signals()`

Use this editor-signal block:

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_tool)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_apply_mode)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_brush_size)
self.settings_panel.softness_slider.valueChanged.connect(self._sync_softness)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_opacity)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_spacing)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_tolerance)
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._sync_magic_mode)
```

In `_sync_preview_tool_settings()`

Use this exact function:

Python

Run

```
def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(
        tool_name=self.settings_panel.tool_combo.currentText(),
        apply_mode=self.settings_panel.apply_mode_combo.currentText(),
        brush_size=self.settings_panel.brush_size_slider.value(),
        softness=self.settings_panel.softness_slider.value(),
        opacity=self.settings_panel.opacity_slider.value(),
        spacing=self.settings_panel.spacing_slider.value(),
        tolerance=self.settings_panel.tolerance_slider.value(),
    )
    self.preview_canvas.set_magic_mode(
        self.settings_panel.magic_mode_combo.currentText()
    )
```

What to test

Live preview

- switch to Edit Mode
- move the mouse over the image
- confirm a visible tool circle follows the cursor
- switch tools and confirm the preview color changes

Tool defaults

- switch to Erase
- confirm it defaults to brush-oriented values
- switch to Restore
- confirm values change to its defaults
- switch to Magic Erase
- confirm it defaults to Click
- switch to Background Erase
- confirm it defaults to Click

Magic Mode

- choose Magic Erase
- confirm Magic Mode appears

- test both:
 - Connected Region
 - Global Match

Control visibility

- Erase / Restore + Brush
 - brush size, softness, opacity, spacing visible
- Erase / Restore + Click
 - brush controls hidden
- Magic Erase / Background Erase + Brush
 - brush size, opacity, spacing visible
 - tolerance visible
- Magic Erase + any mode
 - magic mode visible

Undo stability

- make many edits
- confirm undo/redo still works
- confirm the app stays responsive

After this, the next several editor steps should be:

- edge-safe click tools
- better per-tool tolerance tuning
- one full drag stroke counting as one cleaner undo unit
- optional subject-edge protection mode



all good, now what

Thought for a couple of seconds

Next best step is **undo batching per full drag stroke**.

Right now the editor is much better, but this is the polish that will make it feel more professional:

- one continuous drag stroke = one undo step
- instead of many tiny internal updates feeling noisy

After that, the next strongest steps are:

Recommended order

1. Undo batching per stroke

This improves:

- brush editing
- Magic Erase brush
- Background Erase brush

2. Edge-safe click tools

Add protection so click-based tools are less likely to damage subject edges or create halos.

3. Better tolerance tuning

Make:

- Magic Erase less jumpy
- Background Erase more predictable
- defaults feel better on common images

4. Save editor preferences

Persist things like:

- last tool
- last apply mode
- brush size
- softness
- opacity
- spacing
- tolerance

- magic mode

So the editor comes back the way you left it.

5. Final editor cleanup

Refine:

- labels
- tooltips
- control grouping
- default values

Best immediate move

I would do this next bundle:

- **undo batching per stroke**
- **save editor preferences**
- **tolerance/default tuning**

That gives a lot of polish in one pass without changing the overall app structure again.

If you want the strongest next coding step, do **undo batching per stroke** first.



ok give me the code for the next two steps

Thought for 17s >

Changed files:

- ui/preview_canvas.py
- ui/main_window.py

This pass does the next two steps:

1. **undo batching per full drag stroke**
2. **save editor preferences between launches**

What changed

1) Undo batching per stroke

Undo is now stored like this:

- one **click** = one undo step
- one full **drag stroke** = one undo step

That applies to:

- Erase
- Restore
- Magic Erase
- Background Erase

2) Save editor preferences

These now persist through QSettings :

- tool
- apply mode
- magic mode
- brush size
- softness
- opacity
- spacing
- tolerance

So when you reopen the app, the editor controls come back the way you left them.

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap, QPainter, QPen, QColor
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

    def leaveEvent(self, event):
        self.owner.handle_mouse_leave(event)
        super().leaveEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
```


ui/main_window.py **changes**

1) In _restore_persistent_settings()

Replace this:

Python

Run

```
def _restore_persistent_settings(self):
    output_dir = self.settings_store.value("output_dir", "", type=str)
    naming_suffix = self.settings_store.value("naming_suffix", "_nobg")

    if output_dir:
        self.settings_panel.output_dir_edit.setText(output_dir)
    if naming_suffix:
        self.settings_panel.naming_edit.setText(naming_suffix)
```

With this:

Python

Run

```
def _restore_persistent_settings(self):
    output_dir = self.settings_store.value("output_dir", "", type=str)
    naming_suffix = self.settings_store.value("naming_suffix", "_nobg")

    if output_dir:
        self.settings_panel.output_dir_edit.setText(output_dir)
    if naming_suffix:
        self.settings_panel.naming_edit.setText(naming_suffix)

    self._restore_editor_settings()
```

2) Add this new function directly below

_restore_persistent_settings()

Python

Run

```
def _restore_editor_settings(self):
    tool_name = self.settings_store.value("editor/tool_name", "Erase",
    apply_mode = self.settings_store.value("editor/apply_mode", "Brush
    magic_mode = self.settings_store.value("editor/magic_mode", "Conne

    brush_size = self.settings_store.value("editor/brush_size", 42, ty
    softness = self.settings_store.value("editor/softness", 35, type=i
    opacity = self.settings_store.value("editor/opacity", 100, type=in
    spacing = self.settings_store.value("editor/spacing", 18, type=int
    tolerance = self.settings_store.value("editor/tolerance", 35, type=

    controls = [
        self.settings_panel.tool_combo,
        self.settings_panel.apply_mode_combo,
        self.settings_panel.magic_mode_combo,
        self.settings_panel.brush_size_slider,
        self.settings_panel.softness_slider,
        self.settings_panel.opacity_slider,
        self.settings_panel.spacing_slider,
        self.settings_panel.tolerance_slider,
    ]

    for control in controls:
        control.blockSignals(True)

    if self.settings_panel.tool_combo.findText(tool_name) >= 0:
        self.settings_panel.tool_combo.setCurrentText(tool_name)

    if self.settings_panel.apply_mode_combo.findText(apply_mode) >= 0:
        self.settings_panel.apply_mode_combo.setCurrentText(apply_mode

    if self.settings_panel.magic_mode_combo.findText(magic_mode) >= 0:
        self.settings_panel.magic_mode_combo.setCurrentText(magic_mode

    self.settings_panel.brush_size_slider.setValue(max(1, min(200, bru
    self.settings_panel.softness_slider.setValue(max(0, min(100, softn
    self.settings_panel.opacity_slider.setValue(max(1, min(100, opacit
    self.settings_panel.spacing_slider.setValue(max(1, min(100, spacin
    self.settings_panel.tolerance_slider.setValue(max(0, min(255, tole

    for control in controls:
        control.blockSignals(False)

    self.settings_panel.update_tool_visibility()
```

3) Add this new function directly below

`_persist_naming_suffix()`

Python

Run

```
def _persist_editor_settings(self):
    self.settings_store.setValue("editor/tool_name", self.settings_panel.tool_name)
    self.settings_store.setValue("editor/apply_mode", self.settings_panel.apply_mode)
    self.settings_store.setValue("editor/magic_mode", self.settings_panel.magic_mode)
    self.settings_store.setValue("editor/brush_size", self.settings_panel.brush_size)
    self.settings_store.setValue("editor/softness", self.settings_panel.softness)
    self.settings_store.setValue("editor/opacity", self.settings_panel.opacity)
    self.settings_store.setValue("editor/spacing", self.settings_panel.spacing)
    self.settings_store.setValue("editor/tolerance", self.settings_panel.tolerance)
```

4) In `_connect_signals()`

Replace this editor signal block:

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync_tool_name)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync_apply_mode)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync_brush_size)
self.settings_panel.softness_slider.valueChanged.connect(self._sync_softness)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync_opacity)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync_spacing)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync_tolerance)
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._sync_magic_mode)
```

With this:

Python

Run

```
self.settings_panel.tool_combo.currentTextChanged.connect(self._sync)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._sync)
self.settings_panel.brush_size_slider.valueChanged.connect(self._sync)
self.settings_panel.softness_slider.valueChanged.connect(self._sync)
self.settings_panel.opacity_slider.valueChanged.connect(self._sync)
self.settings_panel.spacing_slider.valueChanged.connect(self._sync)
self.settings_panel.tolerance_slider.valueChanged.connect(self._sync)
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._sync)

self.settings_panel.tool_combo.currentTextChanged.connect(self._per)
self.settings_panel.apply_mode_combo.currentTextChanged.connect(self._per)
self.settings_panel.brush_size_slider.valueChanged.connect(self._per)
self.settings_panel.softness_slider.valueChanged.connect(self._per)
self.settings_panel.opacity_slider.valueChanged.connect(self._pers)
self.settings_panel.spacing_slider.valueChanged.connect(self._pers)
self.settings_panel.tolerance_slider.valueChanged.connect(self._pers)
self.settings_panel.magic_mode_combo.currentTextChanged.connect(self._pers)
```

What to test

Undo batching

- choose Erase
- drag one long stroke
- click Undo
- confirm the **whole stroke** undoes in one step
- repeat with:
 - Restore
 - Magic Erase in brush mode
 - Background Erase in brush mode

Click undo

- use a click-only edit
- confirm one click = one undo step

Preference persistence

Change these:

- tool

- apply mode
- magic mode
- brush size
- softness
- opacity
- spacing
- tolerance

Then close and reopen the app.

Confirm they restore correctly.

What's next after this

The strongest next bundle is:

- **edge-safe click tools**
- **tolerance tuning**
- **optional subject-edge protection mode**

That is the next real polish step for the editor.



ok ready

Thought for 12s >

Next bundle should be:

Phase 33 — safer click tools

Add protection so Magic Erase and Background Erase are less likely to damage subject edges.

That would include:

- **edge-safe click behavior**
 - reduce harsh removal near boundaries
- **better tolerance tuning**
 - make Magic Erase less jumpy
 - make Background Erase more predictable
- **subject-edge protection toggle**
 - optional checkbox for safer click erasing

After that

Then do:

- **tool-specific tolerance defaults**
- **editor preference polish**
- **performance cleanup for heavy edits**

Best immediate next coding step

The strongest next step is:

1. add an **Edge Protect** checkbox for click tools
2. refine Magic Erase and Background Erase so they respect that protection
3. tune defaults so they feel better out of the box

If your `ui/preview_canvas.py` and `ui/settings_panel.py` have not changed since the last update, I can use those versions directly for the next code pass.



ok

Thought for 6s

Changed files:

- `ui/preview_canvas.py`
- `ui/settings_panel.py`
- `ui/main_window.py`

This pass adds:

- **Edge Protect** checkbox for click tools
- safer Magic Erase
- safer Background Erase
- persistence for the new editor setting

The first version of **Edge Protect** works like this:

- it reduces erasing on pixels that look like likely edges
- it applies to click tools and their brush mode
- turning it off gives the old more aggressive behavior

Full updated file: `ui/preview_canvas.py`

Python

Run

```
from io import BytesIO
from pathlib import Path
import math

import numpy as np
from PIL import Image

from PySide6.QtCore import Qt, Signal, QPoint, QSize
from PySide6.QtGui import QPixmap, QPainter, QPen, QColor
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QSizePolicy,
    QScrollArea,
    QHBoxLayout,
    QPushButton,
    QWidget,
)

class _CanvasLabel(QLabel):
    def __init__(self, owner):
        super().__init__()
        self.owner = owner

    def mousePressEvent(self, event):
        self.owner.handle_mouse_press(event)
        super().mousePressEvent(event)

    def mouseMoveEvent(self, event):
        self.owner.handle_mouse_move(event)
        super().mouseMoveEvent(event)

    def mouseReleaseEvent(self, event):
        self.owner.handle_mouse_release(event)
        super().mouseReleaseEvent(event)

    def leaveEvent(self, event):
        self.owner.handle_mouse_leave(event)
        super().leaveEvent(event)

class PreviewCanvas(QFrame):
    image_edited = Signal(str)
    edits_reset = Signal()

    def __init__(self):
```


Full updated file: ui/settings_panel.py

Python

Run

```
from PySide6.QtWidgets import (
    QFrame,
    QVBoxLayout,
    QLabel,
    QComboBox,
    QPushButton,
    QLineEdit,
    QFileDialog,
    QCheckBox,
    QSlider,
    QHBoxLayout,
)
from PySide6.QtCore import Qt

class SettingsPanel(QFrame):
    def __init__(self):
        super().__init__()

        self.setObjectName("settingsPanel")
        self setFrameShape(QFrame.StyledPanel)
        self.setMinimumWidth(320)

        self.model_descriptions = {
            "u2net": "General-purpose model with good overall quality.",
            "u2netp": "Smaller and faster general-purpose model.",
            "u2net_human_seg": "Best suited for people / portraits.",
            "isnet-general-use": "General-use model, often higher quality |",
            "isnet-anime": "Best suited for anime / illustrated characters",
            "birefnet-general-lite": "Lighter BiRefNet model for general in",
            "birefnet-general": "General BiRefNet model, potentially bette",
            "birefnet-portrait": "BiRefNet model intended for portraits / |",
            "bria-rmbg": "BRIA background removal model. May vary in speed.",
        }

        self.tool_defaults = {
            "Erase": {
                "apply_mode": "Brush",
                "brush_size": 42,
                "softness": 35,
                "opacity": 100,
                "spacing": 18,
                "tolerance": 35,
            },
            "Restore": {
                "apply_mode": "Brush",
                "brush_size": 42,
                "softness": 45,
```


ui/main_window.py **changes**

1) In _connect_signals()

Replace this block:

Python

Run

```

self.settings_panel.tool_combo.currentTextChanged.connect(self._sy
self.settings_panel.apply_mode_combo.currentTextChanged.connect(se
self.settings_panel.brush_size_slider.valueChanged.connect(self._s
self.settings_panel.softness_slider.valueChanged.connect(self._syn
self.settings_panel.opacity_slider.valueChanged.connect(self._sync
self.settings_panel.spacing_slider.valueChanged.connect(self._sync
self.settings_panel.tolerance_slider.valueChanged.connect(self._sy
self.settings_panel.magic_mode_combo.currentTextChanged.connect(se

self.settings_panel.tool_combo.currentTextChanged.connect(self._pe
self.settings_panel.apply_mode_combo.currentTextChanged.connect(se
self.settings_panel.brush_size_slider.valueChanged.connect(self._p
self.settings_panel.softness_slider.valueChanged.connect(self._per
self.settings_panel.opacity_slider.valueChanged.connect(self._pers
self.settings_panel.spacing_slider.valueChanged.connect(self._pers
self.settings_panel.tolerance_slider.valueChanged.connect(self._pe
self.settings_panel.magic_mode_combo.currentTextChanged.connect(se

```

With this:

Python

Run

```

self.settings_panel.tool_combo.currentTextChanged.connect(self._sy
self.settings_panel.apply_mode_combo.currentTextChanged.connect(se
self.settings_panel.brush_size_slider.valueChanged.connect(self._s
self.settings_panel.softness_slider.valueChanged.connect(self._syn
self.settings_panel.opacity_slider.valueChanged.connect(self._sync
self.settings_panel.spacing_slider.valueChanged.connect(self._sync
self.settings_panel.tolerance_slider.valueChanged.connect(self._sy
self.settings_panel.magic_mode_combo.currentTextChanged.connect(se
self.settings_panel.edge_protect_check.toggled.connect(self._sync_

self.settings_panel.tool_combo.currentTextChanged.connect(self._pe
self.settings_panel.apply_mode_combo.currentTextChanged.connect(se
self.settings_panel.brush_size_slider.valueChanged.connect(self._p
self.settings_panel.softness_slider.valueChanged.connect(self._per
self.settings_panel.opacity_slider.valueChanged.connect(self._pers
self.settings_panel.spacing_slider.valueChanged.connect(self._pers
self.settings_panel.tolerance_slider.valueChanged.connect(self._pe
self.settings_panel.magic_mode_combo.currentTextChanged.connect(se
self.settings_panel.edge_protect_check.toggled.connect(self._persi

```

2) In _restore_editor_settings()

Replace this:

Python

Run


```
def _restore_editor_settings(self):
    tool_name = self.settings_store.value("editor/tool_name", "Erase",
    apply_mode = self.settings_store.value("editor/apply_mode", "Brush
    magic_mode = self.settings_store.value("editor/magic_mode", "Conne

    brush_size = self.settings_store.value("editor/brush_size", 42, ty
    softness = self.settings_store.value("editor/softness", 35, type=i
    opacity = self.settings_store.value("editor/opacity", 100, type=in
    spacing = self.settings_store.value("editor/spacing", 18, type=int
    tolerance = self.settings_store.value("editor/tolerance", 35, type=

    controls = [
        self.settings_panel.tool_combo,
        self.settings_panel.apply_mode_combo,
        self.settings_panel.magic_mode_combo,
        self.settings_panel.brush_size_slider,
        self.settings_panel.softness_slider,
        self.settings_panel.opacity_slider,
        self.settings_panel.spacing_slider,
        self.settings_panel.tolerance_slider,
    ]

    for control in controls:
        control.blockSignals(True)

    if self.settings_panel.tool_combo.findText(tool_name) >= 0:
        self.settings_panel.tool_combo.setCurrentText(tool_name)

    if self.settings_panel.apply_mode_combo.findText(apply_mode) >= 0:
        self.settings_panel.apply_mode_combo.setCurrentText(apply_mode

    if self.settings_panel.magic_mode_combo.findText(magic_mode) >= 0:
        self.settings_panel.magic_mode_combo.setCurrentText(magic_mode

    self.settings_panel.brush_size_slider.setValue(max(1, min(200, bru
    self.settings_panel.softness_slider.setValue(max(0, min(100, softn
    self.settings_panel.opacity_slider.setValue(max(1, min(100, opacit
    self.settings_panel.spacing_slider.setValue(max(1, min(100, spacin
    self.settings_panel.tolerance_slider.setValue(max(0, min(255, tole

    for control in controls:
        control.blockSignals(False)

    self.settings_panel.update_tool_visibility()
```

With this:

Python

Run

```
def _restore_editor_settings(self):
    tool_name = self.settings_store.value("editor/tool_name", "Erase",
    apply_mode = self.settings_store.value("editor/apply_mode", "Brush
    magic_mode = self.settings_store.value("editor/magic_mode", "Conne
    edge_protect = self.settings_store.value("editor/edge_protect", Tr

    brush_size = self.settings_store.value("editor/brush_size", 42, ty
    softness = self.settings_store.value("editor/softness", 35, type=i
    opacity = self.settings_store.value("editor/opacity", 100, type=in
    spacing = self.settings_store.value("editor/spacing", 18, type=int
    tolerance = self.settings_store.value("editor/tolerance", 35, type=

    controls = [
        self.settings_panel.tool_combo,
        self.settings_panel.apply_mode_combo,
        self.settings_panel.magic_mode_combo,
        self.settings_panel.edge_protect_check,
        self.settings_panel.brush_size_slider,
        self.settings_panel.softness_slider,
        self.settings_panel.opacity_slider,
        self.settings_panel.spacing_slider,
        self.settings_panel.tolerance_slider,
    ]

    for control in controls:
        control.blockSignals(True)

    if self.settings_panel.tool_combo.findText(tool_name) >= 0:
        self.settings_panel.tool_combo.setCurrentText(tool_name)

    if self.settings_panel.apply_mode_combo.findText(apply_mode) >= 0:
        self.settings_panel.apply_mode_combo.setCurrentText(apply_mode)

    if self.settings_panel.magic_mode_combo.findText(magic_mode) >= 0:
        self.settings_panel.magic_mode_combo.setCurrentText(magic_mode)

    self.settings_panel.edge_protect_check.setChecked(edge_protect)
    self.settings_panel.brush_size_slider.setValue(max(1, min(200, bru
    self.settings_panel.softness_slider.setValue(max(0, min(100, softn
    self.settings_panel.opacity_slider.setValue(max(1, min(100, opacit
    self.settings_panel.spacing_slider.setValue(max(1, min(100, spacin
    self.settings_panel.tolerance_slider.setValue(max(0, min(255, tole

    for control in controls:
        control.blockSignals(False)

    self.settings_panel.update_tool_visibility()
```

3) In `_persist_editor_settings()`

Replace this:

Python

Run

```
def _persist_editor_settings(self):
    self.settings_store.setValue("editor/tool_name", self.settings_pan
    self.settings_store.setValue("editor/apply_mode", self.settings_pa
    self.settings_store.setValue("editor/magic_mode", self.settings_pa
    self.settings_store.setValue("editor/brush_size", self.settings_pa
    self.settings_store.setValue("editor/softness", self.settings_pane
    self.settings_store.setValue("editor/opacity", self.settings_panel
    self.settings_store.setValue("editor/spacing", self.settings_panel
    self.settings_store.setValue("editor/tolerance", self.settings_pan
```

With this:

Python

Run

```
def _persist_editor_settings(self):
    self.settings_store.setValue("editor/tool_name", self.settings_pan
    self.settings_store.setValue("editor/apply_mode", self.settings_pa
    self.settings_store.setValue("editor/magic_mode", self.settings_pa
    self.settings_store.setValue("editor/edge_protect", self.settings_
    self.settings_store.setValue("editor/brush_size", self.settings_pa
    self.settings_store.setValue("editor/softness", self.settings_pane
    self.settings_store.setValue("editor/opacity", self.settings_panel
    self.settings_store.setValue("editor/spacing", self.settings_panel
    self.settings_store.setValue("editor/tolerance", self.settings_pan
```

4) In `_sync_preview_tool_settings()`

Replace this:

Python

Run

```
def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(
        tool_name=self.settings_panel.tool_combo.currentText(),
        apply_mode=self.settings_panel.apply_mode_combo.currentText(),
        brush_size=self.settings_panel.brush_size_slider.value(),
        softness=self.settings_panel.softness_slider.value(),
        opacity=self.settings_panel.opacity_slider.value(),
        spacing=self.settings_panel.spacing_slider.value(),
        tolerance=self.settings_panel.tolerance_slider.value(),
    )
    self.preview_canvas.set_magic_mode(
        self.settings_panel.magic_mode_combo.currentText()
    )
```

With this:

Python

Run

```
def _sync_preview_tool_settings(self):
    self.preview_canvas.set_tool_settings(
        tool_name=self.settings_panel.tool_combo.currentText(),
        apply_mode=self.settings_panel.apply_mode_combo.currentText(),
        brush_size=self.settings_panel.brush_size_slider.value(),
        softness=self.settings_panel.softness_slider.value(),
        opacity=self.settings_panel.opacity_slider.value(),
        spacing=self.settings_panel.spacing_slider.value(),
        tolerance=self.settings_panel.tolerance_slider.value(),
    )
    self.preview_canvas.set_magic_mode(
        self.settings_panel.magic_mode_combo.currentText()
    )
    self.preview_canvas.set_edge_protect_enabled(
        self.settings_panel.edge_protect_check.isChecked()
    )
```

What to test

Edge Protect visibility

- choose Magic Erase
- confirm Edge Protect appears
- choose Background Erase
- confirm Edge Protect appears

- choose Erase or Restore
- confirm it hides

Magic Erase

- use Magic Erase with Edge Protect on
- click near subject edges
- confirm it is less harsh
- turn Edge Protect off
- try again
- confirm it becomes more aggressive

Background Erase

- use Background Erase with Edge Protect on
- test near boundaries
- confirm it is safer
- turn it off and compare

Persistence

- toggle Edge Protect
- close and reopen app
- confirm it restores correctly

The next strongest polish step is a tolerance pass:

- better default tolerance per click tool
- maybe a small Tolerance Preset idea later
- possibly a "protect semi-transparent edges" refinement

